

Interview Prep

Print Edition · 6-up Landscape

Part 1 — DSA Reference
Part 2 — System Design Q&A;
Part 3 — Behavioral Q&A;

DSA Reference

73 cards · Templates · Tutorials · Code Snippets

Binary Search

DSA Templates · 2 cards

Upper Middle (even → take upper)

For even-length arrays, takes the upper middle element.

• C++

```
int search(vector<int>& nums, int target) {
    int n = nums.size(), l = 0, r = n - 1;
    while (l < r) {
        // for even number of elements, take the upper one
        int m = l + (r - l + 1) / 2;
        if (target < nums[m]) r = m - 1;
        else l = m;
    }
    return nums[l] == target ? l : -1;
}
```

• Java

```
int search(int[] nums, int target) {
    int n = nums.length, l = 0, r = n - 1;
    while (l < r) {
        // for even number of elements, take the upper one
        int m = l + (r - l + 1) / 2;
        if (target < nums[m]) r = m - 1;
        else l = m;
    }
    return nums[l] == target ? l : -1;
}
```

Lower Middle (even → take lower)

For even-length arrays, takes the lower middle element.

• C++

```
int search(vector<int>& nums, int target) {
    int n = nums.size(), l = 0, r = n - 1;
    while (l < r) {
        // for even number of elements, take the lower one
        int m = l + (r - l) / 2;
        if (target > nums[m]) l = m + 1;
        else r = m;
    }
    return nums[l] == target ? l : -1;
}
```

• Java

```
int search(int[] nums, int target) {
    int n = nums.length, l = 0, r = n - 1;
    while (l < r) {
        // for even number of elements, take the lower one
        int m = l + (r - l) / 2;
        if (target > nums[m]) l = m + 1;
        else r = m;
    }
    return nums[l] == target ? l : -1;
}
```

Bit Manipulation

DSA Templates · 5 cards

DSA — Bit Manipulation

Sum of Submasks (SOS DP)

- C++

```
template<typename T_out, typename T_in>
vector<T_out> submask_sums(int n, const vector<T_in> &values) {
    assert(int(values.size()) == 1 << n);
    vector<T_out> dp(values.begin(), values.end());
    for (int i = 0; i < n; i++)
        for (int base = 0; base < 1 << n; base += 1 << (i + 1))
            for (int mask = base; mask < base + (1 << i); mask++)
                dp[mask + (1 << i)] += dp[mask];
    return dp;
}
```

DSA — Bit Manipulationp.7

DSA — Bit Manipulation

Sum of Supermasks

- C++

```
template<typename T_out, typename T_in>
vector<T_out> supermask_sums(int n, vector<T_in> values) {
    reverse(values.begin(), values.end());
    vector<T_out> result = submask_sums<T_out>(n, values);
    reverse(result.begin(), result.end());
    return result;
}
```

DSA — Bit Manipulationp.8

DSA — Bit Manipulation

Largest Bit Set

- C++

```
int largest_bit(int x) {
    return x == 0 ? -1 : 31 - __builtin_clz(x);
}
```

DSA — Bit Manipulationp.9

DSA — Bit Manipulation

Lowest Bit Set

- C++

```
int lowest_bit(int x) {
    return x & (-x);
}

// Index of lowest bit (1-based)
int lowest_bit_index(int x) {
    return __builtin_ffs(x);
}
```

DSA — Bit Manipulationp.10

DSA — Bit Manipulation

Enumerate All Submasks of a Mask

- C++

```
for (int s = m; s; s = (s - 1) & m) {
    // use s
}
```

DSA — Bit Manipulationp.11

DSA — Euler Path

Euler Path

DSA Templates · 1 card

DSA — Bit Manipulationp.12

DSA — Euler Path

Hierholzer's Algorithm

Finds Euler path/circuit in a directed graph. Used in Reconstruct Itinerary (LC 332).

• C++

```
void euler(unordered_map<string, queue<string>>& g,
          string src, vector<string>& ans) {
    while (!g[src].empty()) {
        string nxt = g[src].front();
        g[src].pop();
        euler(g, nxt, ans);
    }
    ans.push_back(src);
}

// Usage (LC 332 - Reconstruct Itinerary)
vector<string> findItinerary(vector<vector<string>>& tickets) {
    vector<string> ans;
    sort(tickets.begin(), tickets.end(),
        [](const vector<string>& x, const vector<string>& y) {
            return x[1] < y[1];
        });
    unordered_map<string, queue<string>> g;
    for (auto x : tickets) g[x[0]].push(x[1]);
    euler(g, "JFK", ans);
    reverse(ans.begin(), ans.end());
    return ans;
}
```

DSA — Fenwick Tree (BIT)

Fenwick Tree (BIT)

DSA Templates · 1 card

DSA — Fenwick Tree (BIT)

Binary Indexed Tree — 1-indexed

Supports point update and range sum query in $O(\log n)$. Also supports range update via difference array trick.

• C++

```
template <class T>
struct BIT { // 1-indexed
    int n;
    vector<T> t;

    BIT() {}
    BIT(int _n) { n = _n; t.assign(n + 1, 0); }

    T query(int i) {
        T ans = 0;
        for (; i >= 1; i -= (i & -i)) ans += t[i];
        return ans;
    }

    void upd(int i, T val) {
        if (i <= 0) return;
        for (; i <= n; i += (i & -i)) t[i] += val;
    }

    // Range update [l, r]
    void upd(int l, int r, T val) {
        upd(l, val);
    }
};
```

```
        upd(r + 1, -val);
    }

    // Range query [l, r]
    T query(int l, int r) {
        return query(r) - query(l - 1);
    }
};
// BIT<int> bit(n);
```

DSA — Graph Theory

Graph Theory

DSA Templates · 9 cards

DSA — Graph Theory

Preorder Traversal

Visit root → left → right.

• C++

```
void preorder(TreeNode* node) {
    if (node == NULL) return;
    // process node->val here
    preorder(node->left);
    preorder(node->right);
}
```

• Python

```
def preorder(node):
    if node is None: return
    # process node.val here
    preorder(node.left)
    preorder(node.right)
```

Inorder Traversal

Visit left $\rightarrow$ root $\rightarrow$ right. Gives sorted order for BSTs.

• C++

```
void inorder(TreeNode* node) {
    if (node == NULL) return;
    inorder(node->left);
    // process node->val here
    inorder(node->right);
}
```

• Python

```
def inorder(node):
    if node is None: return
    inorder(node.left)
    # process node.val here
    inorder(node.right)
```

Postorder Traversal

Visit left $\rightarrow$ right $\rightarrow$ root.

• C++

```
void postorder(TreeNode* node) {
    if (node == NULL) return;
    postorder(node->left);
    postorder(node->right);
    // process node->val here
}
```

• Python

```
def postorder(node):
    if node is None: return
    postorder(node.left)
    postorder(node.right)
    # process node.val here
```

BFS (Breadth First Search)

Explore all closest nodes first before going deeper.

• Python

```
def bfs(root, targetValue):
    if root is None: return None
    currentLevel = [root]
    while len(currentLevel) > 0:
        nextLevel = []
        for node in currentLevel:
            if node is None: continue
            if node.val == targetValue: return node
            nextLevel.append(node.left)
            nextLevel.append(node.right)
        currentLevel = nextLevel
    return None
```

Bellman-Ford Algorithm

Shortest paths from single source. Handles negative weights.

• C++

```
template<typename T_a3, typename T_vector>
void bellman_ford(T_a3 &g, T_vector &dist, int src, int mx_edges) {
    dist[src] = 0;
    for (int i = 0; i <= mx_edges; i++) {
        T_vector ndist = dist;
        for (auto x : g) {
            auto [from, to, cost] = x;
            ndist[to] = min(ndist[to], dist[from] + cost);
        }
        dist = ndist;
    }
}
```

Dijkstra's Algorithm

Shortest paths from single source. Non-negative weights only.

• C++

```
template<typename T_pair, typename T_vector>
void dijkstra(T_pair &g, T_vector &dist, int start) {
    priority_queue<pair<int,int>, vector<pair<int,int>>,
        greater<pair<int,int>>> pq;
    dist[start] = 0;
    pq.push({start, 0});
    while (!pq.empty()) {
        auto [u_node, u_cost] = pq.top(); pq.pop();
        if (u_cost > dist[u_node]) continue;
        for (auto [v_node, v_cost] : g[u_node]) {
            if (dist[v_node] > dist[u_node] + v_cost) {
                dist[v_node] = dist[u_node] + v_cost;
                pq.push({v_node, dist[v_node]});
            }
        }
    }
}
```

Topological Sort (BFS)

Linear ordering of vertices in a DAG.

• C++

```
struct TopologicalSort {
    int n, steps = 0, nodes = 0;
    vector<int> indegree, orders;
    vector<vector<int>> G;
    bool isTopologicalSorted = false;

    TopologicalSort(vector<vector<int>>& g, vector<int>& in) {
        G = g; n = G.size(); indegree = in;
        queue<int> q;
        for (int i = 0; i < n; i++)
            if (indegree[i] == 0) q.push(i);
        while (!q.empty()) {
            int sz = q.size();
            steps++; nodes += sz;
            for (int i = 0; i < sz; i++) {
                auto u = q.front(); q.pop();
                orders.push_back(u);
                for (auto v : G[u])
                    if (--indegree[v] == 0) q.push(v);
            }
        }
        isTopologicalSorted = nodes == n;
    }
};
```

Kahn's Algorithm

Alternative topological sort. Returns empty if cycle detected.

• C++

```
template<typename T_vector, typename T_vector_vector>
T_vector kahn(int n, T_vector_vector &edges) {
    vector<int> ordering, indegree(n, 0);
    vector<vector<int>> g(n);
    for (auto e : edges) {
        --e[0]; --e[1];
        indegree[e[1]]++;
        g[e[0]].push_back(e[1]);
    }
    queue<int> q;
    for (int i = 0; i < n; i++)
        if (indegree[i] == 0) q.push(i);
    int visited = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        ordering.push_back(u);
        visited++;
        for (auto v : g[u])
            if (--indegree[v] == 0) q.push(v);
    }
    if (visited != n) return T_vector{};
    reverse(ordering.begin(), ordering.end());

    return ordering;
}
```

Disjoint Set Union (DSU / Union-Find)

Combines sets and checks connectivity in near O(1) per operation.

• C++

```
class dsu {
public:
    vector<int> root, rank;
    int n, cnt;

    dsu(int _n) : n(_n) {
        root.resize(n); rank.resize(n);
        for (int i = 0; i < n; i++) { root[i] = i; rank[i] = 1; }
        cnt = n;
    }

    int getCount() { return cnt; }

    int get(int x) {
        return x == root[x] ? x : (root[x] = get(root[x]));
    }

    bool unite(int x, int y) {
        x = get(x); y = get(y);
        if (x == y) return false;
        if (rank[x] > rank[y]) root[y] = x;
        else if (rank[x] < rank[y]) root[x] = y;

        else { root[y] = x; rank[x]++; }
        cnt--;
        return true;
    }
};
```

Linked List

DSA Templates · 1 card

Floyd's Fast & Slow Pointer

Detects cycles and finds midpoints in linked lists.

• C++

```
ListNode* slow = head;
ListNode* fast = head;
while (fast != nullptr && fast->next != nullptr) {
    // do something here
    slow = slow->next;
    fast = fast->next->next;
}
```

• Java

```
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    // do something here
    slow = slow.next;
    fast = fast.next.next;
}
```

Manacher's Algorithm

DSA Templates · 1 card

Palindromic Substrings — All Centers

Finds all palindromic substrings in O(n). Converts even/odd cases by inserting # separators.

• Python

```
def palindromicSubstrings(s: str) -> list[str]:
    if not s: return []
    # Insert # to handle even-length palindromes
    string = "#".join(s)
    pLengths = [0] * len(string)
    c = R = 0
    for i in range(len(string)):
        if i < R:
            pLengths[i] = min(R - i, pLengths[2 * c - i])
            while (i - pLengths[i] - 1 >= 0 and
                    i + pLengths[i] + 1 < len(string) and
                    string[i + pLengths[i] + 1] == string[i - pLengths[i] -
1]):
                pLengths[i] += 1
            if pLengths[i] + i > R:
                c, R = i, i + pLengths[i]
        result = []
        for i, p in enumerate(pLengths):
            s = string[i - p:i + p + 1].replace("#", "")
            if s: result.append(s)
        return result
```

• C++

```
vector<string> palindromicSubstrings(string s) {
    if (s.empty()) return {};
    string str = "";
    for (int i = 0; i < (int)s.length() - 1; i++) {
        str += s[i]; str += "#";
    }
    str += s.back();
    int len = str.length();
    vector<int> pL(len, 0);
    int c = 0, R = 0;
    for (int i = 0; i < len; i++) {
        if (i < R) pL[i] = min(R - i, pL[2 * c - i]);
        while (i - pL[i] - 1 >= 0 && i + pL[i] + 1 < len &&
            str[i + pL[i] + 1] == str[i - pL[i] - 1])
            pL[i]++;
        if (pL[i] + i > R) { c = i; R = i + pL[i]; }
    }
    vector<string> res;
    for (int i = 0; i < len; i++) {
        string p = str.substr(i - pL[i], 2 * pL[i] + 1);
        string r = "";
        for (char ch : p) if (ch != '#') r += ch;
        res.push_back(r);
    }
    return res;
}
```

DSA — Ordered Set (PBDS)

Ordered Set (PBDS)

DSA Templates · 1 card

DSA — Ordered Set (PBDS)

GNU C++ Policy-Based Ordered Set

O(log n) rank queries: order_of_key(k) = # elements < k, find_by_order(k) = k-th element.

- C++

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

// less<int> → ascending order set
// greater<int> → descending order set
// less_equal<int> → allows duplicates (multiset)
tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> oset;

void usage() {
    oset.insert(5);
    // # elements strictly less than k
    int rank = oset.order_of_key(5);
    // k-th element (0-indexed)
    auto it = oset.find_by_order(0);
}
```

DSA — Prefix & Suffix Sum

Prefix & Suffix Sum

DSA Templates · 4 cards

DSA — Prefix & Suffix Sum

Prefix Sum (starts at index 0)

pref[i] = sum of a[0..i]

- C++

```
vector<int> pref(n);
pref[0] = a[0];
for (int i = 1; i < n; i++)
    pref[i] = pref[i - 1] + a[i];

// Query sum [l, r]:
// l == 0 ? pref[r] : pref[r] - pref[l - 1]
```

DSA — Prefix & Suffix Sum

Prefix Sum (1-indexed, starts at 0)

pref[i+1] = sum of a[0..i]. Clean range queries.

- C++

```
vector<int> pref(n + 1, 0);
for (int i = 0; i < n; i++)
    pref[i + 1] = pref[i] + a[i];

// Query sum [l, r] (0-indexed):
// pref[r + 1] - pref[l]
```

Suffix Sum (starts at last index)

suff[i] = sum of a[i..n-1]

• C++

vector<int> suff(n);
suff[n - 1] = a[n - 1];
for (int i = n - 2; i >= 0; i--)
 suff[i] = suff[i + 1] + a[i];

Suffix Sum (1-indexed, starts at 0)

suff[i] = sum of a[i..n-1] with extra 0 at end.

• C++

vector<int> suff(n + 1, 0);
for (int i = n - 1; i >= 0; i--)
 suff[i] = suff[i + 1] + a[i];

// Query sum [l, r] (0-indexed):
// suff[l] - suff[r + 1]

Segment Tree

DSA Templates · 1 card

Segment Tree — Range Sum + Point Update

O(log n) point update and range sum query.

• C++

struct segtree {
 vector<long long> sums;
 int size;

 void init(int n) {
 size = 1;
 while (size < n) size *= 2;
 sums.assign(size * 2, 0LL);
 }

 void set(int i, int v, int x, int lx, int rx) {
 if (rx - lx == 1) { sums[x] = v; return; }
 int m = (lx + rx) / 2;
 if (i < m) set(i, v, 2*x+1, lx, m);
 else set(i, v, 2*x+2, m, rx);
 sums[x] = sums[2*x+1] + sums[2*x+2];
 }
 void set(int i, int v) { set(i, v, 0, 0, size); }

 long long sum(int l, int r, int x, int lx, int rx) {
 if (lx >= r || l >= rx) return 0;
 if (lx >= l && rx <= r) return sums[x];
 }
};

int m = (lx + rx) / 2;
return sum(l, r, 2*x+1, lx, m) + sum(l, r, 2*x+2, m, rx);
}
long long sum(int l, int r) { return sum(l, r, 0, 0, size); }
} st;

// Usage:
// st.init(n);
// st.set(i, val); // point update
// st.sum(l, r + 1); // range sum [l, r] (exclusive r)

Sparse Table

DSA Templates · 1 card

DSA — Sparse Table

Sparse Table — O(1) Range Queries

Static arrays only (no updates). Ideal for range min/max/gcd in O(1) after O(n log n) build.

- C++

```
template<typename it, typename bin_op>
struct sparse_table {
    using T = typename remove_reference<decltype(*declval<it>())>::type;
    vector<vector<T>> t;
    bin_op f;

    sparse_table(it first, it last, bin_op op) : t(1), f(op) {
        int n = distance(first, last);
        t.assign(32 - __builtin_clz(n), vector<T>(n));
        t[0].assign(first, last);
        for (int i = 1; i < (int)t.size(); i++)
            for (int j = 0; j < n - (1 << i) + 1; j++)
                t[i][j] = f(t[i-1][j], t[i-1][j + (1 << (i-1))]);
    }

    // O(1) range query f(a[l..r])
    T query(int l, int r) {
        int h = floor(log2(r - l + 1));
        return f(t[h][l], t[h][r - (1 << h) + 1]);
    }
};

// Usage - range GCD:
// sparse_table g(a.begin(), a.end(), [](int x, int y){ return gcd(x, y); });
// g.query(l, r);
```

DSA — Sparse Tablep.43

DSA — Two Pointers

Two Pointers

DSA Templates · 1 card

DSA — Sparse Tablep.44

DSA — Two Pointers

Two Pointers — Converging

l and r start at opposite ends and move toward each other.

- C++

```
int f(vector<int>& v) {
    int ans = 0;
    int l = 0, r = (int)v.size() - 1;
    while (l < r) {
        // logic here
        if (/* condition */) l++;
        else r--;
    }
    return ans;
}
```

- Java

```
int f(int[] v) {
    int ans = 0;
    int l = 0, r = v.length - 1;
    while (l < r) {
        // logic here
        if (/* condition */) l++;
        else r--;
    }
    return ans;
}
```

DSA — Two Pointersp.45

Tutorial — Arrays

Basic Topics — Arrays

Tutorials · 1 card

DSA — Two Pointersp.46

Basic Topics — Arrays

Array Operations & Complexity

Arrays store collections in contiguous memory. Support O(1) access by index but O(n) insertion/deletion at arbitrary positions.

Complexity: Access O(1) · Insert/Delete middle O(n) · Append O(1) amortized

- C++

```
// Sort ascending
sort(v.begin(), v.end());
// Sort descending
sort(v.begin(), v.end(), greater<int>());
// Custom comparator
sort(v.begin(), v.end(), [](const auto& a, const auto& b) {
    return a[0] != b[0] ? a[0] < b[0] : a[1] < b[1];
});
// Max / min / sum
int mx = *max_element(v.begin(), v.end());
int mn = *min_element(v.begin(), v.end());
int s = accumulate(v.begin(), v.end(), 0);
```

- Python

```
# Sort ascending / descending
arr.sort()
arr.sort(reverse=True)
arr.sort(key=lambda x: x[1]) # by second element

# Max / min / sum
mx = max(arr)
mn = min(arr)
s = sum(arr)
```

Tutorial — Arraysp.47

Tutorial — Backtracking

Basic Topics — Backtracking

Tutorials · 1 card

Tutorial — Arraysp.48

Tutorials · 1 card

- Java

```
int binarySearch(int[] nums, int target) {
    int lp = 0, rp = nums.length - 1;
    while (lp <= rp) {
        int mid = lp + (rp - lp) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) lp = mid + 1;
        else rp = mid - 1;
    }
    return -1;
}
```

Tutorials · 1 card

Brute Force — Nested Loops

Try every possible combination without optimization. Use as a baseline before optimizing. $O(n^2)$ or worse.
Complexity: Time $O(n^2)$ typical · Space $O(1)$

- C++

```
// Count pairs with difference == k
int countPairs(vector<int>& nums, int k) {
    int count = 0;
    for (int i = 0; i < nums.size(); i++)
        for (int j = i + 1; j < nums.size(); j++)
            if (abs(nums[i] - nums[j]) == k)
                count++;
    return count;
}

// Running sum (brute force  $O(n^2)$  - optimize with prefix sum)
vector<int> runningSum(vector<int>& nums) {
    int n = nums.size();
    vector<int> ans(n);
    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = 0; j <= i; j++) sum += nums[j];
        ans[i] = sum;
    }
    return ans;
}
```

Tutorial — Greedy

Basic Topics — Greedy

Tutorials · 1 card

Greedy — Locally Optimal Choices

Makes the best immediate choice at each step hoping to reach the global optimum. Simple but not always optimal.
Complexity: Usually $O(n \log n)$ with sorting · Space $O(1)$

- C++

```
// LC 455: Assign Cookies
// Strategy: match smallest sufficient cookie to least greedy child
int findContentChildren(vector<int>& g, vector<int>& s) {
    sort(s.begin(), s.end());
    sort(g.begin(), g.end());
    int ans = 0;
    for (int j = 0; j < s.size() && ans < g.size(); j++)
        if (g[ans] <= s[j]) ans++;
    return ans;
}

// LC 605: Can Place Flowers
bool canPlaceFlowers(vector<int>& bed, int n) {
    int count = 0, sz = bed.size();
    for (int i = 0; i < sz && count < n; i++) {
        if (bed[i] == 0 &&
            (i == 0 || bed[i-1] == 0) &&
            (i == sz-1 || bed[i+1] == 0)) {
            bed[i] = 1;
            count++;
        }
    }

    return count >= n;
}
```

Basic Topics — Hash Map

Tutorials · 1 card

Hash Map — $O(1)$ Lookup

Maps keys to values using a hash function. Enables $O(1)$ average insert/lookup vs $O(n)$ linear search.
Complexity: Time $O(1)$ avg · Space $O(n)$

- C++

```
// LC 1: Two Sum -  $O(n)$  with hash map
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> mp; // value -> index
    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (mp.count(complement))
            return {mp[complement], i};
        mp[nums[i]] = i;
    }
    return {};
}

// Frequency count
unordered_map<int, int> freq;
for (int x : nums) freq[x]++;
// Check if exists
if (freq.count(key)) { /* exists */ }
// Iterate
for (auto& [k, v] : freq) { /* use k, v */ }
```

- Python

```
# Two Sum
def twoSum(nums, target):
    seen = {} # value -> index
    for i, num in enumerate(nums):
        if target - num in seen:
            return [seen[target - num], i]
    seen[num] = i
    return []

# Frequency count
from collections import Counter
freq = Counter(nums)
freq['a'] # 0 if not present (defaultdict behavior)
```

Basic Topics — Heap / Priority Queue

Tutorials · 1 card

Basic Topics — Heap / Priority Queue

Min/Max Heap Operations

Efficiently retrieves minimum (or maximum) element in $O(\log n)$. Python `heapq` is min-heap; negate for max-heap. C++ `priority_queue` is max-heap by default.

Complexity: Push/Pop $O(\log n)$ · Peek $O(1)$

• Python

```
import heapq

# Min-heap
heap = []
heapq.heappush(heap, 3)
heapq.heappush(heap, 1)
smallest = heapq.heappop(heap) # 1

# Max-heap (negate values)
max_heap = []
heapq.heappush(max_heap, -10)
heapq.heappush(max_heap, -7)
largest = -heapq.heappop(max_heap) # 10

# Heapify in-place  $O(n)$ 
heapq.heapify(arr)
```

• C++

```
// Max-heap (default)
priority_queue<int> max_pq;
max_pq.push(3);
int top = max_pq.top(); // 3
max_pq.pop();

// Min-heap
priority_queue<int, vector<int>, greater<int>> min_pq;

// Min-heap of pairs {cost, node}
priority_queue<pair<int,int>,
               vector<pair<int,int>>,
               greater<>> pq;

pq.push({cost, node});
auto [c, n] = pq.top(); pq.pop();
```

Basic Topics — Kadane's Algorithm

Tutorials · 1 card

Basic Topics — Kadane's Algorithm

Maximum Subarray Sum

Finds the maximum sum of a contiguous subarray in $O(n)$. Resets the running sum to 0 whenever it goes negative.

Complexity: Time $O(n)$ · Space $O(1)$

• C++

```
// LC 53: Maximum Subarray
int maxSubArray(vector<int>& nums) {
    int globalSum = INT_MIN, localSum = 0;
    for (int x : nums) {
        localSum += x;
        globalSum = max(globalSum, localSum);
        if (localSum < 0) localSum = 0;
    }
    return globalSum;
}
```

• Python

```
def maxSubArray(nums):
    global_sum = float('-inf')
    local_sum = 0
    for x in nums:
        local_sum += x
        global_sum = max(global_sum, local_sum)
        if local_sum < 0:
            local_sum = 0
    return global_sum
```

Basic Topics — Linear Search

Tutorials · 1 card

Linear Search

Traverses every element checking against the target. Simplest search — no sorting required. $O(n)$ worst case.

Complexity: Best $O(1)$ · Average/Worst $O(n)$

• C++

```
// Generic linear search
int linearSearch(vector<int>& nums, int target) {
    for (int i = 0; i < nums.size(); i++)
        if (nums[i] == target) return i;
    return -1;
}
```

```
// LC 1295: Find Numbers with Even Digits
int findNumbers(vector<int>& nums) {
    int ans = 0;
    for (int x : nums) {
        int len = log10(x) + 1;
        if (len % 2 == 0) ans++;
    }
    return ans;
}
```

Basic Topics — Linked List

Tutorials · 1 card

Linked List — Insert / Delete / Search

Nodes connected by next pointers. $O(1)$ insert/delete at head but $O(n)$ to access arbitrary elements.

Complexity: Lookup $O(n)$ · Insert/Delete head $O(1)$ · Insert/Delete tail $O(n)$

• C++

```
struct ListNode {
    int val; ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};
```

```
// Insert at head
ListNode* insertHead(ListNode* head, int val) {
    ListNode* node = new ListNode(val);
    node->next = head;
    return node;
}
```

```
// Delete head
ListNode* deleteHead(ListNode* head) {
    if (!head) return nullptr;
    ListNode* tmp = head;
    head = head->next;
    delete tmp;
    return head;
}
```

```
// Floyd's cycle detection / find mid
```

```
ListNode* findMid(ListNode* head) {
    ListNode* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

Basic Topics — MOD (1e9 + 7)

Tutorials · 1 card

Modular Arithmetic

Apply % at each step to prevent overflow. $1e9+7$ is prime, enabling modular inverse via Fermat's Little Theorem.

Complexity: $O(1)$ per operation · $O(\log n)$ for modular inverse

• C++

```
const int MOD = 1e9 + 7;
```

```
// Correct: apply mod each step
long long factorial(int n) {
    long long fact = 1;
    for (int i = 2; i <= n; i++)
        fact = (fact * i) % MOD;
    return fact;
}
```

```
// Modular properties:
// (a + b) % m = (a%m + b%m) % m
// (a * b) % m = (a%m * b%m) % m
// (a - b) % m = (a%m - b%m + m) % m ← +m to avoid negatives
```

```
// Modular inverse (m must be prime): a^(m-2) mod m
long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = res * base % mod;
        base = base * base % mod;
    }
}
```

```
        exp >= 1;
    }
    return res;
}
long long modInverse(long long a) { return power(a, MOD - 2, MOD); }
```

Basic Topics — Prefix Sum

Tutorials · 1 card

Prefix Sum — O(1) Range Queries

Pre-computes cumulative sums to answer range sum queries in O(1) after O(n) build.

Complexity: Build O(n) · Query O(1) · Space O(n)

• C++

```
// Build prefix sum (0-indexed, padded)
vector<int> buildPrefix(vector<int>& a) {
    int n = a.size();
    vector<int> pref(n + 1, 0);
    for (int i = 0; i < n; i++)
        pref[i + 1] = pref[i] + a[i];
    return pref;
}

// Range sum [l, r] (0-indexed):
// pref[r + 1] - pref[l]

// LC 303: Range Sum Query
class NumArray {
public:
    vector<int> pref;
    NumArray(vector<int>& nums) {
        pref.resize(nums.size() + 1, 0);
        for (int i = 0; i < nums.size(); i++)
            pref[i+1] = pref[i] + nums[i];
    }
    int sumRange(int l, int r) { return pref[r+1] - pref[l]; }
};
```

Basic Topics — Queue & Stack

Tutorials · 2 cards

Stack — LIFO

Last In First Out. Use for matching brackets, monotonic problems, undo operations.

Complexity: Push/Pop O(1)

• Python

```
# LC 20: Valid Parentheses
def isValid(s: str) -> bool:
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}' }
    for c in s:
        if c in "([{":
            stack.append(c)
        elif not stack or stack[-1] != pairs[c]:
            return False
        else:
            stack.pop()
    return len(stack) == 0
```

• C++

```
// Stack operations
stack<int> st;
st.push(1); st.push(2);
int top = st.top(); st.pop();
bool empty = st.empty();
```

Queue — FIFO

First In First Out. Use for BFS, level-order traversal, sliding window problems.

Complexity: Push/Pop O(1)

• Python

```
from collections import deque
queue = deque()
queue.appendleft(1) # enqueue front
queue.appendleft(2)
val = queue.pop() # dequeue back
```

• C++

```
// Queue operations
queue<int> q;
q.push(1); q.push(2);
int front = q.front(); q.pop();
bool empty = q.empty();

// Deque (double-ended)
deque<int> dq;
dq.push_front(1); dq.push_back(2);
dq.pop_front(); dq.pop_back();
```

Basic Topics — Sliding Window

Tutorials · 2 cards

Basic Topics — Sliding Window

Sliding Window — Fixed Size

Maintains a window of fixed size k, sliding one element at a time. Avoids recomputing from scratch each step.

Complexity: Time $O(n)$ · Space $O(1)$

• C++

```
// Maximum sum of subarray of size k
int maxSumSubarray(vector<int>& nums, int k) {
    int windowSum = 0, maxSum = 0;
    for (int i = 0; i < k; i++) windowSum += nums[i];
    maxSum = windowSum;
    for (int i = k; i < nums.size(); i++) {
        windowSum += nums[i] - nums[i - k];
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;
}
```

Basic Topics — Sliding Window

Sliding Window — Variable Size

Expands right pointer and shrinks left pointer based on a condition. Common for longest/shortest subarray problems.

Complexity: Time $O(n)$ · Space $O(k)$ with hash map

• C++

```
// Longest substring without repeating characters
int lengthOfLongestSubstring(string s) {
    unordered_map<char, int> last;
    int ans = 0, l = 0;
    for (int r = 0; r < s.size(); r++) {
        if (last.count(s[r]) && last[s[r]] >= l)
            l = last[s[r]] + 1;
        last[s[r]] = r;
        ans = max(ans, r - l + 1);
    }
    return ans;
}
```

• Python

```
def lengthOfLongestSubstring(s: str) -> int:
    last = {}
    ans = l = 0
    for r, c in enumerate(s):
        if c in last and last[c] >= l:
            l = last[c] + 1
        last[c] = r
        ans = max(ans, r - l + 1)
    return ans
```

Basic Topics — Sorting

Tutorials · 4 cards

Basic Topics — Sorting

Built-in Sort — Usage & Comparators

C++ std::sort is introsort ($O(n \log n)$). Python sort is Timsort (stable). Custom comparators allow multi-key sorting.

Complexity: Time $O(n \log n)$ · Space $O(\log n)$

• C++

```
// Sort ascending
sort(v.begin(), v.end());
// Descending
sort(v.begin(), v.end(), greater<int>());
// Custom: multi-key
sort(v.begin(), v.end(), [](const array<int,3>& x, const array<int,3>& y){
    return x[0] != y[0] ? x[0] < y[0] :
           x[1] != y[1] ? x[1] < y[1] : x[2] < y[2];
});
```

Basic Topics — Sorting

Bubble Sort

Repeatedly swaps adjacent out-of-order elements. Early termination with a flag. Simple but $O(n^2)$.

Complexity: Time $O(n^2)$ · Space $O(1)$

• C++

```
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool swapped = true;
    for (int i = 0; i < n - 1 && swapped; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                swapped = true;
            }
        }
    }
}
```

Merge Sort

Divide array in half, recursively sort each half, merge. Stable, $O(n \log n)$ all cases.

Complexity: Time $O(n \log n)$ all cases · Space $O(n)$

• C++

```
void merge(vector<int>& a, int l, int m, int r) {
    vector<int> L(a.begin()+l, a.begin()+m+1);
    vector<int> R(a.begin()+m+1, a.begin()+r+1);
    int i=0, j=0, k=l;
    while (i<L.size() && j<R.size())
        a[k++] = (L[i]<=R[j]) ? L[i++] : R[j++];
    while (i<L.size()) a[k++]=L[i++];
    while (j<R.size()) a[k++]=R[j++];
}

void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r-l)/2;
    mergeSort(a, l, m);
    mergeSort(a, m+1, r);
    merge(a, l, m, r);
}
```

Tim Sort

Hybrid of insertion sort + merge sort. Used in Python sort() and Java Arrays.sort(). Splits into runs of size 32, sorts each with insertion sort, then merges.

Complexity: Time $O(n \log n)$ · Space $O(1)$

• C++

```
const int RUN = 32;

void insertionSort(vector<int>& nums, int l, int r) {
    for (int i = l+1; i <= r; i++) {
        int tmp = nums[i], j = i-1;
        while (j >= l && nums[j] > tmp) { nums[j+1]=nums[j]; j--; }
        nums[j+1] = tmp;
    }
}

void timMerge(vector<int>& nums, int l, int m, int r) {
    vector<int> L(nums.begin()+l, nums.begin()+m+1);
    vector<int> R(nums.begin()+m+1, nums.begin()+r+1);
    int i=0, j=0, k=l;
    while (i<L.size()&&j<R.size())
        nums[k++]=(L[i]<=R[j])?L[i++]:R[j++];
    while (i<L.size()) nums[k++]=L[i++];
    while (j<R.size()) nums[k++]=R[j++];
}

void timSort(vector<int>& nums) {
    int n = nums.size();
    for (int i=0; i<n; i+=RUN)
```

```
        insertionSort(nums, i, min(i+RUN-1, n-1));
    for (int sz=RUN; sz<n; sz*=2)
        for (int l=0; l<n; l+=2*sz) {
            int m=l+sz-1, r=min(l+2*sz-1,n-1);
            if (m<r) timMerge(nums, l, m, r);
        }
}
```

Basic Topics — Time Complexity

Tutorials · 1 card

Input Size → Required Complexity

Use input size n to determine which algorithm complexity is fast enough. LeetCode typical time limit is $\sim 10^8$ operations/sec.

• Reference

```
n ≤ 10 →  $O(n!)$ ,  $O(n^7)$ ,  $O(n^6)$ 
n ≤ 20 →  $O(2^n)$ 
n ≤ 80 →  $O(n^4)$ 
n ≤ 400 →  $O(n^3)$ 
n ≤ 7,500 →  $O(n^2)$ 
n ≤  $10^6$  →  $O(n \log n)$ ,  $O(n)$ 
n >  $10^6$  →  $O(\log n)$ ,  $O(1)$ 

// Loop complexities:
O(n) – single loop
O(n*m) – nested loops
O(√n) – loop: i*i <= n
O(log n) – halving each step (binary search)
O(n log n)– sort + single loop
```

Basic Topics — Trie

Tutorials · 1 card

Trie — Insert / Search / StartsWith

Prefix tree for string storage and retrieval. Each node has 26 children (a–z) and an end-of-word flag. O(m) per operation where m = word length.
Complexity: Time O(m) per op · Space O(26 × n × m)

```
• C++

struct TrieNode {
    TrieNode* children[26];
    bool isEnd;
    TrieNode() : isEnd(false) {
        fill(children, children+26, nullptr);
    }
};

class Trie {
    TrieNode* root;
public:
    Trie() { root = new TrieNode(); }

    void insert(string word) {
        TrieNode* cur = root;
        for (char c : word) {
            int i = c - 'a';
            if (!cur->children[i])
                cur->children[i] = new TrieNode();
            cur = cur->children[i];
        }
        cur->isEnd = true;
    }
};
```

```

}

bool search(string word) {
    TrieNode* cur = root;
    for (char c : word) {
        int i = c - 'a';
        if (!cur->children[i]) return false;
        cur = cur->children[i];
    }
    return cur->isEnd;
}

bool startsWith(string prefix) {
    TrieNode* cur = root;
    for (char c : prefix) {
        int i = c - 'a';
        if (!cur->children[i]) return false;
        cur = cur->children[i];
    }
    return true;
}
};
```

Tutorial — Two Pointers

Basic Topics — Two Pointers

Tutorials · 1 card

Two Pointers — Converging

Start one pointer at each end, move inward based on condition. Common for sorted arrays.
Complexity: Time O(n) · Space O(1)

```
• C++

// LC 977: Squares of Sorted Array
vector<int> sortedSquares(vector<int>& nums) {
    int l = 0, r = nums.size() - 1;
    vector<int> res;
    while (l <= r) {
        if (abs(nums[l]) > abs(nums[r])) {
            res.push_back(nums[l]*nums[l]); l++;
        } else {
            res.push_back(nums[r]*nums[r]); r--;
        }
    }
    reverse(res.begin(), res.end());
    return res;
}

• Python

def sortedSquares(nums):
    l, r = 0, len(nums) - 1
    res = []
    while l <= r:
        if abs(nums[l]) > abs(nums[r]):
            res.append(nums[l]**2); l += 1
        else:
            res.append(nums[r]**2); r -= 1
    return res[::-1]
```

Graph Theory — Tree Traversals

Tutorials · 1 card

Preorder / Inorder / Postorder

Three DFS traversal orders. Preorder: Root→L→R. Inorder: L→Root→R (sorted for BST). Postorder: L→R→Root.

```
• C++

void preorder(TreeNode* node) {
    if (!node) return;
    // process node->val
    preorder(node->left);
    preorder(node->right);
}

void inorder(TreeNode* node) {
    if (!node) return;
    inorder(node->left);
    // process node->val
    inorder(node->right);
}

void postorder(TreeNode* node) {
    if (!node) return;
    postorder(node->left);
    postorder(node->right);
    // process node->val
}

• Python
```



```
def preorder(node):
    if not node: return
    # process node.val
    preorder(node.left); preorder(node.right)

def inorder(node):
    if not node: return
    inorder(node.left)
    # process node.val
    inorder(node.right)

def postorder(node):
    if not node: return
    postorder(node.left); postorder(node.right)
    # process node.val
```

Graph Theory — BFS

Tutorials · 1 card

Graph Theory — BFS

Breadth-First Search

Explores all nodes at current depth before going deeper. Uses a queue. Guarantees shortest path in unweighted graphs.
Complexity: Time $O(V+E)$ · Space $O(V)$

- Python

```
def bfs(root, target):
    if not root: return None
    level = [root]
    while level:
        next_level = []
        for node in level:
            if not node: continue
            if node.val == target: return node
            next_level.append(node.left)
            next_level.append(node.right)
        level = next_level
    return None
```

• C++

```
// BFS on graph
vector<int> bfs(vector<vector<int>>& adj, int src, int n) {
    vector<int> dist(n, -1);
    queue<int> q;
    dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}
```

Graph Theory — DFS

Tutorials · 1 card

Graph Theory — DFS

Depth-First Search

Explores one branch completely before backtracking. Uses recursion (implicit stack). Great for path finding, cycle detection, connected components.
Complexity: Time $O(V+E)$ · Space $O(V)$

- Python

```
def dfs(node):
    if not node: return 0
    left = dfs(node.left)
    right = dfs(node.right)
    return max(left, right) + 1 # max depth example
```

- C++

```
// DFS on graph with visited set
void dfs(vector<vector<int>>& adj, vector<bool>& vis, int u) {
    vis[u] = true;
    // process u
    for (int v : adj[u])
        if (!vis[v]) dfs(adj, vis, v);
}
```

Graph Theory — Bellman-Ford

Tutorials · 1 card

Bellman-Ford Algorithm

Shortest paths from single source. Handles negative weights. Runs $|V|-1$ relaxation passes over all edges.

Complexity: Time $O(V \times E)$ · Space $O(V)$

• C++

```
template<typename T_a3, typename T_vector>
void bellman_ford(T_a3& g, T_vector& dist, int src, int mx_edges) {
    dist[src] = 0;
    for (int i = 0; i <= mx_edges; i++) {
        T_vector ndist = dist;
        for (auto x : g) {
            auto [from, to, cost] = x;
            ndist[to] = min(ndist[to], dist[from] + cost);
        }
        dist = ndist;
    }
}
```

• Python

```
def bellman_ford(g, dist, src, mx_edges):
    dist[src] = 0
    for _ in range(mx_edges + 1):
        ndist = dist[:]
        for frm, to, cost in g:
            ndist[to] = min(ndist[to], dist[frm] + cost)
        dist = ndist
    return dist
```

Graph Theory — Dijkstra's

Tutorials · 1 card

Dijkstra's Algorithm

Shortest paths from single source. Non-negative weights only. Uses min-heap priority queue.

Complexity: Time $O((V+E) \log V)$ · Space $O(V)$

• C++

```
template<typename T_pair, typename T_vector>
void dijkstra(T_pair& g, T_vector& dist, int start) {
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<>> pq;
    dist[start] = 0;
    pq.push({0, start});
    while (!pq.empty()) {
        auto [u_cost, u] = pq.top(); pq.pop();
        if (u_cost > dist[u]) continue;
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

Graph Theory — DSU

Tutorials · 1 card

Disjoint Set Union (Union-Find)

Tracks connected components. Two operations: Find (get root) and Union (merge sets). Path compression + union by rank gives near $O(1)$ per op.

Complexity: $O(\alpha(n)) \approx O(1)$ per op with optimizations

• C++

```
class DSU {
public:
    vector<int> root, rank_;
    int cnt;
    DSU(int n) : root(n), rank_(n, 1), cnt(n) {
        iota(root.begin(), root.end(), 0);
    }
    int find(int x) {
        return x == root[x] ? x : root[x] = find(root[x]);
    }
    bool unite(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return false;
        if (rank_[x] < rank_[y]) swap(x, y);
        root[y] = x;
        if (rank_[x] == rank_[y]) rank_[x]++;
        cnt--;
        return true;
    }
    bool connected(int x, int y) { return find(x) == find(y); }
};
```

Graph Theory — Topological Sort

Tutorials · 1 card

Graph Theory — Topological Sort

Topological Sort (Kahn's BFS)

Linear ordering of a DAG where u comes before v for every edge (u,v).
Detects cycles if not all nodes are processed.
Complexity: Time $O(V+E)$ · Space $O(V)$

```
• C++

struct TopologicalSort {
    vector<int> orders;
    bool isSorted = false;
    int steps = 0;

    TopologicalSort(vector<vector<int>>& g, vector<int>& indegree) {
        int n = g.size(), nodes = 0;
        queue<int> q;
        for (int i = 0; i < n; i++)
            if (indegree[i] == 0) q.push(i);
        while (!q.empty()) {
            int sz = q.size(); steps++;
            nodes += sz;
            for (int i = 0; i < sz; i++) {
                int u = q.front(); q.pop();
                orders.push_back(u);
                for (int v : g[u])
                    if (--indegree[v] == 0) q.push(v);
            }
        }
        isSorted = nodes == n;
    }
}
```

```
};

// LC 207: Course Schedule
bool canFinish(int n, vector<vector<int>>& pre) {
    vector<vector<int>> g(n); vector<int> ind(n);
    for (auto& p : pre) { g[p[1]].push_back(p[0]); ind[p[0]]++; }
    TopologicalSort ts(g, ind);
    return ts.isSorted;
}
```

Graph Theory — Binary Search Tree

Tutorials · 1 card

Graph Theory — Binary Search Tree

BST — Insert / Delete / Search

Left subtree < node < right subtree. All ops $O(n)$ worst case (unbalanced), $O(\log n)$ average.
Complexity: Time $O(n)$ worst · $O(\log n)$ balanced

```
• C++

TreeNode* insert(TreeNode* root, int key) {
    if (!root) return new TreeNode(key);
    if (key < root->val) root->left = insert(root->left, key);
    else root->right = insert(root->right, key);
    return root;
}

TreeNode* search(TreeNode* root, int key) {
    if (!root || root->val == key) return root;
    return key < root->val ? search(root->left, key)
        : search(root->right, key);
}

TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) root->left = deleteNode(root->left, key);
    else if (key > root->val) root->right = deleteNode(root->right, key);
    else {
        if (!root->left) return root->right;
        if (!root->right) return root->left;
        TreeNode* succ = root->right;
        while (succ->left) succ = succ->left;
    }
}
```

```
        root->val = succ->val;
        root->right = deleteNode(root->right, succ->val);
    }
    return root;
}
```

Graph Theory — Kruskal's MST

Tutorials · 1 card

Kruskal's Algorithm — Minimum Spanning Tree

Greedy MST: sort edges by weight, add edge if it doesn't form a cycle (using DSU). $O(E \log E)$.

Complexity: Time $O(E \log E)$ · Space $O(V)$

• C++

```
int kruskalMST(int n, vector<array<int,3>>& edges) {
    // edges: {weight, u, v}
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    int total = 0;
    for (auto& [w, u, v] : edges)
        if (dsu.unite(u, v)) total += w;
    return total;
}
```

Graph Theory — Tarjan's Algorithm

Tutorials · 1 card

Tarjan's — SCCs & Bridges

Finds Strongly Connected Components in $O(V+E)$. Also used to find bridges ($low[v] > tin[u]$) and articulation points.

Complexity: Time $O(V+E)$ · Space $O(V)$

• C++

```
struct Tarjan {
    int timer = 0;
    vector<int> tin, low;
    vector<bool> onStack;
    stack<int> stk;
    vector<vector<int>> sccs;

    void dfs(int u, vector<vector<int>>& g) {
        tin[u] = low[u] = timer++;
        stk.push(u); onStack[u] = true;
        for (int v : g[u]) {
            if (tin[v] == -1) { dfs(v, g); low[u] = min(low[u], low[v]);

                else if (onStack[v]) low[u] = min(low[u], tin[v]);
            }
            if (low[u] == tin[u]) { // root of SCC
                vector<int> scc;
                while (stk.top() != u) {
                    int v = stk.top(); stk.pop();
                    onStack[v] = false; scc.push_back(v);
                }
                stk.pop(); onStack[u] = false; scc.push_back(u);
                sccs.push_back(scc);
            }
        }
    }
};
```

```
    }
}

vector<vector<int>> run(int n, vector<vector<int>>& g) {
    tin.assign(n,-1); low.assign(n,-1); onStack.assign(n,false);
    for (int u = 0; u < n; u++) if (tin[u] == -1) dfs(u, g);
    return sccs;
}
};
```

Graph Theory — LCA

Tutorials · 1 card

Lowest Common Ancestor

Deepest node that has both x and y as descendants. $\text{dist}(x,y) = \text{depth}(x) + \text{depth}(y) - 2 \cdot \text{depth}(\text{LCA})$.

Complexity: Time $O(n)$ · Space $O(n)$

• C++

```
// LC 236: LCA of Binary Tree
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q)
{
    if (!root || root == p || root == q) return root;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root; // p and q on opposite sides
    return left ? left : right;
}
```

Math — Bit Manipulation

Tutorials · 2 cards

Bit Operations Cheat Sheet

Direct binary manipulation. All operations $O(1)$. Essential for subset enumeration and optimization.

• C++

```
// Common bit tricks
n & (n-1) // remove lowest set bit (isPowerOfTwo: n>0 && !(n&(n-1)))
n & (~n) // isolate lowest set bit
n | (1<<i) // set bit i
n & ~(1<<i) // clear bit i
(n >> i) & 1 // get bit i
n ^ n = 0 // XOR with self cancels
n ^ 0 = n // XOR with 0 unchanged
__builtin_popcount(n) // count set bits
__builtin_ctz(n) // count trailing zeros
31 - __builtin_clz(n) // index of highest set bit

// Enumerate all submasks of mask m:
for (int s = m; s; s = (s-1) & m) { /* use s */ }

// Subsets (bit masking):
for (int mask = 0; mask < (1<<n); mask++)
    for (int j = 0; j < n; j++)
        if ((mask>>j)&1) { /* j is in subset */ }
```

Bit Manipulation — Common Problems

XOR to find single number, Hamming weight, missing number, subsets.

• C++

```
// Single Number (LC 136) — XOR cancels pairs
int singleNumber(vector<int>& nums) {
    int ans = 0;
    for (int x : nums) ans ^= x;
    return ans;
}

// Hamming Weight (LC 191)
int hammingWeight(uint32_t n) {
    int ans = 0;
    for (; n &= n-1; n++) // remove lowest set bit each time
        return ans;
}

// Missing Number (LC 268)
int missingNumber(vector<int>& nums) {
    int n = nums.size(), ans = n;
    for (int i = 0; i < n; i++) ans ^= i ^ nums[i];
    return ans;
}
```

Math — Combinatorics

Tutorials · 1 card

Combinatorics — nCr with Modular Inverse

$C(n,r) = n! / (r!(n-r)!)$. Use modular inverse for large values. Pascal's Triangle gives $C(n,r) = C(n-1,r) + C(n-1,r-1)$.

Complexity: Precompute $O(n)$ · Query $O(1)$

• C++

```
struct Comb {
    int mod;
    vector<long long> fact, inv_fact;

    Comb(int N, int mod = 1e9+7) : mod(mod), fact(N+1), inv_fact(N+1) {
        fact[0] = 1;
        for (int i = 1; i <= N; i++) fact[i] = fact[i-1] * i % mod;
        inv_fact[N] = power(fact[N], mod-2);
        for (int i = N-1; i >= 0; i--) inv_fact[i] = inv_fact[i+1] * (i+1) % mod;
    }

    long long power(long long a, long long b) {
        long long res = 1; a %= mod;
        for (; b > 0; b >= 1, a = a*a%mod)
            if (b&1) res = res*a%mod;
        return res;
    }

    long long nCr(int n, int r) {
        if (r < 0 || r > n) return 0;
        return fact[n] % mod * inv_fact[r] % mod * inv_fact[n-r] % mod;
    }
};

// Comb c(1000000); c.nCr(100, 5);
```

Math — Matrix Exponentiation

Tutorials · 1 card

Math — Matrix Exponentiation

Matrix Exponentiation — $O(\log n)$ Recurrences

Compute linear recurrences (e.g., Fibonacci) in $O(k^3 \log n)$ by using binary exponentiation on the transformation matrix.

Complexity: Time $O(k^3 \log n)$ · Space $O(k^2)$

• C++

```
const int MOD = 1e9+7;
using Mat = vector<vector<long long>>;

Mat multiply(const Mat& A, const Mat& B) {
    int n = A.size();
    Mat C(n, vector<long long>(n, 0));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < n; k++) if (A[i][k])
            for (int j = 0; j < n; j++)
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
    return C;
}

Mat matpow(Mat A, long long p) {
    int n = A.size();
    Mat res(n, vector<long long>(n, 0));
    Mat res(n, vector<long long>(n, 1)); // identity
    for (; p > 0; p >= 1, A = multiply(A, A))
        if (p & 1) res = multiply(res, A);
    return res;
}
```

```
// Fibonacci in  $O(\log n)$ 
long long fib(int n) {
    Mat T = {{1,1},{1,0}};
    Mat A = {{1},{0}};
    return multiply(matpow(T, n), A)[1][0];
}
```

Math — Number Theory

Tutorials · 2 cards

Math — Number Theory

Binary Exponentiation — $O(\log n)$ Powers

Efficiently computes a^n in $O(\log n)$ using the binary representation of the exponent. Each step squares the base and halves the exponent.

Complexity: Time $O(\log n)$ · Space $O(1)$

• C++

```
// Basic – computes base^exp
long long fastpow(long long base, long long exp) {
    long long res = 1;
    while (exp > 0) {
        if (exp & 1) res *= base;
        base *= base;
        exp >>= 1;
    }
    return res;
}

// With modulo – prevents overflow in modular arithmetic
long long modpow(long long base, long long exp, long long mod) {
    base %= mod;
    long long res = 1;
    while (exp > 0) {
        if (exp & 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp >>= 1;
    }
    return res;
}
```

```
// e.g.,  $3^{13} \bmod 1e9+7$ 
// modpow(3, 13, 1e9+7)
```

Math — Number Theory

Sieve of Eratosthenes — All Primes $\leq N$

Finds all prime numbers up to N . Iteratively marks multiples of each prime as composite. Outer loop runs only to $\sqrt{N}$ — composites already marked by smaller primes.

Complexity: Time $O(n \log \log n)$ · Space $O(n)$

• C++

```
vector<bool> sieveOfEratosthenes(const int n) {
    vector<bool> isPrime(n + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= n; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= n; j += i) {
                isPrime[j] = false;
            }
        }
    }
    return isPrime;
}
```

```
// Usage: count primes up to  $10^6$ 
// auto p = sieveOfEratosthenes(1e6);
// count(p.begin(), p.end(), true) → 78498
```

Math — Prime Factors

Tutorials · 2 cards

Math — Prime Factors

Prime Factorization — Trial Division

A number has at most $\log(n)$ prime factors since $2^{\log(n)} \geq n$. Trial division up to $\sqrt{n}$ finds all factors.

Complexity: Time $O(\sqrt{n})$ · Space $O(\log n)$

• C++

```
vector<int> primeFactors(int n) {
    vector<int> factors;
    for (int i = 2; i * i <= n; i++) {
        while (n % i == 0) {
            factors.push_back(i);
            n /= i;
        }
    }
    if (n > 1) factors.push_back(n);
    return factors;
}
```

Math — Prime Factors

Sieve — $O(\log n)$ Factorization per Query

Precompute minimum prime factor for every number $\leq \text{MAX}$. Then factor any n in $O(\log n)$.

Complexity: Sieve $O(\text{MAX})$ · Factorize $O(\log n)$ · Space $O(\text{MAX})$

• C++

```
vector<int> buildSieve(int MAX) {
    vector<int> minP(MAX+1);
    iota(minP.begin(), minP.end(), 0);
    for (int i = 2; i*i <= MAX; i++)
        if (minP[i] == i) // i is prime
            for (int j = i*i; j <= MAX; j += i)
                minP[j] = min(minP[j], i);
    return minP;
}

vector<int> factorize(int n, vector<int>& minP) {
    vector<int> factors;
    while (n > 1) {
        factors.push_back(minP[n]);
        n /= minP[n];
    }
    return factors;
}
```

Strings — Manacher's Algorithm

Tutorials · 1 card

Strings — Manacher's Algorithm

Manacher's — $O(n)$ Palindrome Detection

Finds all palindromic substrings in $O(n)$ by inserting # separators and reusing previously computed palindrome lengths via mirroring.

Complexity: Time $O(n)$ · Space $O(n)$

• Python

```
def manachers(s: str) -> list[int]:
    # Insert # to unify odd/even cases
    t = '#'.join(s)
    n = len(t)
    p = [0] * n # palindrome radii
    c = r = 0
    for i in range(n):
        if i < r:
            p[i] = min(r - i, p[2*c - i])
            while i-p[i]-1 >= 0 and i+p[i]+1 < n and t[i-p[i]-1] == t[i+p[i]+1]:
                p[i] += 1
            if i + p[i] > r:
                c, r = i, i + p[i]
    return p # p[i] = radius of palindrome centered at t[i]
```

• C++

```
string longestPalindrome(string s) {
    string t = "#";
    for (char c : s) { t += c; t += '#'; }
    int n = t.size();
    vector<int> p(n, 0);
    int c = 0, r = 0;
    for (int i = 0; i < n; i++) {
        if (i < r) p[i] = min(r-i, p[2*c-i]);
        while (i-p[i]-1 >= 0 && i+p[i]+1 < n && t[i-p[i]-1] == t[i+p[i]+1])
            p[i]++;
        if (i+p[i] > r) { c=i; r=i+p[i]; }
    }
    int best = max_element(p.begin(), p.end()) - p.begin();
    return s.substr((best - p[best])/2, p[best]);
}
```

Strings — Palindrome

Tutorials · 1 card

Strings — Palindrome

Palindrome Check — Multiple Methods

A string reads the same forwards and backwards. Check by comparing from both ends inward.

Complexity: Time $O(n)$ · Space $O(1)$

• C++

```
// Method 1: Two pointers
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        if (s[l] != s[r]) return false;
        l++; r--;
    }
    return true;
}

// Method 2: Reverse compare
bool isPalindrome2(string s) {
    string rev = s;
    reverse(rev.begin(), rev.end());
    return s == rev;
}

// Method 3: STL equal with reverse iterators
bool isPalindrome3(string s) {
    return equal(s.begin(), s.begin() + s.size()/2, s.rbegin());
}
```

• Python

```
# All in one line
def isPalindrome(s: str) -> bool:
    return s == s[::-1]

# Two pointers
def isPalindrome2(s: str) -> bool:
    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]: return False
        l += 1; r -= 1
    return True
```

Strings — Z Algorithm

Tutorials · 1 card

Strings — Z Algorithm

Z Algorithm — $O(n)$ Pattern Matching

Builds Z-array where $Z[i]$ = length of longest substring starting at i matching a prefix. $Z[0] = 0$ by convention.

Complexity: Time $O(n)$ · Space $O(n)$

• C++

```
vector<int> zFunction(string s) {
    int n = s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = min(r-i, z[i-l]);
        while (i+z[i] < n && s[z[i]] == s[i+z[i]]) z[i]++;
        if (i+z[i] > r) { l=i; r=i+z[i]; }
    }
    return z;
}

// Pattern matching: find all occurrences of pattern in text
// Concat: pattern + '$' + text, check z[i] == pattern.size()
vector<int> findPattern(string pat, string txt) {
    string s = pat + '$' + txt;
    auto z = zFunction(s);
    vector<int> matches;
    int m = pat.size();
    for (int i = m+1; i < s.size(); i++)
        if (z[i] == m) matches.push_back(i - m - 1);
    return matches;
}
```

• Python

```
def z_function(s: str) -> list[int]:
    n = len(s)
    z = [0] * n
    l = r = 0
    for i in range(1, n):
        if i < r:
            z[i] = min(r - i, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
        if i + z[i] > r:
            l, r = i, i + z[i]
    return z
```


Part 2

System Design Q&A

65 cards · 5 categories

Design Problems

System Design · 29 cards

Design a URL Shortener (TinyURL)

CLARIFY FIRST

- Read-heavy or write-heavy? DAU? Custom aliases?
- Expiry needed? Analytics on clicks?

DATA MODEL

- urls: id, short_code (VARCHAR 7), original_url, user_id, created_at, expires_at
- Short code: Base62 encode a counter (0-9a-zA-Z) → $62^7 = 3.5$ trillion URLs

HIGH-LEVEL DESIGN

Client → LB → App Server → Cache (Redis) → DB (MySQL)

KEY DECISIONS

- Code generation: Counter-based (no collisions) > random hash
- Counter: Redis INCR or DB auto-increment with distributed lock
- Redirect: 301 (permanent, browser caches) vs 302 (you track every click)
- Cache: Redis stores short→long, LRU eviction, TTL for expiry

SCALE

- 100M URLs/day = ~1,160 writes/sec, reads 10x higher
- Read replicas for DB, CDN in front of redirect layer

WOW FACTOR

- Analytics: stream click events to Kafka async (never slow down redirects)
- Custom aliases: check uniqueness with DB unique constraint before insert
- Bloom filter: quick "does this alias exist?" check before DB hit

Design Twitter / News Feed

CLARIFY FIRST

- Just feed? Likes/retweets? DAU? Celebrity accounts?
- Real-time or near-real-time? Ranked or chronological?

DATA MODEL

- tweets: id, user_id, content, created_at, media_url
- follows: follower_id, followee_id
- feed_cache: Redis sorted set per user_id → [tweet_ids] scored by timestamp

FAN-OUT STRATEGIES

- Fan-out on WRITE (push): On tweet, write to all followers' feed caches
- Fast reads, slow writes, bad for celebrities (Lady Gaga = 80M writes)
- Fan-out on READ (pull): Merge followees' timelines at query time
- Slow reads, storage efficient
- HYBRID (Twitter's actual approach): push for normal users, pull for celebs

FEED SERVICE

- Redis sorted set: ZADD feed:{user_id} timestamp tweet_id
- Pagination: ZREVRANGE feed:{user_id} 0 19

SCALE

- Separate services: Tweet, Feed, User, Media
- Kafka: async fan-out workers consume tweet events

WOW FACTOR

- Hot partition: celebrity tweet causes thundering herd → rate-limit fan-out
- ML ranking: replace pure chronological with engagement-based ranking
- Media: pre-signed S3 upload URL → client uploads direct, bypasses servers

Design a Rate Limiter

CLARIFY FIRST

- Per user? Per IP? Per API key? Distributed (multiple servers)?
- Hard block or soft warn first? Global or per-endpoint limits?

ALGORITHMS

1. Token Bucket: tokens added at fixed rate, consumed per request. Allows bursts.
2. Sliding Window Log: log every request timestamp. Memory heavy.
3. Sliding Window Counter: fixed window + rolling weight. Best balance.
4. Leaky Bucket: queue requests, process at fixed rate. No bursts allowed.

DESIGN

Client → LB → Rate Limiter Middleware → App Server
Middleware checks Redis before forwarding

REDIS IMPLEMENTATION

- Use Lua script: check + decrement is atomic (prevents race conditions)
- Key: rate:{user_id} Value: counter TTL: window size

DISTRIBUTED RATE LIMITING

- Redis cluster: centralized, consistent, small network latency
- Local + Redis: fast local counter first, sync to Redis periodically
- Sloppy counting is acceptable for most use cases

System Design — Design Problems

WOW FACTOR

- Return headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
- Soft limits: warn user at 80% before hard block
- Circuit breaker: if Redis is down, fail open (allow) vs fail closed (deny) — discuss the tradeoff

System Design — Design Problems

Design WhatsApp / Chat Application

CLARIFY FIRST

- 1-on-1 only or group chats? Max group size? Message history? Read receipts?
- Offline support? End-to-end encryption?

DATA MODEL

- users: id, phone, name, last_seen
- conversations: id, type (direct/group), name
- conversation_members: conversation_id, user_id, joined_at
- messages: id, conversation_id, sender_id, content, created_at, status

REAL-TIME MESSAGING

- WebSocket: persistent connection per user for real-time push
- Connection Service: maps user_id → server (stored in Redis)
- If recipient on different server → route via message queue (Kafka)

MESSAGE FLOW

- Sender → WebSocket → Chat Server
- Chat Server saves message to Cassandra (write-heavy, time-series)
- Fan-out: publish to Kafka topic per conversation
- Workers deliver to online recipients via their WebSocket servers
- Offline users: store in queue, deliver on reconnect with sequence numbers

SCALE

- Cassandra: partition by (conversation_id, month) → fast range reads
- Group chats: cap at 1000 members for fan-out manageability

System Design — Design Problems

WOW FACTOR

- E2E encryption: store encrypted blobs, keys never leave client
- Message status: sent → delivered → read (double tick via ACKs back to sender)
- Group chat ≈ Multiplayer card game: same pub/sub pattern underneath

System Design — Design Problems

Design Uber / Ride-Sharing

CLARIFY FIRST

- Just matching? Include real-time tracking? Pricing? DAU and # drivers?

DATA MODEL

- drivers: id, current_location (lat/lng), status, vehicle_info
- riders: id, name, payment_method
- trips: id, rider_id, driver_id, status, pickup, dropoff, price, created_at
- Driver location updated every 4 seconds

LOCATION SERVICE (the core problem)

- Redis Geo: GEOADD driver locations, GEODIST, GEORADIUS for nearby search
- Driver heartbeat: update every 4s, TTL=30s (expired entry = offline driver)
- QuadTree or Google S2 for accurate geo-cell partitioning

MATCHING FLOW

- Rider requests trip → Trip Service creates record (status: SEARCHING)
- Location Service finds drivers within 5km
- Matching Service ranks by ETA + rating, sends offer via WebSocket
- Driver accepts → status: MATCHED → both sides get real-time updates

SCALE

- 1M drivers × 15 updates/min = 250K writes/sec → Redis cluster required
- Surge pricing: supply/demand ratio per geo-cell triggers multiplier

System Design — Design Problems

WOW FACTOR

- Trip state machine: SEARCHING→MATCHED→ENROUTE→ARRIVED→INPROGRESS→COMPLETED
- Payment: async after trip ends via Saga pattern (idempotent retries)
- Fraud detection: ML scoring on unusual pickup/dropoff patterns

System Design — Design Problems

Design Instagram

CLARIFY FIRST

- Photo upload + feed? Stories? DMs? Start with core photo + feed.
- DAU? Global or regional? Strong consistency on likes needed?

DATA MODEL

- photos: id, user_id, s3_url, caption, created_at
- follows: follower_id, followee_id
- likes: photo_id, user_id, created_at
- feed_cache: Redis sorted set per user_id

PHOTO UPLOAD FLOW

- Client requests pre-signed S3 URL from Upload Service
- Client uploads DIRECTLY to S3 (never touches your app servers)
- S3 event triggers async Transcoding/thumbnail generation
- Metadata saved to DB, CDN URL stored → photo ready

FEED GENERATION

- Hybrid fan-out: push for normal users, pull for celebrities (>10K followers)
- Denormalize: store enough in feed cache to avoid DB lookups on render

SCALE

- Storage: 1B photos × 3MB avg = 3PB → S3 + Glacier for cold content
- CDN: Akamai/CloudFront serves from edge, near zero origin load for reads

WOW FACTOR

- Stories: separate service, 24hr TTL, ring buffer per user
- Explore feed: ML ranking on engagement signals (not just follows)
- Blurhash: generate colour placeholder on upload → instant perceived performance

System Design — Design Problems

Design Google Autocomplete / Search Suggestions

CLARIFY FIRST

- QPS? Personalized results? How fresh must suggestions be? Typo tolerance?
- Top-5 or top-10? Mobile or desktop?

DATA PIPELINE

- Search logs → Kafka stream → Spark aggregation job → frequency DB
- Trie stores prefix → [top 5 queries with frequency scores]
- Updated hourly via offline batch job, not real-time

ARCHITECTURE

- Data Collection: stream every search event to Kafka
- Aggregation: Spark/Flink counts query frequencies with decay factor
- Trie Service: serialized trie in Redis, rebuilt hourly
- API: GET /autocomplete?prefix=face → top 5 in <100ms

STORAGE STRATEGY (prefix table is simpler than trie in practice)

- Store every prefix → top 5 in a DB/cache
- "fa" → ["facebook","fast food","fate"...]
- "fac" → ["facebook","facetime"...]
- Reads are simple key lookups — very fast

SCALE

- Read-heavy: cache results aggressively (TTL 1hr for global suggestions)
- Shard by prefix first character → 26 shards easily handles load

WOW FACTOR

- Personalization: blend global top-K with user's personal search history (weighted)
- Decay factor: recent searches score higher than old ones
- Typo tolerance: BK-tree or edit distance for fuzzy prefix matching

System Design — Design Problems

Design YouTube Video Upload & Streaming

CLARIFY FIRST

- Just upload pipeline? Include streaming/delivery? Comments/likes?
- Video lengths? Global or regional? Live streaming?

DATA MODEL

- videos: id, user_id, title, status, s3_key, created_at
- video_chunks: video_id, chunk_index, s3_url
- video_formats: video_id, resolution, s3_url, duration

UPLOAD PIPELINE

1. Client splits video into 5MB chunks
2. Upload Service returns pre-signed S3 URLs per chunk
3. Client uploads chunks to S3 IN PARALLEL
4. Client signals completion → Transcoding Job queued in Kafka
5. Workers transcode to 1080p/720p/480p/360p via FFmpeg
6. All versions stored in S3, metadata DB updated → status: READY

STREAMING (Adaptive Bitrate)

- HLS/DASH: client requests .m3u8 playlist → downloads 10-sec segments
- Client switches quality automatically based on bandwidth measurement
- CDN serves all segments from edge — origin rarely hit

SCALE

- 500 hours of video uploaded per minute → async transcoding queue critical
- CDN absorbs 90%+ of traffic; origin only for cache misses

WOW FACTOR

- Resume upload: track uploaded chunks in Redis by session → resume on reconnect
- Content moderation: ML scan transcoded video async, don't block upload flow
- Thumbnail: auto-generate keyframes + allow custom upload, store in S3

System Design — Design Problems

Design a Payment Processing System

CLARIFY FIRST

- Card payments only? Refunds? International/multi-currency? Retry logic?
- Most critical requirement: exactly-once processing (NO double charges)

DATA MODEL

- payments: id, user_id, amount, currency, status, idempotency_key, created_at
- payment_events: payment_id, event_type, timestamp, metadata (audit log)

ARCHITECTURE

Client → Payment API → Payment Service → Payment Gateway (Stripe/Braintree)
Async confirmation via webhook → update payment status

IDEMPOTENCY (the key insight)

- Client generates UUID idempotency_key for every request
- Before processing: check if key exists in DB → return same response if yes
- Prevents duplicate charges on network timeout + retry

DISTRIBUTED TRANSACTIONS — SAGA PATTERN

Order created → Payment charged → Inventory reserved → Notification sent
If payment fails → compensating transactions run in reverse (cancel order, release inventory)

System Design — Design Problems

WOW FACTOR

- At-least-once delivery + idempotent consumers = effectively exactly-once
- Reconciliation: nightly batch job compares your DB vs gateway records (catch any drift)
- PCI compliance: never store raw card numbers — use tokenization (Stripe handles this)
- Outbox pattern: write payment event to DB and message queue atomically

System Design — Design Problems

Design a Notification System

CLARIFY FIRST

- Which channels? Push (mobile), email, SMS, in-app?
- Priority levels? Do-not-disturb preferences? Delivery guarantees?

DATA MODEL

- notifications: id, user_id, type, title, body, status, created_at
- user_preferences: user_id, channel, enabled, quiet_hours
- notification_logs: notification_id, channel, status, sent_at

ARCHITECTURE

Event Source → Message Queue (Kafka) → Fan-out Workers → Channel Gateways

- Push: FCM (Android) / APNs (iOS)
- Email: SendGrid / SES
- SMS: Twilio

FLOW

1. Service emits event (OrderShipped, FriendRequest, etc.) to Kafka
2. Notification Service consumes event, checks user preferences
3. Creates notification records, fans out to channel-specific queues
4. Channel workers pick up jobs, call respective gateways
5. Update delivery status via webhook callbacks

SCALE

- Decouple fan-out: don't block the source service, async always
- Priority queues: critical alerts (payment failed) go to high-priority queue

System Design — Design Problems

p.164

System Design — Design Problems

WOW FACTOR

- Template service: centralize notification copy, A/B testable
- Rate limiting: max 3 push notifications/hour per user (avoid spam)
- Idempotency: deduplicate events with event_id to avoid double-sending
- Analytics: track open rates, click rates per notification type

System Design — Design Problems

p.165

System Design — Design Problems

Design a Distributed Key-Value Store (like DynamoDB)

CLARIFY FIRST

- Strong or eventual consistency? Read-heavy or write-heavy?
- How much data? Single-region or multi-region?

CORE ARCHITECTURE

- Consistent Hashing: nodes on ring, key hashes to position, routes to next node
- Virtual nodes: each physical node = 150 ring positions → even distribution
- Replication: store N copies on N consecutive clockwise nodes (N=3 typical)

QUORUM READS/Writes

- W = minimum nodes that must ack a write (typically 2)
- R = minimum nodes that must respond to a read (typically 2)
- W + R > N = strong consistency. W + R ≤ N = eventual consistency

CONFLICT RESOLUTION

- Vector clocks: each write tagged with version vector
- Last-write-wins: use timestamp (simpler, can lose concurrent writes)
- CRDTs: data types that merge automatically (counters, sets)

FAILURE HANDLING

- Hinted handoff: if target node down, store hint on another, deliver on recovery
- Anti-entropy: Merkle trees detect diverged replicas and sync differences
- Gossip protocol: nodes share cluster membership — no central coordinator

System Design — Design Problems

p.166

System Design — Design Problems

WOW FACTOR

- CAP choice: Dynamo chooses AP (always write, reconcile later) = Cassandra
- Sloppy quorum: accept writes to ANY W healthy nodes, not just "correct" ones
- Bloom filters: check if key might exist before expensive disk read

System Design — Design Problems

p.167

System Design — Design Problems

Design Dropbox / File Sync System

CLARIFY FIRST

- Sync across devices? Sharing/collaboration? Conflict resolution?
- File size limits? Versioning / undo?

DATA MODEL

- files: id, user_id, name, size, content_hash, version, s3_key, updated_at
- file_chunks: file_id, chunk_index, chunk_hash, s3_key
- file_events: file_id, event_type, timestamp (for sync log)

UPLOAD FLOW

1. Client watches filesystem for changes (inotify/FSEvents)
2. On change: split file into 4MB chunks, hash each chunk
3. Check with server which chunks are new (delta sync)
4. Upload ONLY new/changed chunks to S3
5. Update file metadata in DB

SYNC ACROSS DEVICES

- Long-poll or WebSocket: server pushes file_events to connected clients
- Each client maintains local sync cursor (last_event_id)
- On reconnect: pull all events since cursor → apply changes

DEDUPLICATION

- Content-addressable storage: chunk_hash as S3 key
- If another user uploads same file → chunk already exists, no re-upload needed
- Saves 30-40% storage across a large user base

System Design — Design Problems

p.168

WOW FACTOR

- Conflict handling: if two devices edit same file → create conflict copy (like Dropbox does)
- Bandwidth: delta sync (only changed chunks) crucial for large files
- Resumable: track uploaded chunks per session in Redis → resume on disconnect

System Design — Design Problems

Design a Web Crawler

CLARIFY FIRST

- Scope: entire web or specific domain? Frequency of re-crawl? How many pages?
- Store HTML? Extract links only? Handle JavaScript-rendered pages?

DATA MODEL

- urls_to_crawl: id, url, priority, depth, added_at (the frontier queue)
- crawled_pages: id, url, content_hash, crawled_at, status_code, links_found
- Dedup store: URL hash → crawled flag (Bloom filter or Redis SET)

HIGH-LEVEL DESIGN

URL Frontier → Fetcher Workers → HTML Parser → Link Extractor → URL Filter → Frontier
↓
Content Storage (S3)

KEY COMPONENTS

- URL Frontier: priority queue (BFS = breadth-first). Partitioned by domain.
- Politeness: respect robots.txt. 1 request/domain/second. Delay per host.
- DNS resolver: cache DNS lookups (expensive). Batch resolution.
- Content dedup: hash page content → skip if already seen (SimHash for near-dups)

SCALE

- 1B pages, 500 bytes avg = 500GB/day → distribute frontier across machines
- Multi-threaded fetchers — one thread per domain (politeness)
- Bloom filter: 1B URLs, 1% false positive rate = ~1.2GB RAM (vs 20GB for hash set)

WOW FACTOR

- SimHash: detect near-duplicate pages (same article, different ads) — not just exact dups
- Recrawl priority: news sites = every hour; static docs = every month
- URL trap detection: infinite pagination loops → max depth limit per domain

System Design — Design Problems

Design a Real-time Leaderboard

CLARIFY FIRST

- Global leaderboard or per-game? Real-time or near-real-time? Top-100 or full rank?
- Score updates frequency? Billions of users or millions?

DATA MODEL

- scores: user_id, score, updated_at (source of truth in DB)
- leaderboard cache: Redis sorted set → ZADD leaderboard score user_id

REAL-TIME LEADERBOARD (Redis Sorted Set)

- ZADD leaderboard 9500 user:123 → add/update score
- ZREVRANK leaderboard user:123 → get rank (0-indexed)
- ZREVRANGE leaderboard 0 99 WITHSCORES → top 100 with scores
- All operations O(log N) — handles millions of users easily

FLOW

1. User completes game → score event → Kafka topic
2. Score Service consumes event → updates Redis sorted set atomically
3. Score Service also writes to DB (async) for durability
4. Leaderboard API reads from Redis — sub-millisecond response

SCALE

- Redis sorted set handles 100M users comfortably on a single instance
- Shard by game_id if running thousands of separate leaderboards
- Historical leaderboards: store snapshots in DB (daily/weekly), don't keep in Redis

WOW FACTOR

- Friend leaderboard: ZRANGEBYSCORE filtered by friend list (store friend IDs in Redis SET)
- Ties: ZADD uses float scores → append microsecond timestamp as decimal for tiebreak
- Near-real-time: batch score updates every 1s with Lua script for atomic multi-update

System Design — Design Problems

Design Google Calendar / Scheduling System

CLARIFY FIRST

- Personal calendar only or shared? Recurring events? Meeting room booking? Timezone handling?
- Conflict detection needed? Notifications? How many users?

DATA MODEL

- events: id, creator_id, title, start_time (UTC), end_time (UTC), recurrence_rule, location
- event_attendees: event_id, user_id, status (accepted/declined/pending)
- calendar_shares: calendar_id, shared_with_user_id, permission
- notifications: event_id, user_id, notify_at, channel

HIGH-LEVEL DESIGN

Client → API Gateway → Event Service → DB (PostgreSQL)
↓
Notification Scheduler → Queue → Workers → FCM/Email

KEY CHALLENGES

Recurring Events:

- Store recurrence rule (RFC 5545: RRULE=FREQ=WEEKLY;BYDAY=MO,WE,FR)
- Expand on-the-fly for display (don't store each occurrence)
- Exception: if one occurrence edited → store as override record

Timezone Handling:

- Always store in UTC. Display in user's local timezone.
- Use IANA timezone IDs (America/New_York), not offsets (DST changes)

Conflict Detection:

- SELECT * WHERE start < newEnd AND end > newStart AND room_id = ?
- Index on (room_id, start_time, end_time) for fast overlap queries

SCALE • Notification scheduler: cron job finds events in next 15 mins → queue jobs • Fan-out: large meeting (1000 attendees) → async worker fan-out
WOW FACTOR • Pub/sub for live updates: attendee accepts → all viewers see update instantly via WebSocket • Smart scheduling: suggest times when all attendees are free (INTERSECT availability windows)

System Design — Design Problems
Design a Real-time Collaborative Document Editor (Google Docs)
CLARIFY FIRST • How many concurrent editors per doc? Conflict resolution strategy? • Rich text or plain text? Version history? Offline support?
DATA MODEL • documents: id, title, owner_id, created_at • document_versions: doc_id, version, snapshot_content, created_at • operations: id, doc_id, user_id, op_type, position, content, version, timestamp
REAL-TIME SYNC • WebSocket: each user holds a persistent connection to a Doc Server • Operations sent as delta changes (not full document) — efficient
CONFLICT RESOLUTION Operational Transformation (OT) — Google Docs approach: • Transform concurrent operations against each other so all clients converge • Insert at pos 5 by User A + Delete at pos 3 by User B → transform A's op after B's • Complex to implement correctly but proven at scale CRDTs (Conflict-Free Replicated Data Types) — Figma/Notion approach: • Data structures that automatically merge without coordination • No central server needed for conflict resolution • Easier to reason about, growing adoption

ARCHITECTURE Client → WebSocket → Doc Server → Operation Queue (Redis) → DB ↓ Broadcast to other connected clients
WOW FACTOR • Cursor presence: show each user's cursor position to others (low-priority WebSocket channel) • Snapshot + ops: store full snapshot every 100 ops → reconstruct by replaying ops from snapshot • Offline: queue ops locally, send on reconnect, server reconciles with OT/CRDT

System Design — Design Problems
Design a Parking Lot System
CLARIFY FIRST • Multiple floors? Vehicle types (compact, large, motorcycle)? Reservations or drive-up only? • Pricing: flat rate or hourly? Payment at kiosk or exit gate?
DATA MODEL • parking_lots: id, name, address, total_spots • parking_spots: id, lot_id, floor, spot_number, type, status (FREE/OCCUPIED/RESERVED) • tickets: id, spot_id, vehicle_id, entry_time, exit_time, amount_charged • reservations: id, spot_id, user_id, start_time, end_time, status
CORE FLOW Entry: Scan plate → Find available spot (matching vehicle type) → Assign → Print ticket → Open gate Exit: Scan ticket → Calculate fee → Process payment → Update spot → Open gate
SPOT ASSIGNMENT • Strategy pattern: NearestEntry (for accessibility), LowestFloor (default), Random • In-memory count per type per floor → fast availability check • DB update on actual assignment (with optimistic locking to prevent double-assign)

KEY CONSIDERATIONS • Concurrency: two cars assigned same spot → use DB transaction + unique constraint • Spot types: COMPACT ← can also fit MOTORCYCLE. LARGE → can park any vehicle. • Pricing: strategy pattern → HourlyPricing, FlatRatePricing, WeekendPricing
SCALE • Single lot: simple DB, no distributed complexity needed • Multi-city network: Central Service per region, aggregate spot counts
WOW FACTOR • License plate recognition (LPR): camera at entry reads plate → auto-lookup reservation • Dynamic pricing: surge on busy periods (airports, events) — adjust base rate per time window • Electric vehicle spots: track charger availability separately, longer minimum dwell time

System Design — Design Problems
Design Google PageRank
CLARIFY FIRST • Full web-scale or subset? Real-time updates or batch? Combined with content ranking?
CORE ALGORITHM PageRank = (1-d) + d × Σ(PageRank(linking_page) / OutLinks(linking_page)) • d = damping factor (0.85) — probability of following a link vs random jump • Iterates until values converge (typically 50-100 iterations) • Intuition: a page is important if important pages link to it
DISTRIBUTED COMPUTATION • Web graph: nodes=pages, edges=hyperlinks → too large for one machine (4T+ pages) • MapReduce / Spark: distribute adjacency matrix across cluster • Map: emit (destination_url, rank_contribution) • Reduce: sum contributions per URL, apply damping factor
DATA MODEL • pages: id, url, current_rank, out_degree • links: source_id, destination_id (sparse adjacency matrix) • Store as CSR (Compressed Sparse Row) format — memory efficient
SCALE • Sparse matrix: only store non-zero edges (most pages link to few others) • Partition graph: split by URL hash across workers • Convergence check: stop when max rank change < 0.001

WOW FACTOR

- Dangling nodes: pages with no outbound links → redistribute their rank uniformly
- Topic-sensitive PageRank: separate rank vectors per topic category
- Modern search: PageRank is one of 200+ signals — content quality, freshness, clicks also matter

System Design — Design Problems

Design Memcache / Redis (LRU Cache)

CLARIFY FIRST

- Single node or distributed? Max memory? Eviction policy? Persistence needed?
- Read/write ratio? Key-value only or richer data structures?

DATA STRUCTURES

- Hash table: O(1) get/set by key
- Doubly linked list: maintain LRU order (head = most recent, tail = least recent)
- Combined: HashMap<key, ListNode> + DoublyLinkedList

LRU OPERATIONS

- GET(key): find in map → move node to head → return value. O(1).
- PUT(key, val): exists → update + move to head. New → add at head.

If at capacity → evict tail node → remove from map. O(1).

DISTRIBUTED CACHE

- Consistent hashing: distribute keys across N cache nodes
- Client-side sharding: client hashes key → picks cache node directly (no proxy)
- Replication: each cache node has a hot standby for HA

SCALE

- Memcached: multi-threaded, pure LRU, key-value only, no persistence
- Redis: single-threaded (fast), rich data types, optional persistence (RDB/AOF)
- Cluster mode: Redis Cluster uses 16,384 hash slots assigned to nodes

WOW FACTOR

- Thundering herd on cold start: warm cache gradually (preload hot keys on startup)
- Cache stamped: add jitter to TTL (not all keys expire simultaneously)
- Write strategies: cache-aside vs write-through vs write-behind — discuss tradeoffs per use case
- Redis persistence: RDB (snapshots) for fast restart, AOF (append-only log) for durability

System Design — Design Problems

Design Ads Fraud Detection System

CLARIFY FIRST

- What types of fraud? Click fraud? Impression fraud? Account fraud?
- Latency requirement? Block in real-time (<100ms) or flag for review?
- How many events/sec?

DATA MODEL

- click_events: id, ad_id, user_id, ip, device_fingerprint, timestamp, geo
- fraud_scores: event_id, score, features, model_version, flagged_at
- blocked_entities: type (ip/user/device), value, reason, blocked_until

ARCHITECTURE

Ad Click → Kafka → Stream Processor (Flink/Spark Streaming) → Fraud Scorer → Decision
↓
Redis (real-time state)

DETECTION SIGNALS

- Click velocity: >50 clicks/IP/hour → suspicious
- CTR anomaly: CTR 10x above baseline for this ad → bot traffic
- Device fingerprint: same fingerprint, 100 different IPs → fraud ring
- Geographic: click from country where ad doesn't run → invalid traffic
- Time pattern: clicks at exactly 1-second intervals → bot

REAL-TIME SCORING

- Feature extraction in stream: sliding window counts in Redis (INCR + EXPIRE)
- ML model inference: lightweight gradient boosting model (<5ms latency)
- Rule engine: hard rules evaluated first (fast), ML score as second layer

WOW FACTOR

- Graph analysis: detect fraud rings — clusters of IPs/devices/users that co-occur suspiciously
- Feedback loop: confirmed fraud → retrain model weekly with new labels
- Advertiser dashboard: show invalid traffic % per campaign with breakdown

System Design — Design Problems

Design Spotify / Music Streaming Service

CLARIFY FIRST

- Audio streaming only or podcasts/videos too? (assume audio + podcasts)
- Free vs premium tiers? Offline playback? Social features?
- Scale: 100M users, 80M songs, 30M DAU

DATA MODEL

Song: song_id, title, artist_id, duration, s3_url, bitrate_versions[]
User: user_id, plan, liked_songs[], playlists[]
Play event: user_id, song_id, timestamp, duration_played, device

HIGH-LEVEL DESIGN

Client → CDN (audio delivery) → API Gateway → Services:

- Catalog Service: song metadata, search (Elasticsearch)
- Streaming Service: signed S3 URLs, adaptive bitrate
- Recommendation Service: ML model (collaborative filtering)
- Playlist Service: CRUD with Redis cache
- Play History: Kafka → analytics pipeline → DynamoDB

KEY DECISIONS

Audio storage: multiple bitrates (128kbps, 256kbps, 320kbps) on S3
Adaptive streaming: HLS chunks (10s segments), client adjusts quality
CDN pre-warm: popular songs at edge nodes globally
Offline: encrypted DRM files stored locally, license server validates

SCALE
Peak streaming: pre-signed S3 URLs bypass app servers Recommendation: batch ML nightly + real-time feedback loop Song upload: background transcode to multiple bitrates via worker queue
WOW FACTOR
• Gapless playback: pre-buffer next track while current track plays • "Audio graph" — Spotify's recommendation is a graph of 30M songs connected by listener co-occurrence • Podcast vs music separated infra — different CDN and latency SLAs

System Design — Design Problems
Design Netflix / Video Streaming Service
CLARIFY FIRST
• Upload (content team) vs streaming (users) — both sides • Global scale: 200M subscribers, 15% of global internet traffic • HD/4K streaming, multiple devices simultaneously
DATA MODEL
Video: video_id, title, s3_keys[], resolutions[], duration, subtitles[] User: user_id, subscription, watchlist[], viewing_history[] ViewingSession: session_id, user_id, video_id, resume_position, quality
HIGH-LEVEL DESIGN
Upload pipeline: raw video → encoding farm → multiple resolutions (360p/720p/1080p/4K) → stored S3 → CDN Streaming pipeline: user → Open Connect CDN (Netflix's private CDN) → adaptive bitrate streaming Services: • Content Catalog: metadata, search (Elasticsearch) • Recommendation: ML service (Netflix uses ~80% of views are recommended) • Streaming: signed manifest URLs, adaptive HLS/DASH • Billing: Stripe integration • Analytics: Kafka → Spark → Redshift
KEY DECISIONS
Netflix Open Connect: private CDN in ISP data centers — reduces transit costs 80% Adaptive bitrate: DASH protocol, client monitors bandwidth every 2s and adjusts Encoding: VP9/H.265 for bandwidth efficiency, parallel encoding farm DRM: Widevine (Android/Chrome), FairPlay (Apple), PlayReady (Microsoft)

SCALE
Pre-position popular content at edge before prime time (predictive caching) Database: Cassandra for viewing history (time-series write heavy) Chaos Engineering: Netflix Simian Army intentionally kills servers to test resilience
WOW FACTOR
• Studio-quality compression: Netflix's encoding cuts file size 20% vs standard H.264 • Personalized thumbnails: same movie shows different thumbnails per user based on taste • 5 nines availability via multi-region active-active with Zuul API gateway

System Design — Design Problems
Design Tinder / Dating App
CLARIFY FIRST
• Core feature: swipe left/right, matches, messaging • Scale: 50M users, 1.6B swipes/day, 26M matches/day • Location-based matching radius?
DATA MODEL
User: user_id, profile{name, bio, photos[], age, gender, location} Swipe: swiper_id, swipee_id, direction (left/right), timestamp Match: match_id, user1_id, user2_id, created_at Message: match_id, sender_id, content, timestamp
HIGH-LEVEL DESIGN
Services: • Profile Service: CRUD + photo upload (S3 + CDN) • Discovery Service: candidate pool generation + ranking • Swipe Service: write swipes, detect mutual likes → create match • Match Service: notify both users via WebSocket/push • Chat Service: WebSocket per match, messages in Cassandra

KEY DECISIONS
Discovery algorithm: 1. Location filter: PostGIS / Geohash to find users within radius 2. Preference filter: age, gender preferences 3. Already-swiped filter: Redis bloom filter (seen set per user) 4. Rank candidates: score by ELO-like attractiveness rating + recency Swipe storage: Redis sorted set per user for fast bloom filter check Match detection: when user A swipes right on B, check if B already swiped right on A → match! Photos: S3 + CloudFront CDN, multiple sizes (thumbnail, full)
SCALE
Swipe volume: write to Redis first, async flush to Cassandra Recommendation batch: pre-compute candidate stacks nightly, refresh on location change Location updates: only update when user moves > 100m (save writes)
WOW FACTOR
• "Smart Photos" — A/B tests your photos automatically and surfaces best-performing one • ELO score: similar to chess rating — your profile is shown more to similar-scored users • Geohash partitions the world into cells, swipe data sharded by geohash region

System Design — Design Problems
Design Reddit / Discussion Platform
CLARIFY FIRST
• Core: posts, comments (threaded), upvotes, subreddits • Scale: 50M DAU, 100K+ communities, 1B+ votes/day • Real-time vote counts? Hot/New/Top ranking?
DATA MODEL
Post: post_id, subreddit_id, user_id, title, body/url, score, created_at Comment: comment_id, post_id, parent_comment_id, user_id, body, score Vote: user_id, entity_id, entity_type, direction (+1/-1) Subreddit: sub_id, name, rules, moderators[], subscriber_count
HIGH-LEVEL DESIGN
Services: • Post/Comment Service: CRUD, stored in PostgreSQL • Vote Service: high write volume → Redis counter + async DB sync • Feed/Ranking Service: computes Hot/New/Top scores • Search: Elasticsearch for cross-subreddit search • Notification Service: mentions, replies → Kafka → push/email Ranking algorithms: • Hot: score / (age + decay_constant)^gravity — Reddit's actual formula • Top: pure vote score • New: chronological

KEY DECISIONS
Vote counting: Redis INCR for real-time, periodic flush to Postgres Threaded comments: nested set model or adjacency list (parent_id) — adjacency simpler, nested faster for reads Feed: each subreddit feed is precomputed and cached in Redis CDN: images/videos in S3 + CloudFront
SCALE
Read-heavy: aggressive Redis caching for hot posts/comments Vote fraud: rate limiting + IP analysis to prevent brigading Fanout: when r/worldnews post goes viral, fan out to 30M subscriber feeds selectively
WOW FACTOR
• Reddit's Hot algorithm is public — "gravity" constant (1.8) controls how fast posts decay • Awards system: premium awards store in separate service, easy to add without core changes • Mod tools: separate moderation service with different SLA — doesn't block main read path

System Design — Design Problems
Design Food Delivery (DoorDash / Uber Eats)
CLARIFY FIRST
• Full flow: customer orders → restaurant accepts → dasher picks up → delivers • Real-time location tracking, ETA updates • Scale: 25M orders/day, 5M dashers
DATA MODEL
Order: order_id, customer_id, restaurant_id, dasher_id, items[], status, total Restaurant: restaurant_id, menu[], location, operating_hours, avg_prep_time Dasher: dasher_id, current_location, status (available/busy), rating Location: dasher_id, lat, lng, timestamp (write-heavy, time-series)
HIGH-LEVEL DESIGN
Services: • Order Service: order lifecycle state machine (placed→confirmed→ready→picked→delivered) • Restaurant Service: menu, availability, ETA estimation • Dasher Matching Service: geospatial query for nearby available dashers • Location Service: dasher GPS updates (every 5s) → Redis + time-series DB • Notification: real-time updates via WebSocket/push at each state change • Payment: Stripe + fraud detection
KEY DECISIONS
Dasher matching: Geohash cells, find dashers within radius, rank by distance + load ETA: restaurant prep time + pickup travel time + delivery travel time (ML model) Location updates: Redis geospatial (GEOADD) for O(log N) nearest dasher queries Order state machine: Kafka events for each transition, audit log in DynamoDB

SCALE
Location writes: 5M dashers × 1 update/5s = 1M writes/sec → Redis + async flush Peak times (dinner rush): autoscale dasher matching service Restaurant surge: dynamic pricing and longer ETAs surfaced proactively
WOW FACTOR
• Predicted demand: ML model pre-positions dashers near restaurants before dinner rush • "Project DASH" — DoorDash pre-computed delivery zones geofenced by Geohash to minimize dispatch time • Multi-order batching: one dasher carries 2 orders to nearby locations — 40% efficiency gain

System Design — Design Problems
Design Amazon E-Commerce / Product Catalog
CLARIFY FIRST
• Scope: product catalog, search, cart, checkout, order tracking • Scale: 300M products, 200M customers, Black Friday peaks 20x normal • Inventory accuracy required? Third-party sellers?
DATA MODEL
Product: product_id, title, description, images[], price, inventory_count, seller_id Order: order_id, user_id, items[], status, payment_id, shipping_address Cart: user_id → [product_id, quantity, price_snapshot] Inventory: product_id, warehouse_id, quantity, reserved_quantity
HIGH-LEVEL DESIGN
Services: • Catalog Service: product data in DynamoDB (read-heavy), Elasticsearch for search • Search: Elasticsearch with typo tolerance, filters, facets, personalized ranking • Cart Service: Redis (session storage), checkout copies to Order Service • Order Service: state machine (placed→payment→fulfillment→shipped→delivered) • Inventory Service: pessimistic locking on reserve, saga pattern for rollback • Recommendation: collaborative filtering ("customers also bought") • Payment: idempotent Stripe integration

KEY DECISIONS
Inventory consistency: optimistic locking for catalog reads, pessimistic reserve at checkout Cart: Redis for fast reads, persisted to DB on checkout Search ranking: ML-based, considers click-through rate, conversion rate, relevance Flash sales: queue-based checkout to prevent oversell, Redis atomic DECR for stock
SCALE
Read/write split: read replicas for catalog, primary only for inventory writes CDN: product images on S3 + CloudFront, compressed and resized per device Black Friday: autoscaling + pre-scaled capacity, circuit breakers on downstream calls
WOW FACTOR
• Amazon's product page = 150+ microservices stitched together via server-side composition • Inventory "reservation" vs "commit" — stock reserved at add-to-cart, committed at payment • A/B testing at massive scale: 1000+ experiments running simultaneously

System Design — Design Problems
Design Airbnb / Property Booking Platform
CLARIFY FIRST
• Host lists property → guest searches → books → stays • Scale: 7M listings, 150M users, 2M+ stays/night • Real-time availability? Double-booking prevention critical
DATA MODEL
Listing: listing_id, host_id, title, location, price_per_night, amenities[], photos[] Booking: booking_id, listing_id, guest_id, check_in, check_out, status, total_price Availability: listing_id, date, status (available/booked/blocked) Review: listing_id, booking_id, guest_id, rating, text
HIGH-LEVEL DESIGN
Services: • Listing Service: CRUD, stored in PostgreSQL + Elasticsearch for search • Search Service: geo-search (PostGIS), date availability filter, price filter, ranking • Booking Service: availability check → reserve → payment → confirm (saga pattern) • Availability Service: calendar management, conflict detection • Review Service: post-stay reviews, aggregate ratings • Payment: Stripe + escrow (holds funds until 24h after check-in) • Notification: booking confirmations, check-in reminders
KEY DECISIONS
Double-booking prevention: pessimistic lock on availability rows during booking transaction Search: Elasticsearch for text + PostGIS for geo, date filter as post-query step Pricing: dynamic pricing engine considers local events, seasonality, demand Availability calendar: separate table per listing × date — fast O(1) lookup

SCALE
Search: read replicas + Elasticsearch cluster; most traffic is search not booking Booking: strong consistency required → PostgreSQL with row-level locking Media: photos stored S3 + CloudFront CDN, watermarked thumbnails generated on upload
WOW FACTOR
• Fraud detection: ML model scores every booking — unusual location combos flagged • "Instant book" vs "request to book" — different code paths, different SLAs • Airbnb's pricing ML considers 70+ signals including local hotel rates and flight searches

System Design — Design Problems
Design Google Maps / Navigation Service
CLARIFY FIRST
• Core: map rendering, search for places, turn-by-turn routing, live traffic • Scale: 1B users, 5M+ map tile requests/sec • Real-time traffic data? ETAs? Multi-modal (driving/transit/walking)?
DATA MODEL
Map tile: z/x/y tile coordinates (zoom/column/row), vector/raster data, version Place: place_id, name, location[lat,lng], category, rating, hours Route: origin, destination, waypoints[], steps[], ETA, distance TrafficSegment: road_segment_id, speed, congestion_level, timestamp
HIGH-LEVEL DESIGN
Map tiles: pre-rendered vector tiles stored in S3 by zoom/x/y, served via CDN Services: • Tile Service: client requests tiles by zoom level + viewport → CDN → S3 • Search Service: place search (Elasticsearch geo), autocomplete • Routing Service: Dijkstra / A* on road graph, real-time traffic weights • ETA Service: ML model using current traffic + historical patterns + route • Traffic Ingestion: GPS pings from 1B devices → Kafka → aggregate speed per road segment
KEY DECISIONS
Map tiles: vector tiles (small, scalable) over raster; client renders locally Road graph: compressed adjacency list, bidirectional A* for fast routing Traffic: anonymous GPS from Google Maps users aggregated per road segment every 2min Routing: precomputed highway routes cached, recalculated for local roads with traffic

SCALE
Tile CDN: 99% of tile requests served from CDN — huge traffic at zoom-in events (concerts etc) Search: Elasticsearch geo-query + autocomplete prefix tree Traffic processing: Kafka consumer groups aggregate GPS → Redis per road segment
WOW FACTOR
• Google's road graph has 30M+ road segments globally — stored as compressed adjacency list • Contraction Hierarchies: preprocessing speeds up Dijkstra 1000x by pre-computing "highway" shortcuts • Live traffic = 1B GPS pings anonymized, aggregated, smoothed per 100m road segment

System Design — Design Problems
Design Zoom / Video Conferencing System
CLARIFY FIRST
• Core: 1:1 calls, group meetings (up to 1000 participants), screen share, recording • Scale: 300M daily meeting participants, peak COVID: 3B meeting minutes/day • Low latency critical (< 150ms end-to-end)
DATA MODEL
Meeting: meeting_id, host_id, participants[], start_time, recording_url, status Participant: user_id, meeting_id, join_time, audio_track, video_track, connection_quality Recording: meeting_id, s3_url, duration, transcript
HIGH-LEVEL DESIGN
Signaling Server: WebSocket server for call setup (SDP offer/answer exchange) Media Servers: handle audio/video mixing and relay • Small meetings (< 5): P2P WebRTC (no media server) • Large meetings: SFU (Selective Forwarding Unit) — relay streams without mixing • Very large: MCU (Multipoint Control Unit) — mix into composite stream Services: • Meeting Service: create/join/leave meetings, persistent state in Redis + PostgreSQL • Auth Service: meeting passwords, waiting room, lobby • Recording Service: capture streams → S3, async transcription • Quality Service: adaptive bitrate based on participant bandwidth

KEY DECISIONS
P2P vs SFU vs MCU: • P2P: best quality, no server cost, but $O(N^2)$ bandwidth for N participants • SFU: each client sends 1 stream up, receives N-1 streams — scales to ~50 participants • MCU: server mixes all streams → 1 stream per client — CPU heavy but bandwidth efficient Jitter buffer: absorbs network packet jitter, smooth playback Forward Error Correction (FEC): sends redundant packets so loss doesn't freeze video
SCALE
Media servers: geo-distributed, participant routed to nearest region Bandwidth: participants send at adaptive bitrate (720p → 360p → audio-only as network degrades) Recording: async — store raw streams in S3, post-process after meeting ends
WOW FACTOR
• Zoom's "subspace tunneling" — proprietary protocol layered over UDP for better packet recovery than standard WebRTC • Virtual backgrounds: ML model runs locally (client-side) — no video sent to Zoom servers • Active speaker detection: audio VAD (voice activity detection) to switch camera focus automatically

System Design — Core Concepts
Core Concepts
System Design · 20 cards
System Design — Design Problems
p.204

Explain the CAP Theorem. How do you apply it when choosing a database?

CAP THEOREM

In a distributed system you can only guarantee 2 of 3:

- C — Consistency: all nodes see the same data at the same time
- A — Availability: every request gets a response (not necessarily latest)
- P — Partition Tolerance: system works despite network failures

THE REALITY

Network partitions ALWAYS happen in distributed systems. So the real choice is: CP or AP — P is never optional.
CP SYSTEMS (sacrifice availability)

- HBase, Zookeeper, etcd, MongoDB (default)
- Rejects requests during partition to maintain consistency
- Use when: banking, inventory (wrong answer = serious problem)

AP SYSTEMS (sacrifice consistency)

- Cassandra, DynamoDB, CouchDB
- Returns potentially stale data — eventually consistent
- Use when: social likes, recommendations, search (stale = acceptable)

WOW FACTOR

- PACELC extends CAP: even WITHOUT partitions you choose Latency vs Consistency
- "Eventual consistency" = all replicas WILL agree, just not instantly
- Cassandra lets you tune per-query: ONE=AP, QUORUM=balanced, ALL=CP
- Real example: bank transfer = CP needed. Twitter like count = AP is fine

What are the main caching strategies? When do you use each?

CACHE-ASIDE (Lazy Loading) — Most Common

- App checks cache first → miss → read DB → write to cache
- Pro: only cache what's needed, resilient to cache failure
- Con: 3 trips on miss, cold start problem
- Use: general purpose, sessions, API responses

WRITE-THROUGH

- Write to cache AND DB simultaneously on every write
- Pro: cache always fresh, no read miss after write
- Con: higher write latency, caches data that may never be read
- Use: when reads closely follow writes (profile updates)

WRITE-BEHIND (Write-Back)

- Write to cache immediately, flush to DB asynchronously
- Pro: extremely fast writes
- Con: data loss if cache crashes before flush
- Use: high-write workloads where small loss is acceptable

EVICTON POLICIES

- LRU (Least Recently Used): evict oldest untouched → Redis default
- LFU (Least Frequently Used): evict least popular over time
- TTL: always set expiry to prevent stale data buildup

WOW FACTOR

- Thundering herd: cache key expires → 10K requests all miss simultaneously
- Fix: mutex lock on first miss, or probabilistic early recompute (PER algorithm)
- Cache stampede protection built into modern Redis with SETNX + Lua scripts

Explain database sharding — strategies, tradeoffs, and when to use it

SHARDING = horizontal partitioning

Split rows across multiple DB servers. Each shard holds a subset. (Different from replication which COPIES all data)

SHARDING STRATEGIES

1. Hash-based: shard = hash(user_id) % N
✓ Even distribution
Adding shards = rehash everything (fix: consistent hashing)
2. Range-based: IDs 1-1M on shard 1, 1M-2M on shard 2
✓ Good for range queries
Hot spots: recent data gets all traffic
3. Directory-based: lookup table maps key → shard
✓ Flexible, easy to rebalance
Lookup service is a single point of failure

PROBLEMS SHARDING INTRODUCES

- Cross-shard joins: painful → denormalize or use application-side joins
- Cross-shard transactions: use Saga pattern, avoid if possible
- Hot shards: celebrity users get disproportionate load
- Rebalancing: consistent hashing minimises data movement

WHEN TO SHARD

- LAST RESORT — try these first:

1. Add read replicas
2. Add caching layer
3. Vertical scaling (bigger machine)
4. Then shard for write scaling only

WOW FACTOR

- Consistent hashing: add/remove shard = only K/n keys remap (not full reshuffle)
- Virtual nodes: each shard gets 150 ring positions = even load despite different hardware

When do you use a message queue? How does Kafka work?

USE A MESSAGE QUEUE WHEN

- Decoupling: producer shouldn't wait for consumer to finish
- Traffic spikes: queue absorbs bursts, consumer processes at own pace
- Fan-out: one event → multiple consumers (email + SMS + push)
- Async: user doesn't need immediate result (report generation, emails)

KAFKA ARCHITECTURE

- Topics: named streams (like a table in a DB)
- Partitions: topic split into N partitions for parallelism
- Producers: write to topic, choose partition by key hash
- Consumers: read from offset (position), track progress themselves
- Consumer Groups: each partition assigned to ONE consumer → parallel processing

DELIVERY GUARANTEES

- At-most-once: fire and forget — possible data loss
- At-least-once: retry on failure → possible duplicates (make consumers idempotent)
- Exactly-once: Kafka transactions + idempotent consumers (harder, use sparingly)

KAFKA vs SQS

- Kafka: replay messages (seek to any offset), persistent, high throughput, complex
- SQS: managed, simpler, messages deleted after consume, no replay
- Use Kafka for event sourcing, audit logs, stream processing
- Use SQS for simple background job queues

WOW FACTOR

- Kafka as source of truth: events ARE the database, rebuild state by replaying
- Consumer lag: if consumers fall behind, queue grows → alert on lag metric
- Compacted topics: keep only latest value per key (acts like a KV store)

System Design — Core Concepts	
Explain Consistent Hashing — why it matters for distributed systems	
THE PROBLEM WITH NAIVE HASHING • shard = hash(key) % N • Add one server (N → N+1) → almost ALL keys remap to different shards • Massive data migration every time you scale CONSISTENT HASHING SOLUTION • Place both servers AND keys on a circular ring (0 to 2^32) • Key is served by the first server clockwise from its hash position • Add a server → only the keys between it and its predecessor remaps • Remove a server → only that server's keys move to the next one • Adding/removing a node moves only K/N keys on average	
VIRTUAL NODES (the practical improvement) • Each physical server = 150+ positions on the ring • Why: even small server clusters get uneven key distribution without this • Result: much better load balancing, even if servers have different capacities • Weighted assignment: powerful machines get more virtual nodes	
REAL-WORLD USAGE • Cassandra: consistent hashing for partition placement • DynamoDB: consistent hashing with preference lists • Redis Cluster: hash slots (16,384 slots) assigned to nodes • CDN: route requests to nearest edge node	
System Design — Core Concepts	p.211

WOW FACTOR • Hot spot problem: if many keys hash near one point → that server overloaded • Fix: virtual nodes spread the hot zone across multiple physical servers • Consistent hashing is also used in load balancers for session stickiness	
System Design — Core Concepts	p.212

System Design — Core Concepts	
SQL vs NoSQL — how do you choose the right database?	
SQL (Relational) • ACID transactions, complex joins, strict schema, foreign keys • Great for: financial data, user accounts, anything needing JOIN queries • Examples: PostgreSQL, MySQL • Scales vertically first, horizontal with sharding (complex)	
NOSQL TYPES & USE CASES	
Document Store (MongoDB, Firestore) • Flexible schema, JSON documents, no joins • Use: user profiles, product catalogs, content management Key-Value (Redis, DynamoDB) • Blazing fast lookups by key, no queries • Use: sessions, caching, leaderboards, rate limiting Wide Column (Cassandra, HBase) • Write-heavy, time-series, massive scale • Use: IoT data, chat messages, activity feeds, logs Graph DB (Neo4j) • Relationships are first-class, complex traversals • Use: social networks, recommendation engines, fraud detection	
DECISION FRAMEWORK • Need complex queries + joins + transactions → SQL • Need massive writes + horizontal scale → Cassandra • Need sub-millisecond reads + caching → Redis • Need flexible schema + document model → MongoDB	
System Design — Core Concepts	p.213

WOW FACTOR • Polyglot persistence: most large systems use 3–4 different DBs for different needs • Don't default to one → explain WHY each component uses a specific DB • PostgreSQL can handle NoSQL too (JSONB columns) — often underestimated	
System Design — Core Concepts	p.214

System Design — Core Concepts	
Explain the 8-Step System Design Interview Process	
THE FRAMEWORK (use this structure every time) 1CLARIFY USE CASES • List major features. Pick 2–3 to focus on. • Ask: read-heavy or write-heavy? DAU? QPS? Global? 2ESTIMATE SCALE • Back-of-envelope: storage/day, requests/sec, bandwidth • 1M users × 10 req/day = ~116 req/sec 3DEFINE DATA MODEL • What entities exist? SQL or NoSQL? How does data flow? 4ABSTRACT DESIGN • Sketch: Client → LB → App Servers → Cache → DB • Show the basic workflow first before going deep 5DETAILED DESIGN • Deep dive on 1–2 components. Explain trade-offs. • Respond to interviewer probes — they're guiding you 6IDENTIFY BOTTLENECKS • Single points of failure? Hot partitions? Cache eviction? • Ask: what breaks at 10x current scale? 7SCALE THE DESIGN • Add: read replicas, sharding, CDN, message queue, async workers 8SHOW EXPERIENCE • Mention industry best practices, real trade-offs you'd consider • Reference real systems: "Twitter uses a hybrid fan-out approach because..."	
WOW FACTOR • Drive the conversation — interviewer should follow you, not lead • Capacity estimation: order of magnitude is enough (TB vs GB, millions vs thousands) • Senior+ tip: casually calculate exact numbers with cost analysis	
System Design — Core Concepts	p.215

System Design — Core Concepts	
Explain DB Indexing and DB Replication	
DB INDEXING	
What it is: data structure (B+ tree or hash) that speeds up reads on a column • B+ tree index: sorted, supports range queries (>, <, BETWEEN, ORDER BY) • Hash index: exact match only, faster for equality (=) When to index: • Columns in WHERE, JOIN, ORDER BY, GROUP BY clauses • Foreign key columns (avoid full table scans on joins) • Avoid over-indexing: every index = slower writes + more storage Cost of indexes: • Reads: dramatically faster (O(log N) vs O(N) full scan) • Writes: every INSERT/UPDATE/DELETE must also update all indexes • Rule: index columns you filter/sort on, not columns you only select	
DB REPLICATION	
Master-Slave (Primary-Replica): • All writes → master (primary) • All reads → slaves (replicas) — scale read throughput horizontally • Replication lag = slaves are slightly behind master • Lag = eventual consistency for reads (may see stale data) Synchronous replication: • Write confirmed only after replica also writes → no data loss, slower writes Asynchronous replication: • Write confirmed at master, replica catches up → faster writes, potential data loss	
System Design — Core Concepts	p.216

WOW FACTOR

- Read-write splitting: route writes to master, reads to nearest replica
- Replication lag monitoring: alert if lag > 1 second (stale reads become a problem)
- Multi-master: both masters accept writes → conflict resolution needed (avoid if possible)
- Covering index: index includes all columns needed by query → zero table access needed

System Design — Core Concepts

Explain Bloom Filters — what they are and where they're used

WHAT IS A BLOOM FILTER?

A space-efficient probabilistic data structure that answers:
"Is this element definitely NOT in the set?" with 100% accuracy
"Is this element possibly in the set?" with configurable accuracy

HOW IT WORKS

- Bit array of M bits, initialized to 0
- K different hash functions
- ADD element: hash with all K functions → set those K bits to 1
- CHECK element: hash with all K functions → if ANY bit is 0 → definitely not in set

→ if ALL bits are 1 → probably in set

PROPERTIES

- False positives: possible (says "maybe yes" when actually "no")
- False negatives: IMPOSSIBLE (if it says "no", it's definitely no)
- Space: 1B elements, 1% false positive rate ≈ 1.2GB (vs 20GB for hash set)
- Cannot delete elements (use Counting Bloom Filter for deletion)

REAL-WORLD USES

- Cassandra/HBase: check if key exists in SSTable before expensive disk read
- Chrome Safe Browsing: check if URL is in malicious list (local bloom filter)
- Web Crawler: check if URL already crawled — avoid re-crawling
- CDN: check if content is cached at edge before going to origin
- Databases: skip disk lookup for keys that definitely don't exist

WOW FACTOR

- Tunable false positive rate: more bits = fewer false positives = more memory
- Formula: $M = -N \times \ln(p) / (\ln 2)^2$ where N=items, p=false positive rate
- Counting Bloom Filter: use counters instead of bits → supports deletion

System Design — Core Concepts

Explain CDN (Content Delivery Network) — how it works and when to use it

WHAT IS A CDN?

A globally distributed network of edge servers that cache content close to users.
Goal: reduce latency, reduce load on origin server, improve availability.

HOW IT WORKS

1. User requests resource (image, video, JS file)
2. DNS routes request to nearest CDN edge server (based on GeoDNS)
3. Edge server checks cache:
 - HIT → return cached content immediately (low latency)
 - MISS → fetch from origin → cache it → return to user
4. Next request for same content → served from edge (no origin hit)

CDN USE CASES

Static assets: JS, CSS, images, fonts → long TTL (1 week+)
Video streaming: HLS/DASH segments cached at edge → adaptive bitrate
API responses: GET endpoints with cache-control headers (short TTL)
Dynamic content: Edge computing (Cloudflare Workers) for personalisation at edge

CACHE INVALIDATION

- TTL-based: content expires after fixed time → simple but may serve stale content
- Purge API: immediately invalidate specific URLs (use after deploy)
- Versioned URLs: /static/app.v1.2.3.js → never expires (immutable)

WOW FACTOR

- Origin shield: one CDN PoP designated as origin shield → prevents thundering herd on origin
- Push vs Pull CDN:
- Pull: CDN fetches from origin on first miss (most common — lazy)
- Push: you proactively upload content to CDN (for large files, predictable access)
- Geographic compliance: CDN can route EU users to EU servers (GDPR)
- Cost: ~90% of bandwidth served from CDN at fraction of origin cost

System Design — Core Concepts

Pessimistic vs Optimistic Locking — when do you use each?

PESSIMISTIC LOCKING

"Assume conflict WILL happen — lock first, then read/write"

How it works:

- SELECT ... FOR UPDATE → locks the row immediately
- No other transaction can read or write that row until you commit/rollback
- Database-level lock

When to use:

- High contention: multiple users likely to update same record simultaneously
- Financial transactions: bank account balance, inventory count
- Short operations where you can't afford any conflict

Downsides:

- Deadlock risk: two transactions waiting for each other's locks
- Low throughput: locks block other readers
- Doesn't scale well for high-concurrency systems

OPTIMISTIC LOCKING

"Assume conflict is RARE — read freely, check at write time"

How it works:

- Add version column to table (version INT)
- Read record, note version (e.g. version=5)
- On update: WHERE id=? AND version=5 → if 0 rows updated → conflict → retry
- No database locks held during business logic

When to use:

- Low contention: users rarely update the same record simultaneously
- Long-running operations: can't hold lock for seconds
- Read-heavy workloads: many readers, few writers

WOW FACTOR

- Optimistic is default in most ORMs (Hibernate, Django ORM)
- For distributed systems: use version vectors or CRDTs instead of DB locks
- Saga pattern: avoids distributed locking entirely with compensating transactions

System Design — Core Concepts

Pull vs Push Model — SSE, WebSockets, Long-Polling explained

PULL MODEL (Client Polls)

Client periodically asks server: "Any updates?"

Short Polling:

- Client sends HTTP request every N seconds
- Simple to implement
- Wasteful: most responses are empty
- Latency: up to N seconds to see new data
- Use: low-frequency updates, simple dashboards

Long Polling:

- Client sends request → server holds connection open until update or timeout
- Better latency, still HTTP-based (works everywhere)
- Server must handle many hanging connections
- Use: chat apps (before WebSockets), notifications

PUSH MODEL (Server Notifies)

Server sends data to client when it's available

Server-Sent Events (SSE):

- One-way: server → client only
- Persistent HTTP connection, text-based
- Auto-reconnect built in
- Use: live feeds, stock prices, notifications

WebSocket:

- Bi-directional: both sides can send/receive
- Full-duplex over single TCP connection
- Lower overhead than HTTP for frequent messages
- Use: chat, collaborative editing, multiplayer games, live tracking

WOW FACTOR

- WebSocket connection per user → connection service maps user_id → server in Redis
- Horizontal scaling: sticky sessions (IP hash LB) OR move connection state to Redis pub/sub
- SSE vs WebSocket: SSE simpler for one-way push, WebSocket when client also sends frequently
- Fallback strategy: WebSocket → SSE → Long Poll (decreasing capability, increasing compatibility)

System Design — Core Concepts

Availability Patterns — Active-Passive, Active-Active, and Chaos Engineering

ACTIVE-PASSIVE (Fail-Over)

- One active server handles all traffic
- Passive standby is ready but idle
- On failure: DNS switches or VIP floats to passive → becomes new active
- Cold standby: passive not running (cost-efficient, slower failover ~minutes)
- Warm standby: passive running but not serving (faster failover ~seconds)
- Hot standby: passive also processing (near-instant failover)
- Use: databases, stateful services where split-brain is dangerous

ACTIVE-ACTIVE

- Multiple servers ALL handle traffic simultaneously
- Load balanced across all nodes
- On failure: remaining nodes absorb traffic (capacity pre-provisioned for this)
- Requires stateless design (or shared state in Redis/DB)
- Use: web app servers, API gateways, caches

AVAILABILITY MATH

- 99% = 3.65 days downtime/year
- 99.9% (3 nines) = 8.77 hours/year
- 99.99% (4 nines) = 52.6 minutes/year
- 99.999% (5 nines) = 5.26 minutes/year → very expensive to achieve

REPLICATION FOR DURABILITY

- N=3 replicas: can lose 2 and still serve data
- Geographic distribution: survive entire data center failure

WOW FACTOR

- Chaos Engineering (Netflix Chaos Monkey): deliberately kill random servers in production
- Forces team to build true resilience — not just paper-resilience
- Circuit breakers + bulkheads: limit blast radius when failure does occur
- SLA vs SLO vs SLI: define what availability means before designing for it

System Design — Core Concepts

Database Concepts · DSA · System Design · Behavioral

System Design — Core Concepts

HyperLogLog, Merkle Tree, and LSM Tree explained

HYPERLOGLOG

Problem: Count distinct items in a stream with billions of elements

Naive: store every element → too much memory

How it works:

- Probabilistic algorithm, ~0.81% error rate
- Uses ~1.5KB memory regardless of cardinality
- Hashes elements, observes patterns in leading zeros

Use cases:

- Redis PFCOUNT: count unique visitors per page
- Unique search queries per day
- Distinct IPs hitting an endpoint

MERKLE TREE

Problem: Efficiently compare two large datasets to find differences

How it works:

- Each leaf = hash of a data block
- Each parent = hash of its children
- Root hash = fingerprint of entire dataset
- Two datasets differ → compare root hashes → traverse to find exactly which blocks differ

Use cases:

- Git: detects which files changed between commits
- Cassandra: anti-entropy repair (sync diverged replicas efficiently)
- Blockchain: verify transaction integrity in a block

LSM TREE (Log-Structured Merge Tree)

Problem: High write throughput — random disk writes are slow

How it works:

- Writes go to in-memory MemTable (sorted) → fast
- When full: flush MemTable to disk as SSTable (immutable, sorted)
- Background: merge/compact SSTables to reduce read amplification

Use cases: Cassandra, RocksDB, LevelDB, HBase (all write-heavy systems)

WOW FACTOR

- LSM vs B-tree: LSM = fast writes, slower reads. B-tree = fast reads, slower writes.
- Bloom filter + LSM: filter eliminates most unnecessary SSTable reads

System Design — Core Concepts

Explain Networking Fundamentals — OSI model, DNS, TCP vs UDP, Proxy vs Reverse Proxy

CLARIFY FIRST

These are the foundation interviewers expect you to know cold.

OSI MODEL (7 layers — remember: "Please Do Not Throw Sausage Pizza Away")

7. Application — HTTP, FTP, SMTP, DNS

6. Presentation — encryption, compression, encoding (TLS)

5. Session — session management, auth tokens

4. Transport — TCP (reliable) / UDP (fast) — ports live here

3. Network — IP addresses, routing between networks

2. Data Link — MAC addresses, LAN communication

1. Physical — cables, wifi, fiber

DNS (Domain Name System)

Browser → Recursive Resolver → Root NS → TLD NS → Authoritative NS → IP

- A record: domain → IPv4
- CNAME: alias to another domain (www → example.com)
- TTL: how long to cache before re-querying
- DNS load balancing: return different IPs per region (GeoDNS)

TCP vs UDP

TCP: connection-oriented, 3-way handshake, guaranteed delivery, ordered
→ Use for: HTTP, file transfers, anything where accuracy > speed
UDP: connectionless, fire-and-forget, no retransmit
→ Use for: video streaming, gaming, DNS, live audio (latency > reliability)

PROXY vs REVERSE PROXY

Forward Proxy: sits in front of CLIENTS — hides client identity, VPNs
Reverse Proxy: sits in front of SERVERS — load balancing, SSL termination, caching, hides server topology

- Nginx / HAProxy / Cloudflare are reverse proxies
- "Client sees one IP; server could be 1000 machines behind it"

WOW FACTOR

- HTTP/3 runs over QUIC (UDP-based) — Google's fix for TCP head-of-line blocking
- DNS over HTTPS (DoH): encrypts DNS queries — prevents ISP snooping
- CDNs are essentially a globally-distributed reverse proxy fleet

System Design — Core Concepts

Explain ACID Transactions and when to use them

CLARIFY FIRST

ACID = guarantees for database transactions. Critical for financial and inventory systems.

ACID BREAKDOWN

A — Atomicity

All operations in a transaction succeed or all fail. No partial writes.
→ "Transfer \$100: debit Alice AND credit Bob — if credit fails, debit is rolled back"

C — Consistency

Every transaction takes the DB from one valid state to another. Constraints are never violated.
→ Account balance can never go negative (if constraint defined)

I — Isolation

Concurrent transactions don't see each other's intermediate state.
Isolation levels (weakest to strongest):

- Read Uncommitted: can read dirty (uncommitted) data — fastest, dangerous
- Read Committed: only read committed data — default in most DBs
- Repeatable Read: same row read twice returns same value in a transaction
- Serializable: transactions behave as if sequential — safest, slowest

D — Durability

Committed data survives crashes. Achieved via Write-Ahead Log (WAL).

WHEN TO USE

Use ACID: payments, banking, inventory, bookings, order management

Relax to BASE (Eventually Consistent): social feeds, analytics, likes/views, search indexes

System Design — Core Concepts

DISTRIBUTED TRANSACTIONS

Single DB: ACID is "free" (DB handles it)
Multi-service: use Saga Pattern (chain of local transactions + compensations) or Two-Phase Commit (coordinator + participants — blocking, avoid if possible)

SCALE TRADEOFFS

ACID with Serializable isolation → throughput drops 10-100x
Most systems use Read Committed + application-level locking for hot rows

WOW FACTOR

- PostgreSQL WAL: every write goes to WAL first → crash recovery replays log
- "Phantom reads" — Repeatable Read doesn't prevent new rows matching a query from appearing
- CockroachDB and Spanner offer distributed ACID with global consistency via atomic clocks

System Design — Core Concepts

Explain Event-Driven Architecture and Microservices

CLARIFY FIRST

When to choose event-driven vs request/response, and how microservices fit in.

EVENT-DRIVEN ARCHITECTURE

Components:

- Producer: publishes events (e.g., "OrderPlaced")
- Event Bus: Kafka / RabbitMQ / SNS+SQS
- Consumer: subscribes to events, processes async

Benefits:

Loose coupling — producer doesn't know consumers

Scalability — consumers scale independently

Resilience — consumer failure doesn't break producer

Audit trail — event log is a history of everything

MICROSERVICES vs MONOLITH

Monolith pros: simple to deploy, no network latency, easy transactions
Monolith cons: one failure can cascade, harder to scale individual parts
Microservices pros: independent deployment, technology diversity, team ownership
Microservices cons: distributed systems complexity, network failures, data consistency hard

WHEN TO USE EACH
Event-driven (async): notifications, sending emails, updating search indexes, analytics Request/response (sync): user-facing APIs where you need the result immediately Common patterns: • Choreography: services react to events independently (no coordinator) • Orchestration: central saga orchestrator tells each service what to do
OPERATIONAL CONCERNS
Service discovery: how do services find each other? (Consul, Kubernetes DNS) Distributed tracing: track a request across services (Jaeger, Zipkin, Datadog) Circuit breaker: stop calling failing services (Hystrix, Resilience4J)
WOW FACTOR
• Netflix: 1000+ microservices, event-driven via Kafka, coordinated by Spring Cloud • "Outbox pattern": write event to DB table in same transaction, then publish → ensures exactly-once event delivery • Event sourcing: store events as the source of truth, derive current state by replaying

System Design — Core Concepts
Explain Change Data Capture (CDC), Pub/Sub, and Message Queues
CLARIFY FIRST
These are the plumbing of async distributed systems.
MESSAGE QUEUES
Point-to-point: one producer, one consumer, message deleted after consumption Examples: SQS, RabbitMQ Use for: task queues, job distribution, rate limiting work PUB/SUB One producer, many consumers (fan-out), each consumer gets a copy Examples: Kafka (persistent log), Google Pub/Sub, SNS+SQS Use for: event broadcasting, activity feeds, notifications to many services Key difference: Queue = each message processed once. Pub/Sub = each subscriber gets every message.
KAFKA DEEP DIVE
• Topics split into Partitions — parallelism unit • Consumer Groups: each group gets all messages; within group, each partition → one consumer • Offset: position in partition, consumers commit after processing • Retention: messages kept for configurable period (default 7 days) — replay possible • At-least-once delivery: may get duplicates → consumers must be idempotent

CHANGE DATA CAPTURE (CDC)
What: capture every insert/update/delete from DB and stream as events How: Debezium reads PostgreSQL WAL (write-ahead log) and publishes to Kafka Why: sync DB changes to search indexes, caches, analytics without polling CDC flow: DB write → WAL entry → Debezium reads WAL → Kafka topic → Elasticsearch / Redis / DW
PATTERNS
Exactly-once: Kafka transactions + idempotent consumers Dead Letter Queue (DLQ): failed messages go to separate queue for inspection Backpressure: consumer signals producer to slow down when overwhelmed
WOW FACTOR
• LinkedIn built Kafka — original use case was tracking user activity events at scale • CDC is how you keep Elasticsearch in sync with PostgreSQL without dual-writes • "Log is the database": Kafka's persistent log is itself a source of truth — replay to rebuild any derived store

System Design — Core Concepts
Explain Idempotency, API Design best practices, and WebSockets vs REST
CLARIFY FIRST
API design decisions affect reliability and client experience at scale.
IDEMPOTENCY
Definition: calling the same operation multiple times has the same effect as calling it once. Critical for: payments, order creation, anything that can be retried Implementation: • Client sends Idempotency-Key header (UUID per request) • Server stores key → response in Redis/DB with TTL • On retry with same key: return cached response, don't re-execute • GET, HEAD, PUT, DELETE are naturally idempotent. POST is NOT — must add idempotency key.

REST vs GraphQL vs gRPC
REST: • Simple, cacheable, stateless • Over-fetching: client gets fields it doesn't need • Under-fetching: multiple calls to get related data • Use for: public APIs, mobile clients, standard CRUD GraphQL: • Client specifies exactly what fields it needs • Single endpoint, flexible queries • Harder to cache, schema overhead • Use for: complex nested data, multiple client types (mobile/web) with different needs gRPC: • Binary protocol (Protocol Buffers), low latency • Bi-directional streaming • Use for: internal microservice communication, real-time data, IoT
WEBSOCKETS vs REST
REST: request/response, stateless, client initiates WebSocket: persistent TCP connection, bidirectional, server can push Use WebSocket for: live chat, real-time collaboration, live sports scores, stock tickers Alternatives: • Long Polling: client polls repeatedly, server holds until data ready — simple but inefficient • Server-Sent Events (SSE): one-way push from server, simpler than WebSocket
RATE LIMITING
Token bucket: X tokens replenish/sec, each request costs 1 token — handles bursts Sliding window: count requests in rolling window — more accurate Fixed window: simple counter per time window — susceptible to boundary bursts

WOW FACTOR
• Stripe uses idempotency keys on every payment API — prevents double-charge on retry • HTTP/2 multiplexing: multiple requests over one connection — makes REST competitive with gRPC for web • Redis SETNX (set-if-not-exists) is the primitive for distributed idempotency key storage

Explain Fault Tolerance, Failover, and Disaster Recovery

CLARIFY FIRST

How to design systems that stay up when things go wrong — and things always go wrong.

KEY CONCEPTS

Single Point of Failure (SPOF): any component whose failure takes down the whole system
→ Solution: eliminate SPOFs by adding redundancy at every layer
Fault Tolerance: system continues operating (possibly degraded) despite failures
Fault Isolation: failures don't cascade — one failing service doesn't kill others
High Availability (HA): expressed as uptime percentage:

- 99% = 87.6 hrs downtime/year
- 99.9% = 8.76 hrs/year
- 99.99% = 52.6 min/year
- 99.999% = 5.26 min/year (five nines)

REDUNDANCY PATTERNS

Active–Passive: primary handles traffic, secondary on standby — fast failover, wastes resources
Active–Active: both handle traffic, load balanced — better utilization, more complex
Multi–region: replicas across geographic regions — survives data center outage
Multi–AZ: replicas across availability zones — survives rack/power failure

FAILOVER STRATEGIES

Health checks: load balancer polls /health endpoint every 5s
Circuit breaker: after N failures, stop calling failing service for X seconds
Retry with exponential backoff: retry(1s) → retry(2s) → retry(4s) → DLQ
Graceful degradation: return cached/stale data instead of error
Bulkhead: isolate failures — thread pool per dependency, one slow DB can't block all requests

DISASTER RECOVERY

RPO (Recovery Point Objective): max data loss acceptable ("we can lose up to 1 hour of data")
RTO (Recovery Time Objective): max downtime acceptable ("we must be back in 15 minutes")
Backup strategies: full backup daily, incremental hourly, WAL streaming for near–zero RPO

WOW FACTOR

- Netflix Chaos Engineering: intentionally kill random servers in production — if you don't test failure, it will surprise you
- "Design for failure" principle: assume every network call fails, every disk dies
- AWS S3: 11 nines (99.999999999%) durability — achieved by storing objects across 3+ AZs with erasure coding

Cloud Patterns

System Design · 6 cards

Explain Circuit Breaker, Bulkhead, and Retry patterns

CIRCUIT BREAKER

Problem: one slow service causes cascading failures across the whole system

- CLOSED: requests flow normally, count failures
- OPEN: after N failures, stop all requests to that service immediately
- HALF–OPEN: after timeout, let one request through to test recovery
- Real example: Netflix Hystrix, Resilience4j

BULKHEAD

Problem: one slow service exhausts all threads → crashes everything else

- Isolate services into separate thread pools (like ship bulkhead compartments)
- Service A gets 20 threads, Service B gets 20 threads — B failure can't starve A
- Apply to: DB connection pools, HTTP client pools

RETRY WITH EXPONENTIAL BACKOFF

Problem: transient failures — don't retry immediately (thundering herd)

- Retry after: 1s → 2s → 4s → 8s → give up
- Add jitter (random delay): prevents all clients retrying in sync
- Only retry idempotent operations (GET, PUT with idempotency key)

TIMEOUT (the often–forgotten one)

- Always set timeouts — never wait forever for a dependency
- Fail fast: better to return error quickly than hang all resources

WOW FACTOR

- Use ALL THREE together: timeout → retry → circuit breaker → bulkhead
- Without them: 1 bad microservice takes down your entire platform
- Chaos engineering (Netflix's approach): deliberately break things to find gaps

Explain the Saga Pattern for distributed transactions

THE PROBLEM

Distributed transactions across multiple services — you can't use traditional ACID.
A two–phase commit (2PC) is blocking and fragile in microservices.
SAGA PATTERN
Break a distributed transaction into a sequence of local transactions.
Each step publishes an event that triggers the next.
If any step fails → run compensating transactions in reverse.

EXAMPLE: E-commerce Order

✓ Place Order → Charge Payment → Reserve Inventory → Ship → Notify
If inventory fails:
Cancel notification ← Release inventory ← Refund payment ← Cancel order
TWO SAGA STYLES

Choreography (event–driven, no central coordinator):

- Each service listens for events and reacts
- Pro: loosely coupled, simple for small flows
- Con: hard to track overall state, debugging complex

Orchestration (central saga coordinator):

- Saga Orchestrator tells each service what to do next
- Pro: clear flow, easy to monitor, centralized error handling
- Con: orchestrator can become a bottleneck

WHEN TO USE

- Multi–service transactions: order + payment + inventory + notification
- Long–running business processes where eventual consistency is acceptable

WOW FACTOR

- Combine with idempotency keys: safe to retry any step without side effects
- Event sourcing: Saga events become your audit log automatically
- Outbox pattern: write event to DB and queue atomically → no lost messages

System Design — Cloud Patterns

What is the difference between horizontal and vertical scaling? When do you use each?

VERTICAL SCALING (Scale Up)

- Give one machine more CPU, RAM, faster disk
- Pro: simple — no code changes, no distributed system complexity
- Con: hard limit (biggest machine available), single point of failure
- Use: start here first. Always try vertical before horizontal.

HORIZONTAL SCALING (Scale Out)

- Add more machines, distribute load across them
- Pro: theoretically unlimited scale, no single point of failure
- Con: app must be stateless, need load balancer, distributed system complexity
- Use: when vertical hits its limit, or for geographic distribution

STATELESS DESIGN (requirement for horizontal scaling)

- No user session stored on server — store in Redis or JWT
- Any server can handle any request → LB can route freely

TOOLS FOR HORIZONTAL SCALING

- Load Balancer: Round-robin, least connections, IP hash for stickiness
- Auto-scaling groups: spin up/down instances based on CPU/request metrics
- Read replicas: horizontally scale reads from DB
- Sharding: horizontally scale DB writes (last resort)

System Design — Cloud Patterns

p.247

REAL DECISION FRAMEWORK

- Is the app stateless? (If no, fix this first)
- Can we cache more aggressively? (Often solves 80% of load)
- Can we add a read replica? (Solves read-heavy workloads)
- Vertical scale the DB? (Usually cheaper than sharding)
- Then horizontal scale app servers + shard DB

WOW FACTOR

- Cattle vs Pets: treat servers as replaceable cattle, not named pets
- Immutable infrastructure: replace servers on deploy, never patch in-place
- Auto-healing: health checks + auto-replace unhealthy instances

System Design — Cloud Patterns

p.248

System Design — Cloud Patterns

Common System Design Mistakes to Avoid

MISTAKE 1: Jumping into solution before clarifying

"I'll design this as a microservices system with Kafka..."

Fix: Always ask 2-3 clarifying questions first. DAU? QPS? Read vs write heavy? Global or regional?

MISTAKE 2: Not driving the conversation

Interviewer is asking all the questions, you're just answering.

Fix: You lead. Say "Let me start with requirements, then data model, then architecture."

MISTAKE 3: Generic answers with no concrete trade-offs

"We'll use a database and a cache."

Fix: "We'll use Cassandra because it's write-heavy time-series data, partitioned by (user_id, month). Redis for hot read cache."

MISTAKE 4: Stubbornly defending your design

Interviewer suggests an alternative → you dismiss it.

Fix: "That's a great point — if we used X instead, it would give us Y but we'd lose Z. For this use case, I'd still prefer A because..."

MISTAKE 5: Skipping capacity estimation

Jumping to architecture without knowing if you need 100 QPS or 1M QPS.

System Design — Cloud Patterns

p.249

Fix: 2 minutes of back-of-envelope math. TB vs GB? Millions vs thousands of QPS?

MISTAKE 6: Not mentioning failure scenarios

Only designing for the happy path.

Fix: "What happens if this service goes down? We'd add a circuit breaker + retry with backoff."

MISTAKE 7: Ignoring schema design

Handwaving "we'll store data in a database"

Fix: Explicitly state the key tables/documents with important columns.

Interviewers judge architecture by data model quality.

WOW FACTOR

- Show adaptability: "I'd choose X, but Y is also valid if the requirements shift toward..."
- Reference real systems: "Twitter had this exact problem and solved it with hybrid fan-out"

System Design — Cloud Patterns

p.250

System Design — Cloud Patterns

Cloud Design Principles — all 8 you must know

1. FAIL FAST

Return error immediately rather than hanging. Timeout aggressively.
"A request that fails in 100ms is better than one that hangs for 30 seconds"

2. DESIGN FOR FAILURE

Assume EVERY component will fail. Build with retries, circuit breakers, bulkheads. No single points of failure. Multi-AZ deployments.

3. IMMUTABLE INFRASTRUCTURE

Never patch servers in-place. Replace them. Use containers/AMIs/images. Eliminates configuration drift. "Snowflake servers" are a liability.

4. CATTLE vs PETS

Pets: named servers you care about (db01, web-prod) — you nurse them when sick

Cattle: numbered, replaceable, auto-replaced on failure (pod-7842 dies → new one spins up)

Design for cattle.

5. AUTO HEALING

Health checks on every service. Auto-restart/replace on failure. Kubernetes liveness probes. Load balancer health checks. Auto Scaling Groups.

6. ASYNC PROGRAMMING

Non-blocking I/O. Message queues for decoupling services. Never block on I/O. Event-driven > synchronous request chains.

7. GITOPS

Infrastructure as code (Terraform, CloudFormation). Version controlled. Reviewed. Audited.

No manual changes. Every change = a pull request. Full audit trail.

System Design — Cloud Patterns

p.251

8. EVENT-DRIVEN ARCHITECTURE

Services communicate via events, not direct calls. Producer doesn't know about consumers. Loose coupling. Example: OrderPlaced event → InventoryService, EmailService, AnalyticsService all react independently.

WOW FACTOR

- Chaos engineering: deliberately break things (Netflix Chaos Monkey) to find real weaknesses
- Observability: logs + metrics + traces (the three pillars). If you can't observe it, you can't fix it.

System Design — Cloud Patterns

p.252

System Design — Cloud Patterns

Cloud Design Patterns — Ambassador, Cache-Aside, Gateway Aggregation, Priority Queue, Strangler Fig

AMBASSADOR

Helper sidecar service handles cross-cutting concerns beside the main service.

- Main service focuses on business logic
- Ambassador handles: retries, circuit breaking, monitoring, TLS, service discovery
- Example: Envoy proxy as sidecar in Kubernetes / Istio service mesh
- Use: when you want to add network capabilities without changing app code

CACHE-ASIDE (Lazy Loading)
App manages cache explicitly:
1. Check cache → HIT → return
2. MISS → fetch from DB → write to cache → return

- App controls what gets cached and when
- Use: general caching, when you need fine-grained control over cache population

GATEWAY AGGREGATION
Client needs data from 5 microservices → Gateway makes 5 calls → returns 1 combined response

- Reduces chattiness between client and backend
- Client makes 1 request instead of 5 → less latency on mobile
- Gateway can cache, transform, and merge responses
- Use: BFF (Backend For Frontend) pattern, mobile API layers

System Design — Cloud Patternsp.253

PRIORITY QUEUE

Different SLAs for different request types → separate queues per priority level

- Premium users: dedicated high-priority queue, processed first
- Free users: standard queue
- Critical alerts: separate queue never gets starved by bulk operations
- Implementation: multiple Kafka topics or SQS queues with different consumer ratios

STRANGLER FIG PATTERN
Incrementally replace a legacy monolith with microservices:
1. Put a facade/proxy in front of the monolith
2. Route new features to new microservices
3. Gradually migrate existing features one at a time
4. Old monolith "strangles" and eventually disappears

- Use: migrating legacy systems without big-bang rewrites

WOW FACTOR

- Ambassador in practice: Linkerd/Envoy are Ambassador pattern at scale
- Strangler Fig: how Netflix migrated from monolith to microservices over years

System Design — Cloud Patternsp.254

System Design — Advanced Data Structures

Advanced Data Structures

System Design · 5 cards

System Design — Cloud Patternsp.255

System Design — Advanced Data Structures

Gossip Protocol and Vector Clocks in distributed systems

GOSSIP PROTOCOL

Problem: How do N nodes share state without a central coordinator?
How it works:

- Each node periodically picks K random neighbors and shares its known state
- Recipients merge info and forward to their random neighbors
- Information propagates exponentially: O(log N) rounds to reach all nodes

Properties:

- Decentralized: no single point of failure
- Eventually consistent: all nodes converge on same view
- Resilient: nodes can fail, info still propagates
- Used in: Cassandra (cluster membership), DynamoDB, Redis Cluster

What gets gossiped:

- Which nodes are alive/dead (failure detection)
- Token assignments (which node owns which data range)
- Schema changes, configuration

VECTOR CLOCKS
Problem: In distributed systems, wall clock time can't determine causation
How it works:

- Each node maintains a vector: [node_A_count, node_B_count, node_C_count]
- On local event: increment own counter
- On message send: attach current vector
- On message receive: merge (take max of each position) + increment own

Example: A=[1,0,0], B=[0,1,0] → concurrent (neither happened before the other)
A=[1,0,0], B=[2,1,0] → B happened after A (B's vector dominates)
Use cases: DynamoDB conflict detection, CRDTs, distributed version control

System Design — Advanced Data Structuresp.256

WOW FACTOR

- Gossip for failure detection: if node not heard from in 3 gossip rounds → suspected dead
- Vector clocks grow with number of nodes → Dynamo uses them selectively
- Physical clocks: Google Spanner uses atomic clocks + GPS for true global ordering

System Design — Advanced Data Structuresp.257

System Design — Advanced Data Structures

CRDTs, Two-Phase Commit (2PC), and Paxos/Raft

System Design — Advanced Data Structuresp.258

CRDTs (Conflict-Free Replicated Data Types)

Problem: How to merge concurrent edits from multiple nodes without coordination?

How it works:

- Special data types designed to always merge deterministically
- G-Counter: each node has its own counter, value = sum of all. Only increments.
- PN-Counter: positive + negative counters → supports increment/decrement
- OR-Set: add-wins set. Each element has unique tag. Remove only removes tagged version.
- Text (RGA): sequence of characters with position identifiers → Google Docs style

Use cases: Figma (collaborative design), Notion, Redis CRDT, shopping carts

TWO-PHASE COMMIT (2PC)

Problem: How to ensure all nodes in a distributed transaction commit or all rollback?

Phase 1 (Prepare):

- Coordinator asks all participants: "Can you commit?"
- Each participant locks resources, writes to WAL, votes Yes or No

Phase 2 (Commit/Abort):

- If ALL voted Yes → Coordinator sends Commit to all
- If ANY voted No → Coordinator sends Abort to all

Problems:

- Blocking: if coordinator crashes after Phase 1 → participants stuck holding locks
- Use Paxos/Raft for coordinator election + recovery

PAXOS / RAFT (Consensus Protocols)

Problem: How do distributed nodes agree on a single value despite failures?

Raft (simpler to understand):

- Elect one leader → leader handles all writes
- Leader sends log entries to followers → majority must ack before commit
- On leader failure → election: node with most up-to-date log wins
- Used in: etcd (Kubernetes), CockroachDB, TiKV

WOW FACTOR

- Raft vs Paxos: same guarantees, Raft designed to be understandable (Diego Ongaro's thesis)
- 2PC vs Saga: 2PC = synchronous, locks. Saga = async, compensating. Saga preferred in microservices

System Design — Advanced Data Structures

Geohash and Quadtree for spatial/location-based systems

GEOHASH

Problem: How do you index and query by geographic location efficiently?

How it works:

- Encode latitude/longitude into a base32 string
- Longer string = smaller area (higher precision)
- Nearby locations share a common prefix

Examples:

- "dp3" = large area around Washington DC area
- "dp3wjz" = specific city block in DC
- Precision 5 chars = ~4.9km × 4.9km area

Use cases:

- Find nearby drivers: GEORADIUS in Redis (uses geohash internally)
- Yelp: find restaurants near user's location
- Partitioning: assign cells to servers based on geohash prefix

Limitation:

- Edge case: two points very close but different geohash prefixes (cross boundary)
- Fix: check 8 surrounding cells as well as target cell

QUADTREE

Problem: Efficiently index 2D space for arbitrary density distributions

How it works:

- Start with bounding rectangle covering entire area
- If a region has more than N points → subdivide into 4 quadrants
- Repeat until each leaf has ≤ N points (e.g. N=100 for Uber drivers)
- Variable cell sizes: dense cities = small cells, rural = large cells

Operations:

- Point query: O(log N) — traverse tree from root
- Range query: traverse all overlapping quadrants

Use cases: Uber driver matching, game collision detection, Google Maps, Yelp

WOW FACTOR

- Geohash vs Quadtree: geohash = fixed grid, quadtree = adaptive to density
- S2 Geometry (Google): spherical geometry library — handles Earth's curvature properly
- Uber uses S2: divides sphere into cells, handles poles and date line correctly

System Design — Advanced Data Structures

Skip List, Ring Buffer, Reservoir Sampling, and Segment Tree

SKIP LIST

- Sorted linked list with express lanes at multiple levels
- Search: start at highest level, drop down when overshoot → O(log N)
- Alternative to balanced BST — simpler to implement concurrently
- Redis ZADD (sorted sets) uses skip list internally
- Use: leaderboards, ordered indexes, priority queues

System Design — Advanced Data Structures

RING BUFFER (Circular Buffer)

- Fixed-size array with head and tail pointers that wrap around
- Producer writes at tail, consumer reads from head
- When full: overwrite oldest (or block, depending on use case)
- O(1) read and write, zero allocation after init
- Use: network I/O buffers, log ringbuffers (Disruptor), audio/video streaming

RESERVOIR SAMPLING

Problem: Sample K items from a stream of unknown size N, each with equal probability

- Initialize reservoir with first K items
- For item i (i > K): keep with probability K/i
- If kept: replace a random item in reservoir
- Result: each of N items has exactly K/N probability of being in final sample
- Use: sample from huge logs, A/B test random user selection, database

TABLESAMPLE

SEGMENT TREE

- Binary tree where each node stores aggregate (sum, min, max) of a range
- Leaf nodes: individual elements
- Internal nodes: aggregate of children
- Range query (e.g. sum of indices 3-7): O(log N)
- Range update: O(log N) — lazy propagation
- Use: leaderboards (rank of score in range), scheduling (free time slots), analytics dashboards

WOW FACTOR

- Segment tree vs BIT (Binary Indexed Tree/Fenwick): BIT simpler, only prefix sums. Segment tree handles any range.
- Ring buffer in kernel: Linux kernel uses ring buffers for network packet queues

System Design — Tradeoffs
Stateful vs Stateless Design — tradeoffs and when to use each
CLARIFY FIRST
Statefulness vs statelessness affects scalability, fault tolerance, and routing complexity.
STATELESS SERVICES
No per-user state stored in the server process itself. Every request contains all information needed (or state is fetched from shared store). Pros:
Any server can handle any request → easy horizontal scaling
Server failures don't lose user data (state is in DB/cache)
Simple load balancing (round-robin works)
Easy to deploy, roll back, restart
Cons: every request must fetch state from DB/cache → latency + load Examples: REST APIs, static file servers, stateless microservices
STATEFUL SERVICES
Server holds per-user state in memory between requests. Pros:

No DB roundtrip for state → lower latency
Rich session context (WebSocket connections, game state)
Cons: "Sticky sessions" required — same user must hit same server Server crash = lost state Harder to scale — can't freely add servers Rolling deploys break sessions Examples: WebSocket servers, multiplayer game servers, live video encoding pipelines
HYBRID APPROACH (most real systems)
Stateless application layer + stateful data layer: • App servers: stateless (scale freely) • Redis: shared state (session data, counters, rate limits) • PostgreSQL: durable state (orders, users) • Kafka: event state (message log)
SESSION MANAGEMENT
JWT tokens: stateless auth — server validates signature, no DB lookup Server-side sessions: stored in Redis, session ID in cookie — revocable
WOW FACTOR
• Kubernetes prefers stateless — StatefulSets are a workaround for databases • "Share nothing" architecture: each server is fully independent — Pinterest's model for massive scale • WebSocket gateway: one stateful layer at the edge, everything behind it stateless

System Design — Tradeoffs
Strong vs Eventual Consistency — CAP theorem in practice
CLARIFY FIRST
The most important tradeoff in distributed systems. Know when you need strong vs eventual.
STRONG CONSISTENCY
After a write completes, all reads return the new value immediately. Every node sees the same data at the same time. Cost: latency (must wait for all replicas to confirm) + reduced availability during partitions Examples: PostgreSQL (single-node), Google Spanner, CockroachDB, Zookeeper Use for: bank balances, inventory (don't oversell), booking systems, user auth
EVENTUAL CONSISTENCY
Replicas will converge to the same value eventually — but reads may be stale temporarily. "Given enough time with no new writes, all replicas will agree." Cost: temporary inconsistency that application must handle Examples: Cassandra, DynamoDB, S3, DNS, social media feeds Use for: social media likes/views, shopping carts (add item locally, sync later), DNS propagation, player scores in games, search index updates

CAP THEOREM APPLIED
You can only pick 2 of 3: • Consistency + Partition Tolerance (CP): ZooKeeper, HBase — unavailable during network partition • Availability + Partition Tolerance (AP): Cassandra, DynamoDB — returns stale data but stays up • CA: traditional RDBMS — not partition-tolerant (single node) PACELC: extends CAP — even without partitions, tradeoff between latency (L) and consistency (C)
PRACTICAL PATTERNS
Read-your-writes consistency: after a user writes, they see their own write (even if others see stale) → Route user's reads to the same replica they wrote to (using sticky sessions or user-hash routing) Monotonic reads: user never sees older data than they've already seen → Route all reads for a user to same replica
WOW FACTOR
• Amazon's Dynamo paper (2007): foundational — chose AP over CP, introduced vector clocks • "Conflict-free" data types (CRDTs): mathematical structures that merge concurrent edits without conflicts — Riak, Redis • Cassandra's tunable consistency: set W (write quorum) + R (read quorum); if W+R > N = strong consistency

System Design — Tradeoffs
Synchronous vs Asynchronous Communication — when to use each
CLARIFY FIRST
One of the most impactful architectural decisions — affects latency, reliability, and coupling.
SYNCHRONOUS (REQUEST/RESPONSE)
Caller waits for response before continuing. Examples: REST API calls, gRPC, database queries Pros:
Simple mental model — call, get response, continue
Immediate feedback — errors surface instantly
Easy to debug and trace
Cons: Caller blocked until response arrives Caller inherits callee's downtime (tight coupling) Cascade failures — slow service slows everything upstream Timeout tuning is tricky Use for: user-facing APIs (need result to render page), auth checks, read queries
ASYNCHRONOUS (EVENT/MESSAGE)
Caller publishes message and continues immediately. Consumer processes later. Examples: Kafka, SQS, RabbitMQ, email systems Pros:

Caller not blocked — better throughput
Loose coupling — services independent
Natural backpressure — queue absorbs traffic spikes
Consumer failure doesn't affect producer
Cons: No immediate result — harder UX patterns ("we'll email you when done") Message ordering and exactly-once delivery complexity Harder to debug — distributed traces needed Use for: sending emails/notifications, order processing, video transcoding, search index updates, analytics events
PRACTICAL PATTERNS
Async with callback: producer sends job, consumer calls webhook when done Async with polling: producer returns job_id, client polls GET /jobs/{id} Hybrid: synchronous for user response, async fan-out for side effects
COMMON MISTAKE
Everything synchronous → slow chain: user request triggers 6 sync service calls → 200ms becomes 1.2s
WOW FACTOR
• Amazon's order pipeline: payment sync (need to know immediately), fulfillment async (kick off after) • The "two generals problem": proves 100% reliable synchronous communication over unreliable network is impossible • Event-driven + CQRS: writes go async via events; reads from pre-built read models — massive read scalability

System Design — Tradeoffs

Concurrency vs Parallelism, and Load Balancing strategies

CLARIFY FIRST

Often confused in interviews — know the precise difference and practical implications.

CONCURRENCY vs PARALLELISM

Concurrency: dealing with multiple tasks at once (may not execute simultaneously)
→ Single CPU switching between tasks (interleaving)
→ Example: Node.js event loop — single thread handles thousands of connections via async I/O
Parallelism: actually executing multiple tasks simultaneously
→ Multiple CPUs/cores running at the same time
→ Example: Java thread pool processing requests on 8 CPU cores simultaneously
"Concurrency is about structure, parallelism is about execution" — Rob Pike
When each matters:
I/O-bound work (DB queries, network calls): concurrency wins → use async/event loop or goroutines
CPU-bound work (image processing, ML inference): parallelism wins → use thread pool or multiple processes

System Design — Tradeoffs

p.277

Behavioral — Achievement

Achievement

Behavioral · 2 questions

System Design — Tradeoffs

p.280

LOAD BALANCING STRATEGIES

Round Robin: distribute requests sequentially across servers
→ Simple, works when requests are roughly equal cost
Weighted Round Robin: servers with more capacity get proportionally more requests
→ Good for heterogeneous fleet
Least Connections: route to server with fewest active connections
→ Best for variable-length requests (some requests take 100ms, others 5s)
IP Hash / Sticky Sessions: same client always hits same server
→ Required for stateful services; breaks if server dies
Least Response Time: route to fastest responding server
→ Most dynamic, adapts to real performance but needs measurement overhead

LAYER 4 vs LAYER 7

L4 (Transport): routes by IP + port — fast, no content inspection (HAProxy, AWS NLB)
L7 (Application): routes by URL, headers, cookies — smarter, enables A/B testing (Nginx, AWS ALB)

HEALTH CHECKS

Active: load balancer sends /health every 5s, removes unresponsive servers
Passive: monitor real traffic error rates, remove servers exceeding threshold

WOW FACTOR

- Go's goroutines: 2KB stack vs Java threads 1MB — can run 1M concurrent goroutines on one machine
- "C10K problem" (1999): how to handle 10K concurrent connections — solved by async I/O (Node.js, Nginx)
- Consistent hashing for load balancing: adding/removing servers only remaps ~1/N of keys

System Design — Tradeoffs

p.278

Behavioral — Achievement

Q40. What's the number one accomplishment you're most proud of?

Story 1: Ericsson Kafka Pipeline — 400K Events

S — Situation

Building the Kafka-streaming microservice at Ericsson that processes 400K+ daily events across 6 topics with sub-200ms latency is the technical accomplishment I'm most proud of.

T — Task

It replaced manual SQL workflows, required solving real distributed systems problems under production SLAs, and was fully shipped within an internship window.

A — Action

I designed the consumer group topology, tuned partition assignment and heartbeat intervals under load, built load tests at 2x production volume, found and fixed a rebalancing bug two weeks before launch, and built a React dashboard to make the system observable.

R — Result

400K+ daily events at sub-200ms latency, replacing an entire class of manual work for the team. It's the accomplishment I'm most proud of technically because it required understanding distributed systems at a depth that courses don't teach — only production experience does.

Story 2: MedStroke — 35% Faster Stroke Treatment

Behavioral — Achievement

p.281

Part 3

Behavioral Q&A

63 questions · 188 STAR stories · 19 categories

S — Situation

Piloting MedStroke AI at a Louisiana hospital and watching it reduce time-to-treatment by 35% is the accomplishment I'm most proud of from an impact perspective.

T — Task

Time-to-treatment in stroke care directly affects patient outcomes — faster treatment means less brain damage. Building something that moved that number is a different kind of impact than shipping a feature.

A — Action

I built the full system — ML pipeline, FastAPI backend, React frontend, NIHSS scoring, tPA eligibility automation — and managed the hospital stakeholder relationship through a genuinely skeptical clinical team.

R — Result

35% faster time-to-treatment. The accomplishment I'm most proud of isn't the technical build — it's that doctors who were skeptical of AI tools actually adopted it and changed their workflow. That's harder to achieve than any engineering challenge.

Story 3: NSBE — 300+ Member Community

S — Situation

Growing NSBE at Grambling from low attendance to 300+ active members and \$3K in corporate partnerships is the accomplishment I'm most proud of in terms of community impact.

Behavioral — Achievement

p.282

T — Task

Engineering communities don't grow automatically — someone has to do the unglamorous work of outreach, event planning, and relationship building.

A — Action

I restructured the event calendar around real member needs, built a mentorship system, personally cold-contacted 12 companies for sponsorships, and made every member placement and offer visible and celebrated within the chapter.

R — Result

300+ active members. \$3K in partnerships. Students getting access to opportunities they wouldn't have found otherwise. I'm proud of it because it required skills I had to develop — public speaking, sales, community organising — and the impact is compounding. Those students will pay it forward.

T — Task

The feedback was largely positive but pointed at one real gap: I sometimes over-prepared for meetings to the point where I didn't leave enough room for other voices.

A — Action

I use both the positive and the critical feedback as my template. I hope a first annual review would say: I grew the team's capabilities, I showed up consistently and reliably, and I made the community or product measurably better in at least one concrete way. I'd also want you to be honest if there's something I over-indexed on — over-preparation in my case means I need to work on creating space for the people around me, not just filling it.

R — Result

The honest answer is: I'd want to hear that I delivered on my commitments, raised the bar on something specific, and showed the team I could be trusted with more. And I'd want to hear the one thing I should do differently in year two.

Behavioral — Achievement

Q57. If this were your first annual review, what would I be telling you right now?

Story 1: The Ericsson Internship Review

S — Situation

My Ericsson internship ended with a formal evaluation. My manager told me I had exceeded expectations on delivery, communication, and initiative — and he was specific about each.

T — Task

I use that feedback as a benchmark for what I aim to hear in any first-year review.

A — Action

On delivery: I shipped the Kafka microservice processing 400K+ daily events at sub-200ms latency on schedule, and the event-driven pricing system and React dashboard alongside it. On communication: I flagged blockers early, wrote clear PRs, and never surprised my manager with a missed deadline. On initiative: I built the React dashboard without being asked, which he described as one of the most impactful contributions of the internship cycle.

R — Result

He told me I was one of the strongest interns he'd worked with and that the team wanted me back. I hope at a first annual review you'd be telling me something similar: that I shipped reliably, that I made the team around me faster, and that I took ownership of something I wasn't assigned to fix.

Story 2: The MedStroke Leadership Review

Behavioral — Background

Background

Behavioral · 1 question

S — Situation

MedStroke AI was a project I led from idea to hospital pilot. There was no manager reviewing me — the review was the pilot outcome itself and the team's response to how I led the build.

T — Task

I think the review my teammates would have given me if they were asked was: strong technical execution, good at making hard calls under time pressure, sometimes needed to delegate earlier.

A — Action

I owned the ML pipeline, the FastAPI backend integration, and the clinical framing strategy. I made the call to delay the pilot by four weeks to improve accuracy, which the team initially pushed back on but later credited as the right decision. The area where I grew most was learning to bring the team along on hard decisions rather than presenting them as done deals.

R — Result

In a first annual review at this company, I'd hope you'd say I shipped reliably, I grew in my ability to collaborate on difficult tradeoffs, and I identified at least one problem that wasn't on anyone's radar and fixed it. That's what I'm aiming to earn.

Story 3: The NSBE Leadership Review

S — Situation

As NSBE Assistant VP I ran a chapter of 300+ members and secured \$3K in corporate partnerships. At the end of the year the outgoing board gave me informal feedback on how the chapter had grown.

Behavioral — Background

Q1. Tell me about yourself.

Story 1: The Engineer Builder

S — Situation

I'm Emmanuel Oppong, a Computer Science student at Grambling State University with a 3.93 GPA, graduating May 2028.

T — Task

My goal has been to build real engineering skills — not just classroom theory — through internships, open source, and personal projects.

A — Action

At Ericsson I engineered a Kafka microservice processing 400K+ daily events with sub-200ms latency. Before that at Noon in Abu Dhabi I built an IoT pipeline reducing downtime detection by 40%. Outside work I contribute to facebook/react, apache/kafka, pgjdbc, and kubernetes-client/java — with merged PRs in two of those repos.

R — Result

I'm a full-stack engineer comfortable from low-level C and Go all the way up to React dashboards. I'm looking for a role where I can keep shipping real systems at scale.

Story 2: The Impact Maker

S — Situation

I'm a software engineer and CS student at Grambling State (3.93 GPA, Dean's List, Presidential Scholarship) who has shipped production systems across two countries.

T — Task

Everything I build has to create measurable impact — not just work, but move the needle.

A — Action

At Ericsson I built a React dashboard that cut debugging time from 60 minutes to 10 minutes for a 30-engineer team. I also built MedStroke AI, a stroke diagnosis system piloted at a Louisiana hospital that reduced time-to-treatment by 35%. As NSBE Assistant VP I grew our chapter to 300+ members and secured \$3K in company partnerships.

R — Result

Whether it's developer tooling, healthcare AI, or community building — I focus on outcomes. I'd love to bring that mindset to your team.

Story 3: The Lifelong Learner

S — Situation

I'm Emmanuel — a CS student, open source contributor, and engineer who learns fastest by building hard things from scratch.

T — Task

I believe the best way to master a concept is to implement it yourself.

Behavioral — Collaboration

Q26. Give an example of a time you had to collaborate effectively with a team from a different department.

Story 1: MedStroke with Medical Staff

S — Situation

Building MedStroke AI required deep collaboration with the medical staff at the Louisiana hospital — neurologists and nurses who had no software engineering background and were skeptical of AI tools.

T — Task

I needed clinical input to design the NIHSS scoring interface and tPA eligibility logic correctly, but communicating across the technical-clinical divide was genuinely hard.

A — Action

I started by listening rather than presenting. I shadowed a clinician through one patient assessment to understand their actual workflow. Then I translated every technical question into clinical terms: not 'what data fields do you need' but 'what information do you look at first when a stroke patient comes in?' I created paper mockups before building anything and validated with the medical team before writing code.

R — Result

The tool fit into the clinical workflow naturally because it was built around that workflow. Time-to-treatment reduced by 35%. Cross-functional collaboration works when the technical team does the work of translation — not the other way around.

Story 2: NSBE with University Administration

A — Action

I built a container runtime in Go using Linux namespaces and cgroups, and a SQLite-clone database engine in C with B-tree indexing — both from scratch to deeply understand systems. I then applied that foundation professionally at Ericsson building Kafka microservices and at Noon building IoT data pipelines.

R — Result

I come in already knowing the internals of the tools we use, which lets me debug faster, design better, and ship with confidence. I'm excited to keep building at that depth with your team.

S — Situation

To secure room bookings, co-sponsorship from the university, and access to career fair infrastructure, NSBE needed to collaborate with university administration — a department with very different incentives and communication styles.

T — Task

I needed to negotiate partnerships and access that served our members without creating friction with admin processes.

A — Action

I learned the university's processes before making requests. I brought documentation, showed how our events aligned with the university's diversity and retention goals, and always gave admins enough lead time. I treated it as a partnership, not a transaction — checking in after events with attendance numbers they could use in their own reporting.

R — Result

We built a reliable working relationship with administration that made future events smoother to organise. They became proactive partners rather than gatekeepers. The key insight: administrative departments are easier to work with when you understand their constraints and goals.

Story 3: Ericsson Cross-Functional Pipeline Work

S — Situation

At Ericsson, the Kafka pipeline I was building needed to integrate with an infrastructure team managing the Kubernetes deployment environment — a team I had no direct relationship with.

Behavioral — Collaboration

Collaboration

Behavioral · 2 questions

T — Task

I needed configuration support, environment access, and their buy-in on our deployment approach — without any formal reporting line.

A — Action

I requested a short sync early rather than late, came prepared with specific questions and a clear diagram of what I needed, and framed every request around reducing their work — 'here's the config I need, here's the format it needs to be in, and here's what I'll own vs what I need from you.' I also documented everything so they didn't need to chase me for context.

R — Result

The infrastructure team flagged our Kubernetes resource limits were too low before deployment — something I'd have discovered in production. That early collaboration caught a real problem. Cross-team work is easier when you make it easy for the other team.

Behavioral — Background

p.291

T — Task

I needed configuration support, environment access, and their buy-in on our deployment approach — without any formal reporting line.

A — Action

I requested a short sync early rather than late, came prepared with specific questions and a clear diagram of what I needed, and framed every request around reducing their work — 'here's the config I need, here's the format it needs to be in, and here's what I'll own vs what I need from you.' I also documented everything so they didn't need to chase me for context.

R — Result

The infrastructure team flagged our Kubernetes resource limits were too low before deployment — something I'd have discovered in production. That early collaboration caught a real problem. Cross-team work is easier when you make it easy for the other team.

Q32. Have you ever been in a situation where another team and yours were creating a similar product? What happened?

Story 1: Duplicate PR in Open Source

S — Situation

While working on a fix for a pgjdbc issue, I discovered that another contributor had opened a PR solving the same problem two days before me — with a slightly different approach.

T — Task

I needed to decide whether to close my PR, continue it, or find a path that incorporated the best of both.

A — Action

I read their implementation carefully, identified one case my approach handled better (connection timeout edge case), commented on their PR pointing it out constructively, and offered to either contribute that fix to their PR or merge our approaches. I wasn't attached to my PR getting the credit.

R — Result

The maintainer suggested I add my edge case handling to their PR, which I did. It merged with both our contributions credited. Open source collaboration works best when you're optimising for the best outcome, not whose name is on the commit.

Story 2: Parallel Pipeline Work at Ericsson

S — Situation

At Ericsson, I discovered midway through my project that another team had been building a separate data ingestion pipeline with partial overlap in functionality with mine.

T — Task

We were duplicating infrastructure effort and potentially creating competing systems that would both need maintenance.

A — Action

I flagged the overlap to my manager and requested a sync with the other team. We mapped out what each pipeline did and found the overlap was in the ingestion layer — both were consuming from similar upstream sources. We agreed to standardise on shared Kafka topics with different consumer groups rather than maintaining two separate ingestion paths.

R — Result

The teams consolidated the ingestion infrastructure, reducing maintenance burden for both. The key was raising the overlap early — a month later it would have been much harder to change. When you see duplication of effort, say something immediately.

Communication

Behavioral · 3 questions

Q24. Describe a time you had to explain a complex technical concept to someone non-technical.

Story 1: Explaining Kafka to Ericsson Stakeholders

S — Situation

At Ericsson, I had to present the Kafka-based pricing system to business stakeholders who needed to understand why we chose event streaming over a database-based approach.

T — Task

These were senior business people — not engineers. They needed to understand the value proposition, not the implementation.

A — Action

I used an analogy: instead of the system checking the price list every few minutes like a store clerk asking 'anything changed?', Kafka was like having a manager shout every price change the instant it happened — 400,000 times a day. I focused on what it meant for them: real-time pricing data that used to take hours of manual SQL work now appeared instantly.

R — Result

Stakeholders understood and approved the approach. One said it was the first time a technical presentation had made them feel confident rather than confused. Analogies that tie to business outcomes are the fastest path to non-technical understanding.

Story 2: Explaining ML to Hospital Doctors

S — Situation

When piloting MedStroke AI, I needed to explain to neurologists how the Random Forest model made predictions — doctors are clinically sophisticated but not ML practitioners.

T — Task

I needed them to understand enough to trust and appropriately use the tool — not to distrust it blindly or over-rely on it.

A — Action

I explained it as pattern recognition over thousands of past cases: 'The model has seen 10,000 patients with these blood values, and 73% of them had stroke events within 24 hours. It's showing you the historical pattern, not making a diagnosis.' I then showed the confidence score and explained what low confidence meant in clinical terms.

R — Result

The doctors engaged with the tool critically and constructively — exactly what I wanted. They flagged two cases where their clinical judgment overrode the model, which was correct behaviour. The explanation built appropriate trust, not blind faith.

Story 3: Explaining Containers to NSBE Members

S — Situation

At an NSBE workshop I presented Go Container, my container runtime project. Most attendees were first and second year CS students unfamiliar with Linux internals.

T — Task

I needed to explain Linux namespaces, cgroups, and chroot in a way that made people excited rather than overwhelmed.

A — Action

I used the analogy of a rented apartment: a container is like each tenant getting their own front door, mailbox, and utilities meter — they share the building (the kernel) but can't see into each other's apartments or use more than their allocated electricity (cgroups). I live-coded a simple namespace isolation demo.

R — Result

Three students came up after to ask how to start building systems projects. One joined a Linux kernel study group I mentioned. Making complex technical ideas vivid and concrete is how you get people to actually go build things.

Behavioral — Communication
Q54. What words would your colleagues use to describe you?
Story 1: The Ericsson 30–Engineer Team
S — Situation
At Ericsson I worked on a 30–engineer team where I was the only intern on several key deliverables — the Kafka microservice, the event–driven pricing system, and the React dashboard.
T — Task
I had to establish credibility quickly, communicate clearly about my progress, and deliver work that full–time engineers could build on.
A — Action
I asked my manager directly for feedback midway through the internship. He said I was reliable — when I said something would be done by Thursday, it was done by Thursday. He also said I was proactive: I had built the React dashboard without being asked, just because I saw engineers wasting time. A peer on the team said I communicated blockers early, which she said was the thing most junior engineers failed at.
R — Result
I believe the words my Ericsson colleagues would use are: reliable, proactive, and a clear communicator. Those three things are what I try hardest to be, because they make the people around me faster.
Story 2: NSBE Chapter Feedback
Behavioral — Communicationp.301

Behavioral — Communication
Q63. Tell me about a time you had to deliver difficult or bad news to someone.
Story 1: MedStroke Model Accuracy Miss
S — Situation
Two days before the MedStroke hospital pilot, I had to tell my team that the ML model's accuracy was 45% — significantly below the 80% internal target I had set publicly at the start of the project.
T — Task
I was the one who had set the target and was the most visibly responsible for the miss. I needed to communicate it clearly, take ownership, and give the team a path forward — not just bad news.
A — Action
I called the team together rather than sending an async message. I led with the number: 45%, not 80%. I did not soften it or add qualifiers before they heard it. I then walked through the root cause: the training data was too small and imbalanced, and I had underestimated the complexity of the feature engineering. I presented three options with honest tradeoffs: delay the pilot, proceed with lower accuracy and different framing, or proceed with 45% and be transparent with the hospital. I recommended the third with a specific clinical reframing — present the tool as decision support, not a diagnosis, and show confidence scores alongside predictions.
Behavioral — Communicationp.304

S — Situation
As NSBE Assistant VP I led a chapter of 300+ members, which meant working with an executive board of 8 people and managing relationships with corporate partners who funded our \$3K sponsorship.
T — Task
Leadership at that scale requires different communication than engineering — you have to be consistent, motivating, and honest with people who are giving their time voluntarily.
A — Action
After my term I asked several board members what they'd say about working with me. The word that came up most often was dependable — I showed up to every meeting prepared, I followed through on every commitment I made to the board, and I never passed a problem upward without having thought about a solution first. One member said I was direct but fair, which I took as a high compliment.
R — Result
I think colleagues across both engineering and leadership contexts would say: dependable, direct, and someone who takes problems seriously. I think those are the right qualities to be known for.
Story 3: Open Source Maintainer Feedback
S — Situation
My open source contributions to facebook/react and kubernetes–client/java put me in extended dialogue with maintainers who gave detailed feedback on both my code and how I communicated in PRs.
Behavioral — Communicationp.302

R — Result
The team agreed to proceed with the reframing. The pilot ran and reduced time–to–treatment by 35%. The hospital's feedback was that they trusted the tool more because it was honest about its uncertainty. Delivering bad news clearly and paired with a path forward is almost always better than softening it. The team remembers how you handle a miss more than the miss itself.
Story 2: Noon Pipeline Delivery Slip
S — Situation
At Noon, I was three days away from the sprint commitment for the IoT pipeline integration with PostgreSQL when I hit an unexpected schema compatibility issue that I knew would take at least four more days to resolve.
T — Task
I had committed to a delivery date in front of my manager and the downstream team that was counting on the pipeline to start their work. I needed to tell them I was going to miss it.
A — Action
I told my manager and the downstream team lead the same day I knew — not the day the deadline arrived. I was specific: 'I've found a schema compatibility issue that I expect to take four days to resolve. I'll miss Thursday's commitment and I expect to deliver on Monday.' I then offered a workaround: a stubbed data contract they could use to begin their integration while I resolved the real pipeline. I took full ownership of the slip without attributing it to the schema issue — it was my estimation that was wrong, not the schema's fault.
Behavioral — Communicationp.305

T — Task
In open source, your communication is your entire presence — there are no in–person cues, no body language, only written clarity.
A — Action
A maintainer on pgjdbc commented that my PR descriptions were unusually thorough — he said most contributors just submit code but I explained the why, not just the what. A kubernetes–client maintainer said I was patient and receptive: when they pushed back on my approach, I engaged with their reasoning instead of defending my first instinct. I take pride in that, because I know how it feels to review a PR from someone who's already decided they're right.
R — Result
I think the word that spans all my contexts is thorough. Whether it's code, a PR description, or a plan for a chapter meeting — I try to do the thinking before I show up. That's what I'd want people to say about me.
Behavioral — Communicationp.303

R — Result
The downstream team used the stub and stayed unblocked. The real integration landed Monday, on my revised estimate. My manager later told me the early, honest communication was what he remembered most — not the slip. Delivering bad news early is almost always less damaging than delivering it late. A slip communicated early is a solvable problem; a slip communicated on deadline day is a crisis.
Story 3: NSBE Partnership Rejection
S — Situation
As NSBE Assistant VP I had pitched a mid–sized tech company for a \$5K corporate partnership and led the whole process — the pitch, the follow–up meetings, the proposal. The company ultimately declined and allocated the budget elsewhere.
T — Task
I had to tell the chapter leadership, who had been excited about the partnership, that it wasn't happening — and I had to do it in a way that didn't undermine the chapter's confidence or our momentum on other partnership efforts.
A — Action
I brought it to the executive board directly, not through a message. I told them the outcome first, then the reason: the company's budget had been reallocated due to internal priorities, and it wasn't a reflection of the quality of our pitch. I shared what I had learned about their process that would make any future approach more likely to succeed. I then pivoted immediately to what we were doing next: we had two other partnership conversations active and one of them was looking promising.
Behavioral — Communicationp.306

R — Result

We secured \$3K from another company that semester. The board's response to the rejection was stronger because I didn't dwell on it and because I had already framed a path forward before the meeting ended. Delivering bad news well means leaving the room with the team more focused, not less.

Behavioral — Conflict & Communication

Conflict & Communication

Behavioral · 5 questions

Behavioral — Conflict & Communication

Q2. Tell me about a time you had a disagreement with your manager.

Story 1: Kafka Architecture at Ericsson

S — Situation

At Ericsson my manager initially wanted to use a simple REST polling approach to collect pricing events from multiple upstream services.

T — Task

I believed an event-driven Kafka architecture would be significantly better for our volume requirements — 400K+ daily events across 6 topics — but needed to convince my manager without overstepping.

A — Action

I built a quick proof-of-concept showing the latency difference between REST polling and Kafka streaming under simulated load. I presented the data in a one-on-one, framed it around our team's sub-200ms SLA goal, and acknowledged the tradeoffs in operational complexity.

R — Result

My manager agreed to the Kafka approach. We shipped the microservice hitting sub-200ms latency consistently. The system replaced manual SQL workflows entirely and my manager later credited the architecture decision as key to the project's success.

Story 2: Testing Strategy at Noon

S — Situation

At Noon, my manager felt 15-20 unit tests were sufficient coverage for the new IoT data pipeline we were shipping.

T — Task

I disagreed — the pipeline handled sensor data from physical devices where silent failures are costly, and I wanted to write 50+ tests covering edge cases and concurrent scenarios.

A — Action

I scheduled a quick sync, showed two specific edge cases that existing tests would miss, and proposed a timeline where the extra tests added only two days. I framed it around the 40% downtime-detection improvement we were targeting — tests were the only way to trust those numbers.

R — Result

My manager gave me the green light. I delivered 50+ unit and integration tests using SpringBoot, JUnit, and Postman. Backend reliability improved by 80% and we had zero production incidents during my remaining time there.

Story 3: Open Source PR Scope

S — Situation

While contributing to kubernetes-client/java, a maintainer (effectively acting as my manager in that context) pushed back on the scope of my PR, saying it was too large to review safely.

T — Task

I disagreed because the changes were tightly coupled — splitting them would have broken the logic — but I needed to handle this respectfully since it's their codebase.

A — Action

I wrote a detailed comment explaining the dependency chain between the changes, added inline documentation to make review easier, and offered to pair on a live review call. I also added more granular commit messages to make the diff easier to follow.

R — Result

The maintainer appreciated the extra context and agreed to review it as one PR. It was merged successfully. I learned that disagreement in open source is won with evidence and patience, not persistence.

Behavioral — Conflict & Communication

Q3. Tell me about a situation when you had a conflict with a teammate.

Story 1: Kafka Partitioning Debate at Ericsson

S — Situation

At Ericsson, a fellow engineer and I disagreed on how to partition our Kafka topics. He wanted one partition per topic for simplicity; I wanted multiple partitions per topic to support parallel consumer processing at our scale.

T — Task

We needed to resolve this before the system design was locked in, and both of us felt strongly about our approach.

A — Action

Instead of escalating, I suggested we each write up our approach with expected throughput numbers and review together. I showed that at 400K events/day, single partitions would bottleneck our consumers. He showed valid concerns about consumer group complexity. We combined both views — multiple partitions but with a managed consumer group strategy.

R — Result

The hybrid approach worked. We hit sub-200ms latency targets and the system scaled cleanly. The conflict made the design stronger than either of our original proposals.

Story 2: Code Review Disagreement at Noon

S — Situation

At Noon, a senior engineer on my team left a blunt code review on my IoT pipeline implementation, saying my approach to handling MQTT reconnections was 'naive' and would fail under load.

T — Task

I felt the feedback was delivered harshly but I needed to figure out if they were technically right, and respond professionally either way.

A — Action

I took a day to research MQTT reconnection patterns, ran load tests on my implementation, and found they were partially correct — my exponential backoff logic had a bug under high-frequency disconnect events. I fixed the bug, added a test case for it, and replied to the review with both the fix and a note thanking them for catching it.

R — Result

The engineer respected that I engaged with the feedback technically rather than defensively. We ended up pairing on two more features together and the pipeline reduced downtime detection by 40% as planned.

Story 3: Open Source PR Conflict

S — Situation

While contributing to pgjdbc, another contributor submitted a PR that solved the same issue I was working on, but using a different approach. There was a brief back-and-forth in the comments about whose approach should be merged.

T — Task

I needed to either defend my approach or find a collaborative path forward — without making it personal in a public open source thread.

A — Action

I read their implementation carefully, acknowledged what was better about it, and pointed out one specific case my approach handled that theirs didn't. I proposed merging the best parts of both. The maintainer appreciated that I engaged constructively rather than protectively.

R — Result

My PR was merged with an element of their approach incorporated. Both contributors were credited. It reinforced for me that in collaborative environments, the best idea wins — not the loudest voice.

Behavioral — Conflict & Communication

Q19. Describe a time when there was a conflict within your team. How did you help resolve it? Did you do anything to prevent it in future?

Story 1: Architecture Conflict on MedStroke

S — Situation

On MedStroke, two teammates disagreed sharply on the ML model selection — one wanted a neural network for higher potential accuracy, the other wanted Random Forest for interpretability in a clinical context.

T — Task

The disagreement was stalling the project and both were right in different ways.

A — Action

I facilitated a structured comparison rather than letting it stay abstract. I asked each person to document their approach with concrete tradeoffs relevant to our specific constraints: dataset size, interpretability requirement for doctors, and our timeline. Once it was written down, the clinical interpretability requirement made the decision clear.

R — Result

We chose Random Forest. The neural network advocate accepted the decision because they saw the clinical reasoning applied to our specific context. To prevent future conflicts, I proposed we document architectural decisions with explicit context and constraints so debates are grounded in facts, not preferences.

Story 2: Kafka vs REST at Ericsson

S — Situation

At Ericsson, two engineers on the wider team had an ongoing low-level conflict about whether to use Kafka or REST for a new internal service — it was spilling into planning meetings and creating friction.

T — Task

As someone who'd just built both types of systems, I was asked to help resolve it.

A — Action

I suggested a time-boxed spike: each approach gets two days to prototype the same feature with the same performance constraints. Both engineers built their version. The data from the spike made the decision obvious without anyone needing to 'win' the argument.

R — Result

The conflict dissolved because the answer came from evidence, not opinion. I proposed adopting the spike approach as a default for future architecture disagreements — make the decision empirical, not political.

Story 3: Test Priority Conflict at Noon

S — Situation

At Noon, two engineers disagreed on whether to prioritise writing tests or pushing new features before the pipeline launch. The disagreement was creating tension in standups.

T — Task

I saw the conflict was really about risk tolerance, not tests vs features — both were legitimate concerns.

A — Action

I brought both engineers together and reframed the question: 'What's the minimum test coverage that makes us comfortable shipping?' That shifted the conversation from opposition to collaboration. We defined a coverage threshold together and agreed to gates — certain flows needed tests before the feature could merge.

R — Result

The conflict ended because we turned it into a shared policy both could live with. In the future, having a pre-agreed merge policy meant similar conflicts didn't recur.

Behavioral — Conflict & Communication

Q25. Tell me about a time you disagreed with a colleague. How did you handle the situation?

Story 1: Kafka Partitioning Strategy at Ericsson

S — Situation

At Ericsson, a colleague wanted single partitions per Kafka topic for simplicity. I believed we needed multiple partitions to support parallel consumption at 400K daily events.

T — Task

We disagreed technically and needed to resolve it before the architecture was locked.

A — Action

I wrote up a throughput analysis showing consumer lag projections at our event volume with single vs multiple partitions. I presented it as information, not a verdict — and acknowledged their concern about consumer group complexity. We discussed the tradeoffs in a one-on-one before the team meeting.

R — Result

We agreed on multiple partitions with managed consumer group assignment. The system hit sub-200ms latency. The disagreement improved the design. Bringing data to a disagreement moves it from opinion to engineering.

Story 2: Integration vs Mock Testing at Noon

S — Situation

At Noon, a colleague thought integration tests were overkill for the IoT pipeline — unit tests with mocked MQTT brokers were sufficient in their view.

T — Task

I disagreed because MQTT reconnection behaviour under real broker failures was impossible to test with mocks, and that was our highest-risk failure mode.

A — Action

I didn't argue in the abstract. I wrote one integration test that used a real test broker, simulated a disconnect, and demonstrated a failure mode that the mocked tests missed. I showed them the test result and let it speak.

R — Result

They agreed integration tests were worth it for the reconnection layer. We split the test suite — unit tests for transformation logic, integration tests for broker interaction. The empirical approach resolved the disagreement faster than debating principles.

Story 3: PostgreSQL vs MongoDB for MedStroke

S — Situation

On MedStroke, a teammate wanted to use MongoDB for patient data storage. I believed PostgreSQL was the right choice given the relational nature of NIHSS scoring, tPA eligibility, and patient history.

T — Task

We had a genuine technical disagreement and needed to pick one before building the data layer.

A — Action

I drew out the data model on paper — patient records, scoring events, eligibility flags — and showed how every query pattern we needed was a join. Then I asked them to describe a query that MongoDB would handle better. They couldn't find one that applied to our specific use case.

R — Result

We used PostgreSQL. The data model was clean and all queries were straightforward. I've since used this approach regularly: when disagreeing on a technology choice, draw the data model and query patterns first — the right answer often becomes obvious.

Behavioral — Conflict & Communication

Q45. Tell me about a time when you had a hard time working with someone in your team. How did you handle it?

Story 1: Async-Only Teammate on MedStroke

S — Situation

On MedStroke, one teammate did excellent work but communicated only asynchronously — no standups, no check-ins — and their component was on the critical path to the hospital pilot.

T — Task

I needed to understand their progress without micromanaging or damaging a working relationship with someone who was technically strong.

A — Action

I had a direct one-on-one — not a group conversation — and said clearly: 'I'm not getting enough signal on where the backend integration stands, and I'm worried about the pilot timeline. Can we find a lightweight way to sync daily that works for how you work?' We agreed on a one-sentence async update each morning. I adapted to their style rather than demanding they adapt to mine.

R — Result

The updates started immediately and we caught a timing issue in the authentication flow two weeks before the pilot. The hospital pilot launched successfully. Working with someone whose style differs from yours requires finding the minimum viable sync that gives you what you need without imposing a workflow on them.

Story 2: Very Critical Senior Engineer at Noon

S — Situation

At Noon, one senior engineer on the team was deeply critical of junior work in code reviews — technically precise but often blunt to the point of being discouraging.

T — Task

I needed to continue getting useful technical feedback from them without the reviews becoming demoralising.

A — Action

I changed my approach to code reviews with them specifically: I wrote more defensive comments in my PRs pre-empting obvious critiques, and started asking for informal feedback before submitting — 'before I open this PR, does this approach look reasonable to you?' By the time the formal review happened, the big issues were already resolved.

R — Result

The reviews became more collaborative and less adversarial. Their technical depth was genuinely valuable — the friction was about process, not competence. Adapting how you engage with a difficult reviewer can recover most of the value while reducing the friction.

Story 3: Misaligned Priorities in NSBE Officer Team

S — Situation

As NSBE Assistant VP, one officer consistently prioritised events that were interesting to them personally over events that served the most members — creating internal tension about what we should be doing.

T — Task

I needed to redirect their energy without losing their commitment to the chapter.

A — Action

I had a private conversation focused on outcomes rather than preferences: 'Our highest-attendance events are the ones that directly address interview prep and networking. I'd love to find a way to include what you care about — can we design an event that does both?' We co-created an event that incorporated both their interest and a practical skill component.

R — Result

Their engagement increased because they had real ownership. Attendance at their event was strong because it had practical value. Finding the overlap between what someone wants to do and what the organisation needs is more effective than directing them.

Behavioral — Decision Making

Decision Making

Behavioral · 4 questions

Behavioral — Conflict & Communication

p.324

Behavioral — Decision Making

Q7. Provide an example of a time when you had to make a difficult decision.

Story 1: Event-Driven vs REST at Ericsson

S — Situation

At Ericsson, I had to decide between two architectures for the pricing data system: a simpler REST-based approach my manager was comfortable with, or an event-driven Apache Kafka system I believed was the right choice at our scale.

T — Task

The Kafka approach was more complex to build and operate, but REST polling at 400K daily events would have introduced significant latency and load on upstream services.

A — Action

I built a small benchmark, documented the tradeoffs honestly — including the operational overhead of Kafka — and presented both options with data. I recommended Kafka but made clear it was a recommendation, not a demand. I also offered to own the extra complexity.

R — Result

We went with Kafka. The system processed 400K+ daily events across 6 topics at sub-200ms latency and replaced manual SQL workflows entirely. The harder decision produced the better outcome.

Story 2: Hospital Pilot Decision for MedStroke

Behavioral — Decision Makingp.325

S — Situation

With MedStroke AI, I had to decide whether to proceed with the Louisiana hospital pilot when our ML model was only at 45% accuracy — below my internal 80% target.

T — Task

Cancelling would have disappointed the hospital contact and delayed the project significantly. Proceeding meant piloting with a model that wasn't where I wanted it.

A — Action

I reframed the tool's clinical role from 'diagnostic replacement' to 'decision support aid,' consulted with the hospital contact on acceptable accuracy ranges for a first pilot, and was transparent about the model's limitations in all documentation.

R — Result

The pilot went ahead and reduced time-to-treatment by 35%. The hospital's feedback was positive specifically because we were honest about limitations. The difficult decision to go ahead — with transparency — was the right one.

Story 3: Dropping a Feature to Meet Deadline at Noon

S — Situation

At Noon, two weeks before the IoT pipeline go-live, I realised we couldn't finish both the real-time alert feature and the historical replay feature with the quality bar I'd set.

T — Task

I had to decide which to cut — both were on the roadmap and stakeholders expected both.

Behavioral — Decision Makingp.326

A — Action

I mapped each feature to business impact, found that real-time alerts directly supported the 40% downtime-reduction goal while replay was a nice-to-have, and presented this analysis to my manager with a recommended phasing plan. I was clear about why — not to manage my workload, but to protect launch quality.

R — Result

We shipped real-time alerts on time. Downtime detection improved by 40%. The replay feature was scoped into the next sprint. Making the call early gave us time to do the remaining work well.

Behavioral — Decision Makingp.327

Behavioral — Decision Making

Q35. Tell me a time when you had a different opinion than the rest of the team. How did you handle it?

Story 1: Event-Driven vs Traditional at Ericsson

S — Situation

At Ericsson, when I proposed using Apache Kafka for the pricing system, the initial team preference was a simpler REST polling approach — Kafka felt unnecessarily complex for the team at the time.

T — Task

I believed strongly that at 400K events/day with sub-200ms latency requirements, REST polling would not meet our SLA and would create load on upstream services. I needed to make that case without bulldozing team consensus.

A — Action

I built a small benchmark rather than arguing in the abstract. I simulated both approaches under load and showed the latency numbers. I acknowledged the operational complexity Kafka would add and offered to own that complexity personally.

R — Result

The team agreed to Kafka after seeing the data. The system hit sub-200ms latency from launch. Having a different opinion from the team is fine — what matters is bringing evidence, not just conviction.

Story 2: Random Forest vs Neural Network for MedStroke

Behavioral — Decision Makingp.328

S — Situation

On MedStroke, my teammate preferred a neural network for the stroke risk model. I advocated for Random Forest, believing interpretability was essential in a clinical context.

T — Task

This was a genuine disagreement and both approaches had real technical merit — it wasn't obvious who was right.

A — Action

I framed the decision around the clinical user. 'A neurologist needs to be able to challenge the model's reasoning. How do we explain a neural network's prediction to a doctor in 30 seconds?' Random Forest's feature importance scores gave us that. I put the use case first, not my preference.

R — Result

We chose Random Forest. The doctors engaged with the confidence scores and feature outputs constructively. My different opinion was accepted because it was grounded in user need, not technical preference.

Story 3: Multiple Partitions vs Single Partition at Ericsson

S — Situation

When designing the Kafka consumer topology at Ericsson, the team's initial instinct was single partitions for simplicity — easier to reason about, simpler to operate.

T — Task

I was the only one arguing for multiple partitions to support parallelism, and I was the most junior person in the discussion.

Behavioral — Decision Makingp.329

A — Action

I ran the throughput math and showed it to the team without editorialising. 'At 400K events/day, single partition consumers will have X minutes of lag at peak. Here's the calculation.' The data spoke rather than me asserting I was right.

R — Result

We moved to multiple partitions. The system scaled cleanly. Being junior doesn't mean your opinion doesn't count — but it means you need to back it with data twice as thoroughly.

Behavioral — Decision Makingp.330

Behavioral — Decision Making

Q48. Tell me about a time when you had to make a decision without all the information you needed.

Story 1: Ericsson Kafka Design Before Specs Finalised

S — Situation

At Ericsson, I had to commit to the Kafka consumer group architecture before the upstream team had finalised the event schema — I didn't know the exact message format or volume distribution across topics.

T — Task

Waiting for the final spec would have cost two weeks. I needed to make a defensible decision with partial information.

A — Action

I identified which decisions were reversible and which weren't. The consumer group topology was reversible — I could reconfigure partition assignment after the fact. The choice of Kafka over REST was not — that required infrastructure commitment. I made the irreversible decisions based on the information I had (volume range estimates), and explicitly flagged the reversible decisions as provisional. I designed the consumer configuration to be tunable via environment variables so I could adjust it without a code deploy.

R — Result

When the final schema arrived, one topic had 3x the volume I'd estimated — I tuned the consumer group via config without a code change. Distinguishing reversible from irreversible decisions is the most important framework for making decisions under uncertainty.

Story 2: MedStroke Hospital Pilot Decision

Behavioral — Decision Makingp.331

S — Situation

I had to decide whether to proceed with the Louisiana hospital pilot when the ML model accuracy was at 45% — I didn't have clinical validation data showing whether 45% was medically useful or not.

T — Task

The information I needed — clinical impact data — was only available if I ran the pilot. But running the pilot with a poor tool could damage trust in AI tools with the hospital.

A — Action

I consulted the hospital contact directly rather than making the call alone. I was transparent about the accuracy level, explained what it meant clinically, and asked them to assess acceptability given their workflow. They confirmed 45% was useful as a screening aid when paired with clinical judgment. That was the information I needed to proceed.

R — Result

The pilot ran and reduced time-to-treatment by 35%. Going to the stakeholder directly to get the information I needed — rather than guessing or delaying — was the right move. Sometimes the missing information is a conversation away.

Story 3: Noon Pipeline Architecture with Incomplete Client Specs

S — Situation

At Noon, I had to design the IoT pipeline architecture before the client had finalised their device count and data frequency specifications — two numbers that significantly affected the right technology choice.

Behavioral — Decision Makingp.332

T — Task

I couldn't design a scalable system without knowing the scale. But I couldn't wait for specs that might take weeks.

A — Action

I asked for ranges rather than exact numbers. The client gave me 'between 500 and 5000 devices, reporting every 1-5 minutes.' I designed for the high end — 5000 devices, 1-minute frequency — because overprovisioning in an MQTT/Kafka architecture costs almost nothing, while underprovisioning requires a rewrite. I documented the assumption explicitly.

R — Result

The pipeline handled the actual device count comfortably. The documented assumption gave me a clear record of why I made the choice I did — useful when questions came up later. When you're missing information, define the range and design for the hard end.

Behavioral — Decision Makingp.333

Behavioral — Decision Making

Q50. Tell me about a time you made a decision to sacrifice short-term gain for a longer-term goal.

Story 1: Ericsson — Robust Architecture Over Quick Win

S — Situation

At Ericsson, midway through the Kafka microservice build, I identified a shortcut: I could achieve acceptable latency numbers by reducing the number of topics from 6 to 2 and simplifying the consumer group topology. It would have shipped faster.

T — Task

The simplified design would meet the immediate SLA but would make it very hard to add new event types later without a full redesign.

A — Action

I chose the more complex architecture — 6 topics, properly partitioned consumer groups, designed for extensibility. I spent three extra days on it and was transparent with my manager about why. I documented the extensibility rationale so future engineers would understand the decision.

R — Result

The system launched on time (I absorbed the extra days through better prioritisation elsewhere). More importantly, when the team added two new event types after I left, the architecture accommodated them without any structural changes. Short-term gain from the shortcut would have created technical debt that cost more to pay back.

Story 2: Noon — Comprehensive Tests Over Speed

Behavioral — Decision Makingp.334

S — Situation

At Noon, I could have shipped the IoT pipeline two days faster by writing only the happy-path unit tests and skipping the comprehensive integration test suite I was planning.

T — Task

Two days mattered — the client demo was close. But the integration tests would catch failure modes that the unit tests couldn't.

A — Action

I chose to write the full 50+ test suite even though it pushed the demo closer. I worked late two nights to absorb the time without slipping the timeline. I was clear with my manager about the tradeoff I was making and why.

R — Result

The tests caught two real bugs — an MQTT reconnection issue and a PostgreSQL write race condition — that would have appeared in production at the worst possible time. The short-term cost was two late nights. The long-term gain was zero production incidents. Comprehensive tests are one of the clearest cases where short-term investment pays back immediately.

Story 3: MedStroke — Hospital Pilot Delay for Accuracy

S — Situation

We could have run the hospital pilot one month earlier with the model at 32% accuracy. Delaying gave us time to improve it to 45% through better feature engineering and class balancing.

Behavioral — Decision Makingp.335

T — Task

The earlier pilot would have given us feedback sooner. But I was concerned that a 32% accuracy model in a clinical context would undermine the hospital's trust in AI tools in ways that were hard to recover from.

A — Action

I recommended delaying the pilot by four weeks to improve accuracy and refine the clinical framing. I was explicit that this was a short-term sacrifice — we'd be four weeks behind our original plan — for a longer-term goal of a successful adoption.

R — Result

The pilot ran at 45% accuracy with clear confidence scores and clinical framing. The hospital adopted the tool and reduced time-to-treatment by 35%. A 32% model might have been rejected and closed the door to future AI tools in that hospital. The four-week delay was worth it.

Behavioral — Decision Makingp.336

Design & Product

Behavioral · 2 questions

Q23. How would you design or test a product to make sure it's diverse and inclusive to all users?

Story 1: MedStroke Designed for Non-Technical Doctors

S — Situation

MedStroke AI was going to be used by doctors and nurses with varying levels of technology comfort — from residents who used tablets daily to senior neurologists who preferred paper notes.

T — Task

I needed to design an interface that didn't assume technical fluency but also didn't condescend to clinically sophisticated users.

A — Action

I interviewed two nurses and one doctor before designing the UI. I kept the primary workflow to three steps, added clear confidence scores and uncertainty indicators, and made the AI output secondary to the NIHSS scoring tool doctors already trusted. I tested with a non-technical staff member and iterated until they could use it without instructions.

R — Result

The hospital staff adopted the tool during the pilot without a formal training session. Inclusive design isn't about adding features — it's about removing barriers through early user research.

Story 2: Ericsson Dashboard for a 30-Engineer Team

S — Situation

The React dashboard I built at Ericsson needed to serve engineers at every seniority level — from junior devs unfamiliar with Kafka internals to architects who wanted raw partition metrics.

T — Task

A single view couldn't serve both audiences. I needed to design progressive disclosure.

A — Action

I built a default view showing high-level pipeline health — consumer lag, throughput, error rates — with drill-down panels for partition-level metrics. New engineers could read the top-level view immediately; senior engineers could drill down. I tested the default view with two junior engineers before shipping.

R — Result

The dashboard was adopted across the full team. Both senior and junior engineers used it in incident resolution. Progressive disclosure is the core pattern for building tools that work for diverse users.

Story 3: NSBE Events for 300+ Diverse Members

S — Situation

Our NSBE chapter had members ranging from first-year students who'd never seen a technical interview to seniors with multiple internships — plus students from non-CS majors interested in technology careers.

T — Task

Events had to be valuable across that entire range without talking down to advanced members or overwhelming beginners.

A — Action

I structured events in tiers — 'intro' sessions for fundamentals and 'advanced' workshops for students who already had internship experience. I surveyed members after each event and used the feedback to adjust. For networking events I briefed corporate sponsors to bring engineers at different levels.

R — Result

Attendance grew to 300+ active members with strong retention across all cohorts. Diversity-aware programming isn't a nice-to-have — it's the only way to build a community that actually sticks.

Q31. Assume you are given a task to design a system. How would you do it? How would you resolve ambiguity?

Story 1: Ericsson Event-Driven System Design

S — Situation

At Ericsson I was asked to design a system to aggregate 10K+ pricing records from multiple upstream sources into RESTful endpoints — with minimal context on volume, latency requirements, or upstream formats.

T — Task

Before writing a line of code I needed to resolve the ambiguity in requirements and produce a design I could defend.

A — Action

I started with clarifying questions: What's the acceptable latency for a price update to appear downstream? What happens if an upstream source goes down? How stale can data be? From those answers I understood we needed near-real-time (not eventual consistency) and fault tolerance. I then chose event streaming over REST polling because of the latency and decoupling requirements, drew a data flow diagram, identified the failure modes, and presented it for review before building.

R — Result

The design was approved and the system shipped — aggregating 10K+ records into RESTful endpoints with real-time access to previously disconnected historical data. Resolving ambiguity means asking the right questions, not making assumptions.

Story 2: MedStroke System Architecture

S — Situation

When I started MedStroke, the only clear requirement was 'a stroke diagnosis tool for hospitals.' Everything else — data model, user roles, real-time vs batch, ML integration — was undefined.

T — Task

I needed to turn a vague goal into a concrete system design.

A — Action

I did clinical context research first — how do stroke diagnoses actually happen? That told me real-time monitoring was essential, roles (neurologist vs nurse) determined data access, and the ML output needed to be explainable, not just accurate. I drew the component diagram, identified the highest-risk integration points (ML model → clinical display), and validated the design with the hospital contact before building.

R — Result

The design held through the full development cycle with only minor scope adjustments. Clinical context research was the most important step — the system needed to fit a workflow that already existed, not invent one.

Story 3: NanoSQL Database Design

S — Situation

Building NanoSQL — a SQLite clone in C — required designing a full database engine from a blank page: storage format, indexing strategy, query execution, and memory management.

T — Task

I had to make foundational design decisions that would determine everything downstream.

A — Action

I read the SQLite design document and the original B-tree paper before writing any code. I started with the storage layer — page format and persistence — because every other component depended on it. I designed each layer's interface before implementing it, so I could test components independently. I wrote the B-tree specification in pseudocode first, which revealed three edge cases I hadn't thought through.

R — Result

The system delivered B-tree indexing, persistent storage, and SQL operations. Writing the specification before the implementation found more bugs than testing did. For any complex system: design the interfaces first, then implement, then test.

Behavioral — Failure & Growth

Failure & Growth

Behavioral · 5 questions

Behavioral — Failure & Growth

Q4. Tell me about a time you failed. How did you deal with the situation?

Story 1: MedStroke ML Accuracy

S — Situation

On MedStroke AI, I was responsible for the Random Forest model that analysed blood sample data to predict stroke risk. I set an internal target of 80% accuracy before our hospital pilot.

T — Task

When we reached the pilot date, the model was only at 45% accuracy — well below what I'd promised the team and the hospital contact.

A — Action

I was transparent with the team immediately rather than hiding the number. I did a root cause analysis — the training data was too small and imbalanced. I expanded the feature set, applied SMOTE for class balancing, and ran additional cross-validation. I also reframed the tool with the hospital as a 'decision support aid' rather than a diagnostic replacement, which was actually more appropriate clinically.

R — Result

The pilot went ahead and reduced time-to-treatment by 35%. The lower accuracy turned into a lesson about setting honest expectations and designing AI tools for clinical contexts — the doctors trusted a transparent tool more than a confident one.

Story 2: NanoSQL B-Tree Bug

S — Situation

Building NanoSQL, my SQLite clone in C, I implemented the B-tree and shipped it to a self-imposed deadline without fully testing node splitting under large datasets.

T — Task

Days later I discovered a corruption bug — pages were being overwritten during splits, destroying stored data silently.

A — Action

I rolled back the B-tree implementation, wrote a comprehensive test suite for split and rebalance operations first, then rebuilt the implementation test-first. It took an extra week but I caught three more edge cases I hadn't thought of initially.

R — Result

The final B-tree handled dynamic node splitting and rebalancing correctly under all test conditions. I learned to never skip tests on data structures that touch persistent storage — the cost of a bug there is data loss, not just wrong output.

Story 3: Missing Edge Cases at Noon

S — Situation

At Noon, I submitted my first batch of integration tests for the IoT pipeline with confidence. A senior engineer's review found that I'd missed concurrent write scenarios — two sensors reporting simultaneously to the same topic.

T — Task

My tests had passed, but they weren't testing the right things. I had to acknowledge that and fix both the tests and potentially the implementation.

A — Action

I rewrote the test suite to include concurrent scenarios using JUnit's parallel execution support, found two real race conditions in the pipeline code, and patched them. I also adopted a checklist for concurrent systems going forward.

R — Result

The updated suite caught both race conditions before production. The pipeline shipped without any concurrency incidents. My manager noted the test quality improvement in my end-of-internship feedback.

Behavioral — Failure & Growth

Q15. Tell me about a time when you missed a deadline. What happened, and how did you handle it?

Story 1: NanoSQL B-Tree Overrun

S — Situation

I set a personal deadline to complete NanoSQL — my SQLite clone in C — within six weeks. The B-tree implementation took three additional weeks beyond my estimate.

T — Task

The deadline was self-imposed, but I treated it seriously as a commitment to myself and used it as a learning exercise in estimation.

A — Action

When I realised I was going to miss it, I did a root cause analysis rather than just pushing harder. I'd underestimated the complexity of page allocation and node rebalancing under concurrent access. I revised my estimate based on what was actually left, communicated it to myself in writing, and broke the remaining work into daily checkpoints.

R — Result

The final B-tree was correct, well-tested, and documented. The miss taught me that estimation requires decomposing work to the level of individual operations, not just features. I now break tasks into smaller units before estimating anything.

Story 2: MedStroke ML Iteration Delay

S — Situation
The original MedStroke timeline had the ML model ready for integration two weeks before the hospital pilot. I missed that internal deadline by five days due to underestimating data preprocessing complexity.
T — Task
The team needed the model to integrate with the FastAPI backend and I was the blocker.
A — Action
The moment I knew I'd miss it, I told the team immediately — not the day it was due. I gave them an honest revised estimate, offered to work in parallel with their integration once I had a working but imperfect model, and delivered a checkpoint version for them to build against while I continued improving it.
R — Result
Integration started on schedule using the checkpoint model. The final model was delivered two days after the original deadline — but because I communicated early, it cost the project zero days. Early transparency on a slip is almost as good as not slipping.

Story 3: Noon Pipeline Integration Delay

S — Situation
At Noon, the IoT pipeline integration with PostgreSQL hit unexpected schema compatibility issues that delayed delivery by three days relative to the sprint commitment.

T — Task
The team was counting on the pipeline being ready to feed the notification system I was handing off to.
A — Action
I flagged the issue two days before the deadline — not the day of. I explained exactly what the blocker was, proposed a workaround that would let the downstream team begin their work with stubbed data, and took full ownership of the delay rather than attributing it to the schema issue.
R — Result
The downstream team used the stub data and stayed unblocked. The real integration landed three days late but caused zero cascading delays. Missing a deadline gracefully means communicating early and protecting the people depending on you.

Behavioral — Failure & Growth
Q22. Describe a time when you took a big risk and it failed.
Story 1: MedStroke ML Accuracy Miss
S — Situation
For MedStroke AI, I committed to an 80% accuracy target for the Random Forest stroke risk model before I had fully assessed the dataset quality and size.
T — Task
I was confident in the architecture but hadn't pressure-tested the assumption that our dataset was sufficient for that accuracy level.
A — Action
When the model plateaued at 45%, I ran a root cause: the dataset was too small and class-imbalanced. I tried SMOTE balancing, feature engineering, and hyperparameter tuning. I got to 45% — meaningfully better than a random classifier, but not 80%.
R — Result
I missed the accuracy target I'd committed to. I was transparent with the team, reframed the tool's clinical role as decision support rather than diagnosis, and the hospital pilot still ran successfully. The lesson: never commit to an accuracy target before validating your dataset — that's where confidence should come from.

Story 2: NanoSQL Custom Memory Allocator

S — Situation
In NanoSQL, I decided to implement a custom memory allocator for page management rather than using standard malloc — believing it would give me tighter control over fragmentation.
T — Task
This was ambitious and unnecessary for the project's stated goals, but I wanted to try it.
A — Action
I spent a week building the allocator. It worked in simple cases but introduced subtle bugs when pages were freed and reallocated under mixed read/write workloads — corruption that was nearly impossible to reproduce reliably.
R — Result
I scrapped the custom allocator and used standard C memory management. The B-tree became stable and the project shipped. The risk failed because I took on unnecessary complexity for a learning goal — which is fine in a personal project, but taught me to separate learning experiments from production code paths.

Story 3: Over-Scoping the Ericsson Dashboard

S — Situation
When I built the React dashboard at Ericsson, I initially tried to include predictive analytics — forecasting consumer lag before it became critical.
T — Task
I allocated time to the prediction feature and it pulled me away from the core visualisation work.

A — Action
The prediction model required historical baseline data we hadn't collected yet. I spent three days on it before accepting it wasn't going to be ready. I cut the feature entirely and refocused on the core timeline visualisation.
R — Result
The core dashboard shipped and cut debugging time from 60 to 10 minutes. The prediction feature would have added marginal value but nearly sank the timeline. The risk of over-scoping taught me: ship the thing that solves the core problem first.

Behavioral — Failure & Growth
Q43. Give an example of when you took a huge risk and failed.
Story 1: Over-Ambitious ML Target for MedStroke
S — Situation
Before properly assessing our dataset quality and size, I publicly committed to 80% accuracy for MedStroke's stroke risk model — to the team and to the hospital contact.
T — Task
The commitment was made with confidence but without sufficient validation of whether the dataset could support that target.
A — Action
I worked intensively on the model — SMOTE balancing, feature engineering, hyperparameter tuning, cross-validation. Every technique I tried improved the model, but the ceiling was 45–48% given our dataset size. At that point I was out of time and options within the project constraints.
R — Result
I missed the target I'd committed to — publicly and by a significant margin. I was transparent immediately, reframed the tool appropriately, and the hospital pilot still ran successfully. The lesson I carry from this: never commit to an accuracy target before validating your dataset size and quality. Confidence should follow evidence, not precede it.

Story 2: NanoSQL Custom Page Allocator

S — Situation

In NanoSQL I decided to build a custom page allocator instead of using standard malloc — I was convinced I could reduce fragmentation and improve B-tree performance.

T — Task

This was ambitious for a solo project with a self-imposed deadline, and the allocator was not on the critical path for the project's core goals.

A — Action

I invested a full week building the allocator. It worked for simple sequential operations but introduced non-deterministic corruption bugs under mixed read/write workloads — the kind that manifested hours into a test run, not immediately.

R — Result

I scrapped the allocator entirely and used standard C memory management. The B-tree became stable and shipped. The risk failed because I added complexity that wasn't needed for the goal. The lesson: in systems programming, custom memory management is almost never worth the risk unless you have a profiler showing it's the bottleneck.

Story 3: React Dashboard Prediction Feature

S — Situation

When building the Ericsson React dashboard, I decided to add a predictive analytics component that would forecast consumer lag before it became critical.

T — Task

I allocated three days to this feature, believing I could implement a simple time-series model against the pipeline metrics.

A — Action

The prediction required historical baseline data we hadn't been collecting — without it, any prediction was noise. I spent three days before accepting the feature wasn't going to be credible without the historical data foundation.

R — Result

I cut the feature entirely. The core dashboard shipped on time and cut debugging time from 60 to 10 minutes without it. The risk failed because I didn't validate the data dependency before starting the build. Now I check data availability before scoping any analytics feature.

Behavioral — Failure & Growth

Q46. Tell me about a project that didn't go according to plan.

Story 1: MedStroke ML Accuracy Plateau

S — Situation

MedStroke AI was planned around an 80% ML accuracy target for stroke risk prediction. The project did not go to plan — we plateaued at 45% and couldn't get higher within our timeline and dataset constraints.

T — Task

We had a fixed hospital pilot date, a model that wasn't hitting its target, and a team watching the accuracy number not move.

A — Action

I pivoted the plan explicitly. I reframed the tool's clinical role from 'diagnostic replacement' to 'decision support aid,' adjusted the hospital pilot objectives accordingly, and was transparent with both the team and the hospital contact about where we were and why. I documented the limitation clearly in every piece of clinical-facing material.

R — Result

The pilot ran and reduced time-to-treatment by 35%. The hospital staff adopted the tool because we were honest about its capabilities. The project didn't go to plan technically — but it succeeded clinically because we adapted rather than either overpromised or cancelled.

Story 2: NanoSQL B-Tree Full Rewrite

S — Situation

My NanoSQL project plan included a B-tree implementation in week 3. What actually happened: I finished the B-tree in week 3, shipped it, discovered a data corruption bug under mixed read/write loads in week 4, and did a full rewrite in week 5.

T — Task

The rewrite wasn't in the plan — it cost me the entire buffer I'd built into the timeline.

A — Action

I scrapped the original implementation entirely rather than patching it — the corruption was deep enough that patches would have been unreliable. I rewrote the B-tree test-first: wrote the test suite before the implementation, so I was building toward verified correctness from the start. The rewrite took one week.

R — Result

The final B-tree handled all operations correctly with zero corruption. The project shipped two weeks later than planned. The lesson: test-first development isn't optional for data structures that touch persistent storage — correctness can't be verified by inspection alone.

Story 3: Noon IoT Pipeline Scope Expansion

S — Situation

The Noon IoT pipeline project was scoped for real-time ingestion only. Midway through, the requirements expanded to include historical data replay — a feature that required significant schema changes.

T — Task

The expansion was unplanned and would break our timeline if absorbed without adjustment.

A — Action

I did not absorb it silently. I immediately estimated the new work, presented the timeline impact to my manager, and proposed two options: deliver both features with a revised timeline, or deliver real-time first with replay in the next sprint. I recommended the phased approach with clear reasoning.

R — Result

We went phased. Real-time shipped on schedule with 40% downtime improvement. Replay was delivered in the following sprint. Projects rarely go exactly to plan — the skill is detecting the deviation early and reconfiguring rather than silently absorbing scope.

Behavioral — Feedback

Feedback

Behavioral · 2 questions

Behavioral — Failure & Growth

p.360

Behavioral — Feedback

Q10. Describe a time you received tough or critical feedback.

Story 1: Kafka Consumer Design Review at Ericsson

S — Situation

At Ericsson, after I submitted my Kafka consumer implementation for review, a senior engineer left detailed comments saying my consumer group configuration would cause excessive rebalancing under load — and that the design showed I didn't fully understand Kafka's consumer group protocol.

T — Task

The feedback was direct and a bit stinging, but I needed to figure out whether it was right and fix it if so.

A — Action

I took a full day to study Kafka consumer group internals and session timeout tuning before responding. The engineer was correct. I redesigned the consumer configuration using sticky partition assignment, updated the implementation, and wrote back with a clear explanation of what I'd changed and why — not just 'fixed it.'

R — Result

The updated implementation hit sub-200ms latency consistently under production load. The senior engineer later became one of my biggest advocates on the team. Tough feedback delivered well was a gift.

Story 2: pgjdbc PR Review

Behavioral — Feedbackp.361

S — Situation

My first PR to pgjdbc came back with 14 review comments — some technical, some about code style, and one saying my approach to connection handling was 'reinventing what already exists in the library.'

T — Task

I had to absorb a large amount of critical feedback on a public platform, in a codebase maintained by engineers far more experienced than me.

A — Action

I went through every comment carefully, responded to each one individually, and spent two days studying the existing connection handling code the reviewer referenced. Three of the comments I agreed with and fixed. Two I pushed back on with data. I never responded defensively.

R — Result

The PR was merged after two revision rounds. The reviewer commented that my engagement with feedback was 'thorough and professional.' It's now a merged contribution to a library used by Java developers at JPMorgan and other financial institutions.

Story 3: MedStroke Team Code Review

S — Situation

During MedStroke development, a teammate reviewed my FastAPI endpoints and pointed out that my error handling was inconsistent — some endpoints returned 500 errors where I should have returned 422s with validation details.

Behavioral — Feedbackp.362

T — Task

It was embarrassing because I'd considered myself the backend lead and this was a basic REST API design issue.

A — Action

I thanked them genuinely, audited every endpoint immediately, standardised the error response schema, and added a shared error handling middleware so the inconsistency couldn't recur. I also updated the API docs.

R — Result

The hospital pilot ran with zero API-related errors reported. More importantly, I stopped thinking of feedback as a judgment on my ability — it's just information that makes the product better.

Behavioral — Feedbackp.363

Behavioral — Feedback

Q11. Describe a time when you had to give someone difficult feedback. How did you handle it?

Story 1: NSBE Officer Feedback

S — Situation

As NSBE Assistant VP, one of our officers was consistently missing event prep deadlines, and other officers were quietly picking up the slack without saying anything.

T — Task

I needed to address it directly without damaging the relationship or demoralising someone who clearly cared about the chapter.

A — Action

I had a private one-on-one, not a group conversation. I started by acknowledging what they were doing well, then was specific and factual: 'The last three event prep deadlines were missed, and it created last-minute pressure for the team.' I asked them what was going on — and found out they were overwhelmed with coursework. We restructured their responsibilities together.

R — Result

Their reliability improved significantly over the next month. They thanked me later for addressing it privately and directly rather than letting it fester. The chapter ran more smoothly and their confidence actually grew from having clearer ownership.

Story 2: MedStroke ML Implementation

Behavioral — Feedbackp.364

S — Situation

A teammate on MedStroke was responsible for data preprocessing and had been passing raw, unscaled features into the Random Forest model — which was hurting accuracy significantly.

T — Task

I needed to tell them their implementation was wrong, but they'd spent significant time on it and were proud of it.

A — Action

I framed it around the model performance data, not their effort: 'The accuracy is at 45% and I think I found why — let me show you what I'm seeing in the feature distributions.' I showed the unscaled values, explained why Random Forest is sensitive to it in this context, and asked if we could fix it together rather than handing it back to them as a failure.

R — Result

We fixed the preprocessing together in a pair session. The model improved. They felt involved in the solution rather than criticised. That approach — lead with data, collaborate on the fix — became my default for peer feedback.

Story 3: Code Review Feedback to a Peer at Ericsson

S — Situation

During code review at Ericsson, I noticed a fellow intern's Kafka producer implementation had a configuration issue — they were not setting proper acks settings, which would have caused silent data loss under network partitions.

Behavioral — Feedbackp.365

T — Task

They were more senior among the interns and I was unsure how they'd receive feedback from me.

A — Action

I left a code review comment that was specific and explained the 'why': 'Setting acks=1 here means a broker crash between acknowledgment and replication could lose this message — for pricing events that's a real risk. Here's how to set acks=all with retries.' No judgment, just the technical consequence and the fix.

R — Result

They updated the configuration and thanked me in a follow-up comment. It opened up a broader conversation about Kafka reliability guarantees that the whole team benefited from. Good feedback is specific, factual, and explains consequences — not character.

Behavioral — Feedbackp.366

Behavioral — Initiative

Initiative

Behavioral · 4 questions

Behavioral — Feedback

p.367

Behavioral — Initiative

Q8. Describe a time when you went above and beyond the requirements for a project.

Story 1: Building the Ericsson Dashboard Unprompted

S — Situation

At Ericsson, my assigned scope was building the Kafka microservice and the event-driven pricing system. Nobody asked me to build any observability tooling.

T — Task

But I noticed engineers spending 60+ minutes per incident trying to trace events through the distributed pipeline with no visual tooling — just raw logs.

A — Action

On my own initiative, I scoped, designed, and built a React dashboard that visualised distributed pipeline timelines in real time. I aligned with two engineers on what data points mattered most, built it in React, and shipped it within the internship window.

R — Result

The dashboard cut debugging and resolution time from 60 minutes to 10 minutes for a 30-engineer team. My manager called it one of the most impactful contributions of the entire internship cohort — and it was never on my sprint board.

Story 2: 50+ Tests at Noon

Behavioral — Initiative

p.368

S — Situation

At Noon, the expectation for the IoT pipeline project was basic test coverage — maybe 15–20 unit tests to verify core flows.

T — Task

I believed that for a pipeline handling real sensor data, basic coverage wasn't enough. Silent failures in IoT systems cost real money.

A — Action

I wrote 50+ unit and integration tests using SpringBoot, JUnit, and Postman — covering concurrent writes, edge cases in the MQTT broker reconnection logic, and failure scenarios in the PostgreSQL ingestion layer. I went well beyond what was asked, on my own time.

R — Result

Backend reliability improved by 80% through catching issues before production. No pipeline incidents occurred during my remaining tenure. My manager pointed to the test suite as a model for the team going forward.

Story 3: Open Source Contributions

S — Situation

Contributing to open source wasn't part of any job description or academic requirement — I chose to do it entirely on my own.

T — Task

I wanted to work on production systems at scale that millions of developers and companies depend on — facebook/react, apache/kafka, pgjdbc, kubernetes-client/java.

Behavioral — Initiative

p.369

A — Action

I found real bugs, studied the codebases deeply enough to propose fixes, wrote proper test cases and documentation, and engaged with maintainers through the review process. I got PRs merged into pgjdbc and kubernetes-client/java, and have an active PR against the React 19 reconciler targeting a confirmed StrictMode regression.

R — Result

I now have production contributions in systems powering Meta, LinkedIn, and JPMorgan. Nobody asked me to do this — I did it because I want to be the kind of engineer who shapes the tools we all use.

Behavioral — Initiative

Q18. Describe a situation where you saw a problem and took the initiative to correct it rather than waiting for someone else.

Story 1: Ericsson React Dashboard

S — Situation

At Ericsson, engineers on the team were spending 60+ minutes per incident tracing events through the distributed Kafka pipeline using raw logs with no visual tooling.

T — Task

Nobody had been assigned to fix this — it was background pain everyone accepted. I decided not to.

A — Action

I proactively scoped a React dashboard, aligned with two engineers on the most useful data points, built it on top of our existing pipeline metrics, and shipped it within the internship window without it ever appearing on a sprint board.

R — Result

Debugging and resolution time dropped from 60 minutes to 10 minutes for a 30-engineer team. My manager called it one of the most impactful contributions of the internship. The initiative cost me evenings — it returned hours to thirty engineers every week.

Story 2: Automated Tests at Noon

Behavioral — Initiative

p.371

S — Situation

At Noon, I noticed the IoT pipeline had no automated test suite — just manual testing before each deployment.

T — Task

Manual testing at this cadence was unsustainable and risky for a pipeline handling live sensor data. I decided to fix it.

A — Action

I proposed a testing framework to my manager, got a quick green light, and built 50+ unit and integration tests using SpringBoot, JUnit, and Postman — covering normal flows, edge cases, and failure scenarios the team hadn't been testing at all.

R — Result

Backend reliability improved by 80%. The team adopted the test suite as standard and it became the baseline for subsequent development. I took initiative not because it was glamorous but because the risk of not doing it was obvious.

Story 3: NSBE Partnership Drive

S — Situation

When I became NSBE Assistant VP, the chapter had no corporate partnerships and was entirely self-funded through small dues.

T — Task

Funding was limiting what we could offer members. Nobody had been tasked with fixing this — it was just accepted.

Behavioral — Initiative

p.372

A — Action
I built a sponsorship deck, identified companies whose engineering culture aligned with our membership's interests, and cold-contacted 12 companies over two months. I personalised each pitch around their specific campus recruiting goals and our member demographics.
R — Result
We secured \$3K in partnerships with multiple companies. That funding let us run events that wouldn't have been possible otherwise. Taking initiative on the unglamorous work — sales, outreach, follow-ups — is often what actually moves an organisation forward.

T — Task
I had to push against timeline pressure and the team's enthusiasm for getting into the field, because I believed a 32% model in a clinical context would undermine trust in AI tools in ways that were very hard to recover from.
A — Action
I prepared a specific argument: not just 'the accuracy is too low' but what a 32% model would mean in terms of false negatives on stroke diagnosis, what it would look like to a doctor who got the first prediction wrong, and how that experience could close the door on AI tools in that institution permanently. I paired the risk with a concrete plan — four weeks, specific improvements, a milestone accuracy target.
R — Result
The team agreed to delay. We reached 45% accuracy with proper confidence scores. The pilot reduced time-to-treatment by 35% and the hospital adopted the tool. Pushing for the delay was the most important thing I did on that project. The best outcomes sometimes require slowing down.

Behavioral — Initiative
Q55. What is something that you had to push for in your previous projects?
Story 1: Pushing for the React Dashboard at Ericsson
S — Situation
At Ericsson, engineers were spending 60+ minutes per incident tracing events through the Kafka pipeline using raw log files. There was no visual tooling and nobody had been assigned to fix it.
T — Task
I had to push for this project to exist — it wasn't on any sprint board, it wasn't in my official scope, and my initial mention of it got a polite 'we'll think about it' response.
A — Action
I built a working prototype in my own time, brought it to my manager with concrete numbers: 60-minute average debug time, 30 engineers affected, rough calculation of hours saved per week. I framed it not as a nice-to-have but as an engineering productivity investment with a clear ROI. When the first reaction was still hesitation about the timeline, I offered to build it in parallel with my other deliverables and absorb the time myself.
R — Result
My manager gave the green light. The dashboard shipped and cut debugging time from 60 minutes to 10 minutes for the full 30-engineer team. The biggest thing I pushed for in that role was the right to solve a problem I had identified myself. Sometimes the hardest part of initiative is getting the permission to act on it.

Behavioral — Initiative
Q56. What would you hope to achieve in the first six months after being hired?
Story 1: The 90-Day Production Contribution Plan
S — Situation
When I think about joining a new engineering team, I think about it in three phases: understand, contribute, and extend. Each phase has a different primary activity and a different definition of success.
T — Task
My goal in the first six months is to move through all three phases without skipping the first one, which is the phase most eager engineers rush past.
A — Action
In the first 30 days I focus entirely on understanding: the codebase, the architecture decisions, the testing culture, and the people. I ask questions, I read existing PRs, I sit with the debugging process. In days 30–90 I want to be shipping independently on well-scoped tasks — not the hardest problems, but real production work with no hand-holding. By month six I want to have identified one thing that isn't working as well as it should — a monitoring gap, a testing gap, a slow process — and to have fixed it proactively. That's the kind of contribution that builds long-term credibility.
R — Result
I built this pattern from my internship experiences. At Ericsson I was shipping independently within three weeks and had proposed and built the React dashboard — an initiative that wasn't in my original scope — before the internship ended. I want to replicate that trajectory in a full-time role.

S — Situation
At Noon, the IoT pipeline was heading toward launch with only manual testing in place. My manager's position was that 15–20 unit tests were sufficient and we should prioritise delivery speed.
T — Task
I believed that position was wrong for a pipeline handling live sensor data from physical devices, and I had to make that case clearly without undermining my manager.
A — Action
I came to the conversation with two specific examples of edge cases our manual testing had already missed, a timeline showing the extra tests would add only two days to the schedule, and a risk framing that connected test coverage directly to the 40% downtime-reduction goal we were all accountable for. I made it easy to say yes by making the cost concrete and the alternative concrete.
R — Result
I got the two days. I built 50+ tests. Two real bugs were caught — an MQTT reconnection issue and a PostgreSQL write race condition — that would have hit production. Backend reliability improved by 80%. Pushing for the tests was the right call, and having a data-backed case made the push successful.

Story 3: Pushing for a Hospital Pilot Delay on MedStroke
S — Situation
The MedStroke team was ready to run the hospital pilot with our model at 32% accuracy. I was the one advocating for a four-week delay to push accuracy higher and refine the clinical framing of the tool.

Story 2: The Knowledge Transfer Goal
S — Situation
One of the things I noticed at Ericsson was that tribal knowledge about the Kafka pipeline lived in the heads of three engineers. When any of them were out, debugging slowed to a crawl.
T — Task
In the first six months at a new role I want to be a net contributor to knowledge sharing, not just a consumer of existing knowledge.
A — Action
Concretely, I want to document the systems I touch as I learn them — not in exhaustive write-ups, but in the kind of runbook or annotated diagram that I would have wanted to find when I was trying to understand the system myself. I want to be someone who leaves every codebase slightly more legible than it was before. By month six I also want to have given at least one internal knowledge-share or lightning talk on something I've learned, even if it's small.
R — Result
The React dashboard I built at Ericsson was partly a knowledge-sharing tool — it gave engineers visibility they didn't have before. I want to build that kind of compounding contribution from day one, not wait until I've 'earned' the right to improve things.
Story 3: The First Real Ownership Milestone
S — Situation
I know from my internship experiences that the difference between a junior engineer who grows quickly and one who stagnates is usually ownership — whether they treat their work as something assigned to them or something they're responsible for.

T — Task

In the first six months I want to reach a point where there is at least one system or component where I am the person the team goes to, not the person who routes questions to someone else.

A — Action

I plan to get there by going deeper than the task requires: reading the history of the code I'm touching, understanding why decisions were made, asking about the failure modes that were designed around. At Noon, becoming the go-to person on the IoT pipeline data model happened because I had read every migration and every related PR — not because I'd been there longer than others. I'll do the same thing in a full-time role.

R — Result

By month six I want my manager to be able to say, 'ask Emmanuel about that part of the system.' That's my definition of a successful first six months: real ownership of at least one meaningful piece of the product.

Behavioral — Judgment

Judgment

Behavioral · 3 questions

Behavioral — Judgment

Q51. How would you respond if you were the last member of the team in the office on a Friday afternoon and the product owner asks you to develop and deploy a change to production?

Story 1: Risk Assessment First

S — Situation

This has a direct parallel to my Ericsson experience — I was the engineer on call for the Kafka pipeline on a Friday when a non-critical configuration change was requested by a stakeholder.

T — Task

My immediate response was to assess the risk before giving any answer, not to automatically say yes or no.

A — Action

I asked three questions: Is this change reversible quickly if something goes wrong? Is there a monitoring system active over the weekend? And — most importantly — is there a business reason this can't wait until Monday when the full team is available? If the answers are 'yes, yes, no' — the change can wait. If the answer to the last question is yes, we assess further.

R — Result

In my specific case, the change could wait. I communicated that clearly to the stakeholder with a concrete Monday morning commitment. They appreciated the clear timeline and honest reasoning more than they would have appreciated a rushed Friday deploy that failed over the weekend.

Story 2: The Responsible Engineer Framework

S — Situation

At Noon, I was once asked to push a quick fix to production on a Friday afternoon because a client was complaining about a UI issue.

T — Task

It was a one-line change — easy to make. But I was the only engineer available, had no one to review it, and if it broke something, I'd be debugging it alone on a Friday evening.

A — Action

I assessed the risk: the fix was in the frontend, the change was small, I had a rollback path, and the client issue was real and blocking them. I made the call to deploy — but I staged it first, smoke-tested it, confirmed the client issue was resolved before fully releasing, and documented the deployment and my reasoning. I also left a clear note for Monday about what had been deployed.

R — Result

The fix worked, the client was unblocked, and Monday the team had full context on what had changed. The key was not refusing by default or complying blindly — it was making a principled risk assessment and deploying responsibly with a rollback plan.

Story 3: When to Push Back Firmly

S — Situation

The scenario where a product owner asks for a Friday production deploy most often represents a planning failure, not a genuine emergency.

T — Task

My job as an engineer isn't just to execute — it's to protect production stability, and sometimes that means pushing back on requests that create unnecessary risk.

A — Action

If the request is driven by genuine urgency — revenue impact, security vulnerability, customer-blocking bug — I proceed carefully: staging environment first, smoke test, rollback plan ready, monitoring active. If the request is driven by impatience or poor planning — I explain the risk clearly and push back. 'Deploying on Friday afternoon with no team backup is how we end up debugging a production outage over the weekend. Here's what we can do Monday morning.'

R — Result

Engineers who can say no to bad deploys with clear reasoning are more valuable than engineers who say yes to everything. The product owner's short-term satisfaction is less important than not creating a production incident on a Friday. Good judgment means protecting the system even when it's uncomfortable.

Behavioral — Judgment

Q61. What frustrates you?

Story 1: Preventable Failures Without Process

S — Situation

What frustrates me most is when a failure happens that a clear process would have prevented, and the team's response is to fix the specific failure rather than the process gap that allowed it.

T — Task

At Noon, early in the project there was a production incident caused by a deploy that had no smoke test. It was caught quickly, but the root cause was a missing step in the deployment checklist.

A — Action

I flagged the process gap directly: not as a criticism of the individual who deployed, but as a systemic issue. I proposed a specific, lightweight checklist — three steps, two minutes — and offered to write it up. The team adopted it. What would have frustrated me was if the incident had been treated as a one-time mistake and the process hadn't changed.

R — Result

We had no further deploy-related incidents during my time there. My frustration with preventable failures isn't personal — it's that they're by definition avoidable. I'd rather spend the energy on a process fix than repeat the same fire drill. I try to channel that frustration into a concrete proposal rather than a complaint.

Story 2: Ambiguity Without an Owner

S — Situation
The other thing that genuinely frustrates me is when a problem clearly exists, everyone knows it exists, but there's no owner and therefore no action. I ran into this at Ericsson with the debug tooling gap.
T — Task
Engineers were spending 60 minutes per incident in raw logs. Everyone acknowledged it was painful. Nobody had been assigned to fix it.
A — Action
My response to that frustration is always the same: decide whether I can own the fix myself. If yes, I take it on — which is exactly what I did with the React dashboard. If the fix is outside my scope or requires resources I don't have, I name the gap clearly to whoever can act on it and make a specific proposal, not just a complaint.
R — Result
The dashboard I built cut debugging time from 60 minutes to 10 minutes for 30 engineers. My frustration with ownerless problems is productive when I treat it as a signal to either act or escalate with a clear ask. The one version of frustration I've had to manage is impatience with the pace of change — I've learned that proposing a solution is always faster than waiting for one to appear.

Story 3: Overcommunication as a Substitute for Thought

S — Situation
The frustration I've had to manage most consciously is with communication that happens in place of thinking — long meetings where a document would have been faster, Slack threads that grow because nobody is willing to make a decision.

T — Task
I encountered this most acutely during the MedStroke project when a question about model accuracy thresholds generated a long async thread with ten opinions and no resolution.
A — Action
I ended the thread by writing a concise decision document: the tradeoff in two sentences, the options in three bullets, a recommendation, and a request for a single synchronous decision call. The call took twelve minutes. We had a decision. The thing that had frustrated me — the lack of a decision — was actually a symptom of a process problem, not a people problem. Async threads without a clear owner rarely converge.
R — Result
We moved forward on the model threshold decision and the pilot timeline. I've since learned to recognise when a conversation needs to change form — from async to sync, from discussion to document — and to be the person who makes that call. That's more useful than being frustrated by the length of a thread.

Behavioral — Judgment
Q62. What is something 90% of people disagree with you about?
Story 1: Building From Scratch to Learn
S — Situation
Most engineers I talk to think building tools from scratch that already exist — a database engine, a container runtime — is a waste of time when you could just use the existing tool.
T — Task
I hold the opposite view: that building a known system from scratch is one of the most efficient ways to develop deep engineering intuition, and that most engineers who skip this step pay for it later in subtler ways.
A — Action
I built NanoSQL — a SQLite clone in C with B-tree indexing — and a container runtime in Go using Linux namespaces and cgroups. Neither of these will ever be used in production. But when I debug a B-tree-related issue in PostgreSQL, I'm not reasoning by analogy from documentation — I know exactly what a B-tree does under rebalancing because I've written one. When I work with Docker networking, I understand what the runtime is actually doing with namespaces because I've built one. That depth shows up in how fast I debug and how well I design.

R — Result
I think 90% of engineers would say this time would be better spent building something real. My counter is that the real thing you build afterward is better precisely because you built the toy first. I'm not advocating for always reinventing the wheel — I'm advocating for reinventing it once, deliberately, in order to understand it. The engineers I most admire almost all have a version of this in their background.

Story 2: Junior Engineers Should Read Source Code Before Documentation

S — Situation
The conventional advice for learning a new library or framework is: read the docs first, then look at examples, then look at source code if you need to.
T — Task
I've come to believe this order is often wrong for engineers who want to understand systems deeply, and that for complex systems, reading source code early — not late — builds better mental models.
A — Action
When I needed to understand how the React reconciler handled concurrent rendering for my open source contribution, I read the source code before reading any blog posts or tutorials. The source code is the ground truth — blog posts are interpretations of the source code, often written when the version was different. Docs describe the intended behaviour; source code describes the actual behaviour. For debugging and for contributions, the source code is almost always the right starting point. Documentation is for API surface area; source code is for understanding.

R — Result
This view makes most junior engineers uncomfortable because source code is intimidating. But I'd argue the discomfort is part of the value — you can't build a comfortable mental shortcut for something you've never actually read. The engineers I've learned the most from have all been people who went to the source as a first instinct.

Story 3: The Right Time to Push Back is Before, Not After

S — Situation
Most engineers I've worked with tend to raise concerns about a technical decision after it's been made — in code review, in a post-mortem, or in a retrospective.
T — Task
I believe this is a significant engineering failure mode and that the right time to push back on a bad technical decision is always before it's made, even if the social cost feels high.
A — Action
At Ericsson, I pushed back on the initial REST polling architecture before the design was locked in — not because I was confident I was right, but because I had a specific concern and the correct time to surface it was during design, not after we'd shipped a system that would be expensive to change. At MedStroke I pushed for a four-week delay before the hospital pilot, not after a failed one. Both pushbacks required me to be the person in the room saying 'I think we should reconsider.' That's uncomfortable, especially as a junior engineer.

R — Result
Both decisions worked out. The architectural pushback at Ericsson led to a Kafka design that hit sub-200ms latency. The MedStroke delay led to a pilot that reduced time-to-treatment by 35%. I think 90% of engineers are conditioned to defer to decisions that have already been made because changing them is expensive. I believe the real cost is in not pushing back early enough. Being the person who raises the concern before it's a problem is one of the most valuable things a junior engineer can do.

Leadership

Behavioral · 2 questions

Q5. Describe a time when you led a team. What was the outcome?

Story 1: NSBE Chapter Growth

S — Situation

When I became Assistant VP of NSBE at Grambling State, the chapter had low attendance and no corporate partnerships.

T — Task

My goal was to rebuild energy around the chapter, grow membership, and create real opportunities for CS students.

A — Action

I reorganized our events calendar around what students actually needed — resume workshops, technical mock interviews, and networking mixers. I personally reached out to tech companies for sponsorships, pitching the value of our talent pipeline. I delegated event leads to officers so no single person was overwhelmed.

R — Result

We grew to 300+ active members and secured \$3K in company partnerships. Students who joined got access to opportunities they wouldn't have found otherwise, and the chapter became one of the more active technical communities on campus.

Story 2: MedStroke AI Project

S — Situation

MedStroke AI was a team project and I took the lead on the overall system architecture and the ML pipeline specifically.

T — Task

I needed to coordinate frontend, backend, and ML components while keeping the team aligned on the hospital pilot deadline.

A — Action

I set up weekly syncs, created a shared design doc so everyone understood how the pieces connected, and personally owned the highest-risk component — the Random Forest ML model. When we hit accuracy issues, I kept the team focused on what we could control: data quality, feature engineering, and the clinical framing of the tool.

R — Result

We shipped on time, piloted at a Louisiana hospital, and reduced time-to-treatment by 35%. The project became the centerpiece of my portfolio and demonstrated that I can lead both technically and operationally.

Story 3: Ericsson Dashboard Initiative

S — Situation

At Ericsson, nobody had been assigned to build visibility tooling for our distributed Kafka pipeline — engineers were debugging blind.

T — Task

I decided to take ownership of this gap and lead the effort to build a React dashboard for the team, even though it wasn't in my sprint.

A — Action

I scoped the dashboard myself, aligned with two other engineers on what data they needed most, and built it using React — visualizing distributed pipeline timelines in real time. I drove the entire delivery from design to deployment.

R — Result

The dashboard reduced debugging and resolution time from 60 minutes to 10 minutes for the 30-engineer team. My manager highlighted it as one of the most impactful contributions of the internship.

Q37. Describe a time when you needed to motivate a group of individuals or encourage collaboration during a particular project.

Story 1: NSBE Chapter Revitalisation

S — Situation

When I became NSBE Assistant VP, member engagement was low — people showed up to one event and didn't come back. The chapter had a reputation for being only loosely technical.

T — Task

I needed to change the culture and give people a reason to show up consistently.

A — Action

I surveyed members on what they actually needed — most said interview prep and peer community. I restructured our event calendar around those two things. I created a mentorship pairing system between senior students and freshmen. I celebrated every placement and internship offer publicly in the chapter — making progress visible motivated the people who hadn't got there yet.

R — Result

Membership grew to 300+ active members. Retention improved significantly. The key insight was that motivation follows relevance — people engage consistently when the community is directly solving their actual problems.

Story 2: MedStroke Team Through ML Struggles

S — Situation

Midway through MedStroke development, our ML model accuracy was stuck at 35% and the team's energy was flagging. The hospital pilot deadline was fixed.

T — Task

I needed to keep the team motivated and focused during a period where progress felt invisible.

A — Action

I shifted how I measured progress — instead of accuracy as the only metric, I highlighted every other thing going well: the backend was solid, the UI was clean, the NIHSS scoring was clinically accurate. I broke the ML problem into smaller experiments, each with its own result to report at standup. Progress became visible again even when the headline number wasn't moving.

R — Result

The team kept moving and we shipped. The hospital pilot reduced time-to-treatment by 35%. When motivation dips in technical projects, the fix is usually making progress visible — not giving speeches.

Story 3: Cross-Functional Collaboration at Ericsson

S — Situation

The Ericsson React dashboard required input from engineers across the team — each had strong opinions about what data mattered and limited time to engage.

T — Task

I needed to get useful input from busy engineers without making it feel like another meeting.

A — Action
I sent a one-page async survey with three specific questions — 'What's the first metric you check during an incident?' — and followed up individually with the two engineers whose answers were most detailed. I incorporated their input visibly and told them I had, which motivated continued engagement.
R — Result
The dashboard was shaped by real user needs rather than my assumptions. Engineers adopted it immediately because it showed what they actually cared about. Getting collaboration from busy people means making it easy and showing that their input mattered.

Behavioral — Learning & Adaptability

Learning & Adaptability

Behavioral · 8 questions

Behavioral — Learning & Adaptability
Q9. How do you handle a situation where you don't know the answer to a question?
Story 1: Learning F# at Ericsson
S — Situation
At Ericsson, my team needed me to build an event-driven system using Apache Kafka and F# — a functional language I had never used before.
T — Task
Rather than pretend I knew it or ask to swap to a language I was comfortable in, I committed to learning F# properly.
A — Action
I was upfront with my manager that F# was new to me and gave a realistic estimate of the learning ramp. I spent the first week going through F# documentation, functional programming patterns, and async workflows. When I got stuck, I asked specific targeted questions rather than broad 'how do I do this' questions — which got me better answers faster.
R — Result
I delivered the event-driven system in F# on time. It aggregated 10K+ pricing records into RESTful endpoints and enabled real-time access to previously disconnected historical trade data. Being honest about what I didn't know upfront built trust and let me learn faster.

Story 2: MQTT Protocol at Noon

S — Situation
At Noon, the IoT pipeline required expertise in the MQTT protocol — a lightweight messaging protocol for IoT devices I'd never worked with.
T — Task
I needed to design a reliable MQTT-to-PostgreSQL ingestion layer without previous experience in the protocol.
A — Action
I told my manager it was new territory, then spent two days on the MQTT spec and existing open source implementations before writing a line of code. When I had a specific question about QoS levels and message ordering guarantees, I asked a senior engineer — but came with a specific hypothesis first, not a blank question.
R — Result
The pipeline went live handling real sensor data with a 40% improvement in downtime detection. Admitting I didn't know something upfront and then doing the work to find out is faster than pretending and getting it wrong.

Story 3: React 19 Reconciler Internals

S — Situation
When I discovered a StrictMode regression in React 19, I needed to understand the reconciler internals — some of the most complex code in any JavaScript framework.
T — Task
I had no prior experience with React's internal reconciliation algorithm, and needed to go from zero to a credible fix proposal.

A — Action
I read the reconciler source, git-bisected the regression to a specific commit, wrote a minimal reproduction case, and posted a detailed issue before writing any fix. I was explicit in my PR about what I was confident in and what I wanted a maintainer to validate.
R — Result
The PR is active with maintainer engagement. Being transparent about the limits of my understanding — while still doing the hard work to find the root cause — is the approach that gets open source PRs taken seriously.

Behavioral — Learning & Adaptability
Q17. Tell me about a time when you had to deal with a significant change at work. How did you adapt?
Story 1: Learning F# Mid-Project at Ericsson
S — Situation
When I joined Ericsson's pricing data project, I found out the system I was building needed to be written in F# — a functional language I had never used.
T — Task
I had to get productive in F# fast while delivering a production system, not a toy project.
A — Action
I spent the first week fully in learning mode: F# documentation, async workflows, immutable data patterns. I asked senior engineers specific questions — not 'how does F# work' but 'how does F# handle this specific Kafka consumer pattern.' I treated functional programming as a mindset shift, not just syntax.
R — Result
I built the event-driven system in F# and it aggregated 10K+ pricing records into RESTful endpoints enabling real-time access to previously disconnected historical data. The change forced me to grow in a direction I wouldn't have chosen — and it made me a stronger engineer.

Story 2: React 19 Reconciler Changes

S — Situation
When React 19 shipped major changes to the reconciler, code paths I was working with in my open source contribution suddenly behaved differently than the React 18 documentation described.
T — Task
I had to adapt my mental model of React’s rendering behaviour and update my PR accordingly.
A — Action
I read the React 19 migration guide, the relevant RFC, and then the actual reconciler source code. I updated my reproduction case to confirm the regression still existed in the new version, rewrote my PR description to reference the React 19 behaviour explicitly, and added a new test for the changed code path.
R — Result
The PR stayed relevant through the React 19 release. Adapting to change in a fast-moving codebase requires going to primary sources — docs, RFCs, and source code — not waiting for tutorials.

Story 3: Batch to Real-Time at Noon

S — Situation
At Noon, the initial IoT data pipeline design used batch ingestion — processing sensor data every 5 minutes. Mid-project, requirements changed to real-time streaming.
T — Task
I had to pivot the architecture while preserving the work already done and without blowing the timeline.

A — Action
I assessed what was reusable — the PostgreSQL schema and the data transformation logic — and what needed to change: the ingestion layer needed to shift from scheduled pulls to MQTT streaming. I restructured the ingestion module around Kafka Connect and MQTT without touching the transformation layer.
R — Result
The real-time pipeline shipped on schedule. The architectural separation between ingestion and transformation made the pivot possible without a rewrite. Planning for change means keeping your layers clean.

Behavioral — Learning & Adaptability
Q20. Describe a time when you went out of your comfort zone. Why did you do it? What lessons did you learn?
Story 1: Contributing to facebook/react
S — Situation
Contributing to facebook/react meant submitting code that would run in production for millions of developers worldwide — a very different stakes level from any project I'd worked on before.
T — Task
I'd never worked on a codebase at that scale or complexity. The React 19 reconciler is some of the most sophisticated JavaScript ever written.
A — Action
I went in anyway. I spent weeks reading the reconciler source before writing a line, found a genuine StrictMode regression, wrote a minimal reproduction case, submitted a detailed issue, and opened a PR with a test. I was explicit about what I was confident in and invited maintainer review.
R — Result
The PR is active with maintainer engagement. Going out of my comfort zone into the most complex JS codebase in existence taught me that the gap between 'too scary to touch' and 'I can contribute here' is mostly just time spent reading source code.

Story 2: NSBE Public Leadership

S — Situation
Before becoming NSBE Assistant VP, I was comfortable in technical environments but had never led a large community or done public speaking regularly.
T — Task
The role required running chapter meetings for 300+ members, speaking to corporate sponsors, and representing student interests to university administration.
A — Action
I pushed myself to take every speaking opportunity, prepared obsessively for the first few, and actively sought feedback after each event. I also studied how the most effective NSBE leaders at national conferences communicated — specific, story-driven, energetic.
R — Result
By the end of the first semester I was comfortable leading meetings, negotiating with sponsors, and speaking at events. I secured \$3K in partnerships partly on the strength of in-person pitches. Going out of your comfort zone in communication skills compounds — every opportunity makes the next one easier.

Story 3: Building in Go and C from Scratch

S — Situation
Go and C were outside my primary tech stack when I decided to build Go Container and NanoSQL.

T — Task
I chose those languages deliberately — Go for its systems concurrency model, C for direct memory management — because I wanted to understand what higher-level languages abstract away.
A — Action
I started each project by reading language specs and existing implementations before writing code. For Go Container I read the Linux namespaces and cgroups kernel documentation. For NanoSQL I read the SQLite design document before touching a B-tree.
R — Result
I built a container runtime with process isolation, namespace separation, and resource controls — and a database engine with persistent B-tree storage. The discomfort of working in unfamiliar languages at the systems level gave me a depth of understanding I couldn't have gotten any other way.

Behavioral — Learning & Adaptability
Q38. What do you do to enhance your technical knowledge apart from your project work?
Story 1: Open Source Contributions
S — Situation
I contribute to facebook/react, apache/kafka, pgjdbc, and kubernetes-client/java — production systems used by millions of developers and companies.
T — Task
Contributing to these codebases requires understanding them at a depth that tutorials and documentation don't provide.
A — Action
I find real bugs, study the source code to understand the root cause, write reproduction cases, propose fixes with tests, and engage with maintainer feedback. I've had PRs merged into pgjdbc and kubernetes-client/java, and have an active PR against the React 19 reconciler targeting a confirmed StrictMode regression.
R — Result
I now understand how PostgreSQL manages connections at the protocol level, how Kafka consumers manage offsets under the hood, and how React's reconciler handles concurrent rendering. That depth of understanding compounds — every production system I use, I understand better than its documentation.

Story 2: Building Systems from First Principles

S — Situation
Beyond tutorials and courses, I build systems from scratch specifically to understand what higher-level tools abstract away.
T — Task
This is the most direct path to deep technical knowledge — you can't fake understanding when you have to implement it.
A — Action
I built NanoSQL — a SQLite clone in C with B-tree indexing and persistent storage — to understand how databases actually work. I built Go Container — a container runtime using Linux namespaces and cgroups — to understand what Docker actually does. Both projects required reading primary sources: database papers, Linux kernel documentation, language specifications.
R — Result
I use PostgreSQL, Docker, and Kafka with a depth of understanding that makes me faster to debug, better at system design, and more valuable in code reviews. First-principles projects are the fastest path to real expertise.

Story 3: Following Primary Sources in Real Codebases

S — Situation
Most engineers follow newsletters and blog posts to stay current. I follow the actual codebases, RFCs, and issue trackers of major projects.
T — Task
Blog posts tell you what changed. Source code and RFCs tell you why — which is what matters for engineering decisions.

A — Action
I followed the React 19 RFC process from proposal to reconciler implementation. I read Kafka KIPs for consumer group protocol changes. I subscribe to PostgreSQL commit digests. When something changes, I read the PR that introduced it — not just the release notes.
R — Result
This gives me 6–12 months of lead time on where major tools are moving, because I'm reading the discussions while they're still open. It also means when I contribute open source work, I understand the codebase well enough to write fixes that match the maintainers' intent.

Behavioral — Learning & Adaptability
Q42. What will be your course of action if you are assigned a task you don't know at all?
Story 1: Learning F# at Ericsson from Zero
S — Situation
At Ericsson I was assigned to build an event-driven system in F# — a functional language I had never written a line of.
T — Task
I had to get productive in an unfamiliar language quickly while delivering a production system on a real timeline.
A — Action
First, I was upfront with my manager about the learning curve and gave a realistic revised estimate. Then I spent the first five days fully on learning — F# documentation, async/await patterns, functional data transformation. I built a small throwaway prototype to validate my understanding before touching production code. When I got stuck, I asked specific, targeted questions rather than broad ones.
R — Result
I delivered the F# system on time — aggregating 10K+ pricing records into RESTful endpoints. The key was being honest about the ramp, investing in learning before building, and asking precise questions. Pretending to know something you don't is the most expensive mistake in engineering.

Story 2: Learning MQTT Protocol at Noon

S — Situation
At Noon I was assigned to build an IoT pipeline using MQTT — a protocol I'd never worked with.
T — Task
I needed to design a reliable production system around a technology I was starting from zero on.
A — Action
I read the MQTT specification before writing any code — specifically the QoS levels, message ordering guarantees, and reconnection protocol. I built a small local test harness with a real MQTT broker to validate my understanding of edge cases before integrating it into the pipeline. I also looked at the Kafka Connect MQTT connector documentation and source.
R — Result
The pipeline shipped with correct MQTT QoS configuration and reconnection handling. The load reduced downtime detection by 40%. Learning a new protocol correctly from the spec is faster than learning it incorrectly from examples — you avoid a class of bugs that are nearly impossible to debug after the fact.

Story 3: React 19 Reconciler — Open Source

S — Situation
When I found the React 19 StrictMode regression, investigating it required understanding the React reconciler — code I'd never read before and that has no beginner-friendly documentation.
T — Task
I had to understand it well enough to identify a root cause and propose a fix.

A — Action
I started by reading the most recent reconciler RFCs to understand the design intent. Then I used git-bisection to narrow the problem to a specific commit rather than reading all the code. Once I had the specific commit, I read only the changed lines and the surrounding context. I wrote a minimal reproduction case to validate my understanding before proposing any fix.
R — Result
I identified the root cause and opened a PR with a regression test. The maintainers confirmed the issue. Approaching complex unknown code: bisect to narrow, read the minimum necessary, validate with a reproduction case before proposing a solution.

Behavioral — Learning & Adaptability
Q47. What is something new that you've learned recently?
Story 1: F# Functional Programming at Ericsson
S — Situation
At Ericsson I had to learn F# — a functional-first .NET language — to build the event-driven pricing system.
T — Task
F# wasn't a weekend tutorial situation. I was building production systems in it under a real deadline.
A — Action
I spent a focused week on F# async workflows, discriminated unions, and pipeline operators. The biggest conceptual shift was treating data transformation as a series of composable functions rather than mutable state. I built a throwaway prototype to validate my understanding before touching production code.
R — Result
I built and shipped the F# system — aggregating 10K+ pricing records into RESTful endpoints with real-time access to previously disconnected data. More importantly, learning functional programming changed how I think about state management in every language. F# made me a better Python and TypeScript developer.

Story 2: Linux Namespaces and cgroups for Go Container

S — Situation
Building Go Container required learning Linux namespaces and cgroups at the kernel level — documentation that assumes C systems programming knowledge, not Go.
T — Task
I needed to understand how the Linux kernel implements process isolation well enough to use it correctly from Go.
A — Action
I read the Linux man pages for clone(), unshare(), and the cgroups v2 interface. I learned each namespace type independently — PID, mount, UTS — tested each in isolation before combining them. I used strace to observe the system calls my container runtime was making to verify they matched the kernel documentation.
R — Result
I built a working container runtime that correctly isolates processes, enforces resource limits, and maintains filesystem separation. Learning kernel primitives at this level changed how I think about every containerised system I work with. I can now read a Kubernetes pod spec and understand what kernel calls it's generating.

Story 3: React 19 Reconciler Internals

S — Situation
Working on my open source React contribution required understanding the React 19 reconciler — specifically how it handles StrictMode double-invocation in concurrent rendering mode.

T — Task
The reconciler is deliberately underdocumented because it's considered an internal implementation detail. I had to learn it from source code alone.
A — Action
I read the fiber reconciler source systematically — starting from the work loop, through the commit phase, to the StrictMode-specific code paths. I used git-bisection to narrow the relevant changes to specific commits, which reduced the reading surface significantly. I wrote a minimal reproduction case that confirmed my understanding of the mechanism before writing any fix.
R — Result
I can now read React's fiber reconciler and reason about how rendering decisions get made. That understanding is directly applicable to debugging subtle React bugs in production that surface as 'the component renders unexpectedly.' Most engineers debug React by trial and error — understanding the reconciler means debugging from first principles.

Behavioral — Learning & Adaptability
Q58. What is your biggest strength and area of growth?
Story 1: Strength: Deep Systems Thinking
S — Situation
My biggest strength is that I understand systems at multiple levels of abstraction simultaneously — I can reason about why a distributed system behaves unexpectedly at the protocol level and also explain the business consequence to a non-technical stakeholder.
T — Task
That strength has been most valuable when something breaks in a way nobody anticipated, and someone needs to figure out why quickly.
A — Action
At Ericsson I traced a subtle latency spike to uneven Kafka partition key distribution by reading consumer lag metrics, JVM GC logs, and partition assignment data simultaneously. At Noon I found that two seemingly unrelated production issues — message loss and PostgreSQL slowness — had the same root cause: connection pool exhaustion. In both cases the ability to hold multiple layers of a system in mind at once led to a faster, better fix. I built this strength by working at low levels deliberately: building a container runtime in Go with Linux namespaces and cgroups, and a SQLite clone in C with B-tree indexing.
R — Result
My area of growth is communication speed: sometimes I arrive at a conclusion internally before I've brought others along. I'm working on narrating my reasoning as I go, not just presenting the answer. I've gotten better at this — at Ericsson I wrote the root cause analysis in a shared doc even before the fix was deployed — but it's still my active growth edge.

Story 2: Strength: Bias for Impact

S — Situation
I have a strong bias for work that has a measurable outcome, not just work that is technically interesting. That shapes how I prioritise, what I build, and when I push back.
T — Task
The clearest example is the React dashboard at Ericsson — nobody asked me to build it, but I calculated that 30 engineers losing 60 minutes per incident was a recoverable problem, so I built the thing.
A — Action
I think this strength shows up most in how I frame decisions. When my manager hesitated on the dashboard, I didn't argue that it was a good idea in the abstract — I calculated hours saved per week across the team and turned it into a ROI conversation. When I pushed for the hospital pilot delay on MedStroke, I framed it around what a false negative meant in a clinical context, not just 'the accuracy isn't good enough.'
R — Result
My growth area here is patience: because I'm focused on impact, I sometimes want to move to execution faster than is appropriate when I'm in a new environment and haven't yet built the full context. I've learned from that — the first 30 days in a new role should be dominated by listening, not building. That's a lesson I took from my internship onboarding and now apply deliberately.

Story 3: Strength: Speed of Learning

S — Situation
I learn new technical material very quickly, especially when I can engage with it directly rather than passively. I've consistently moved from zero to productive in new technologies faster than expected.
T — Task
At Ericsson I had never used F# before joining. I needed to be productive in it within days, not weeks, because I was the engineer on a live deliverable.
A — Action
I built a structured approach to learning new technologies: start with the language's official documentation, then read the source code of a project that uses it well, then build a small thing to surface what I don't understand. I used that approach for F# at Ericsson, for MQTT at Noon, and for the React reconciler internals in my open source contributions. It has a consistent track record.
R — Result
My growth area is depth over breadth. I'm naturally drawn to learning across many areas — systems, ML, frontend, open source — and I've had to consciously choose to go deeper in certain areas rather than adding new ones. For the next stage of my career, I'm deliberately concentrating on distributed systems and backend engineering rather than widening my surface area further.

Behavioral — Learning & Adaptability
Q59. What aspects of your work are most often criticized?
Story 1: PR Scope: The kubernetes-client Lesson
S — Situation
The most consistent criticism I've received in my open source work is that my PRs are sometimes too large to review comfortably. A maintainer on kubernetes-client/java told me directly that my PR was too broad to be reviewed safely as a single unit.
T — Task
I disagreed initially — the changes were tightly coupled — but I needed to understand the maintainer's perspective and respond to it constructively.
A — Action
I wrote a detailed comment explaining the dependency chain between the changes. I added inline documentation, added granular commit messages to make the diff easier to follow, and offered a live review call. What I took from the experience wasn't just 'split your PRs' but 'reviewers have limited cognitive bandwidth and your job is to manage it.' I now plan PRs with review ergonomics in mind from the start — even when the changes are logically coupled, there are ways to present them that reduce reviewer load.
R — Result
The PR was merged. I now routinely write a 'review guide' comment at the top of complex PRs explaining the logical order in which to read the changes. That habit came directly from this criticism. I've had maintainers explicitly thank me for it on subsequent contributions.

Story 2: Moving Too Fast in New Environments

S — Situation

At Noon, a senior engineer gave me feedback that I had proposed a major refactor of the MQTT ingestion layer before I had fully understood the constraints that had shaped the original design.

T — Task

He wasn't wrong. I had identified a real inefficiency, but I'd jumped to a solution without knowing why the existing approach existed.

A — Action

I took the feedback seriously. I went back and read the git history, the original design doc, and the ticket that had introduced the current approach. There was a valid reason for it — a specific MQTT broker limitation that the original engineer had worked around. My proposed refactor would have reintroduced a bug that had already been fixed once. I revised my proposal to preserve the workaround while still addressing the inefficiency I'd found.

R — Result

The revised proposal was accepted and improved pipeline throughput. The criticism taught me that in a new codebase, the correct order is: understand fully, then propose. Moving fast in an unfamiliar system is how you confidently reintroduce old bugs. I've built a deliberate 'read the history first' habit since then.

Story 3: Over-Engineering for Current Requirements

S — Situation

On MedStroke, one of my teammates pointed out that the data transformation pipeline I built was significantly more complex than our current data volume required. He wasn't wrong — I had designed it for eventual scale rather than our immediate needs.

Q30. Why are you interested in working at [company name]?

Story 1: For a Systems / Infrastructure Company

S — Situation

I've spent the last two years building systems at the infrastructure level — Kafka microservices at Ericsson, IoT pipelines at Noon, a container runtime and database engine in personal projects.

T — Task

I'm looking for a company where that depth of systems thinking is valued and where the engineering problems are genuinely hard.

A — Action

Your team's work on [specific system/product] is exactly the kind of problem I want to work on. The [specific technical challenge they solve] requires the kind of distributed systems thinking I've been developing — and I want to do it at a scale where the engineering decisions have real consequences.

R — Result

I'm bringing production experience with Kafka, Kubernetes, and event-driven systems, open source contributions to production-grade codebases, and the habit of going to first principles. I want to grow at a company where that approach is matched by the people around me.

Story 2: For a Product / Consumer Company

T — Task

He raised this as a maintenance concern: if I was the only one who understood the abstraction layers I'd added, that was a problem when I left the project.

A — Action

I simplified two abstraction layers, removed a caching mechanism that wouldn't be needed until we had 100x the current data volume, and documented the simpler version thoroughly. I kept the extensibility points that were genuinely likely to be needed, but removed the ones that were speculative. The exercise taught me to ask 'what is the simplest thing that solves this problem right now, with a clear path to extend it when needed?' before adding complexity.

R — Result

The simplified version was easier to maintain and faster to debug during the hospital pilot. I still think about future extensibility — that's part of being a good engineer — but I now distinguish between extensibility you know you'll need and extensibility you're adding because it feels right. The latter is often the source of this criticism.

S — Situation

At Ericsson I built a React dashboard that reduced a 30-engineer team's debugging time from 60 to 10 minutes. At Noon I built a notification system delivering updates in under 10ms. In both cases the most satisfying part was seeing engineers and users actually change their behaviour because of what I built.

T — Task

I want to work on products that people use and care about — where the feedback loop between engineering decisions and real user outcomes is short and clear.

A — Action

Your product reaches [X users / market segment] and the engineering challenge of making it fast, reliable, and scalable at that level is exactly what I want to work on. I've followed [specific product decision or engineering blog post] and it shows the kind of engineering rigour I want to be part of.

R — Result

I want to build things that matter at scale. Based on what I've read about your engineering culture and the problems you're solving, I think this is where I can do that.

Story 3: For a Healthcare / Mission-Driven Company

S — Situation

I built MedStroke AI and piloted it at a Louisiana hospital. Watching it reduce time-to-treatment by 35% — and knowing that matters for patient outcomes — was a completely different feeling than shipping a dashboard metric.

Motivation

Behavioral · 4 questions

T — Task

I want to work on technology where the stakes are real and the mission is something I'd work on even if it were harder than the alternatives.

A — Action

Your work on [specific healthcare/mission product] is solving a problem that matters in the same way. The technical challenges — reliability, data sensitivity, regulatory considerations — are harder than most software problems, and I think that's what makes them worth working on.

R — Result

I have experience building software that clinicians and real users depend on, and I want to keep doing work where the engineering quality directly affects outcomes for real people.

Behavioral — Motivation
Q34. Why do you want to change your current company?
Story 1: After Internship / Apprenticeship
S — Situation My Ericsson apprenticeship was a genuinely great experience — I shipped a Kafka microservice processing 400K+ daily events, built observability tooling that helped 30 engineers, and learned distributed systems in a real production environment.
T — Task I'm looking for a full-time role that builds on that foundation with new scale, new problem domains, and engineers I can learn from.
A — Action I'm specifically looking for a role where I can go deeper on [distributed systems / ML / product engineering], work with a team that has strong engineering culture, and take on more ownership than an apprenticeship allows.
R — Result What I built at Ericsson taught me what I'm capable of. I want to find out what's next — and I believe this role is the right environment to do that.
Story 2: Seeking Greater Scope and Impact
S — Situation Both at Ericsson and Noon I shipped systems with real impact — 400K daily events, 40% downtime reduction, 35% faster stroke treatment at a hospital. Each time, the most motivating moment was seeing real users change their behaviour because of what I built.
Behavioral — Motivation
p.427

Behavioral — Motivation
Q52. Where do you see yourself in 5 years?
Story 1: The Systems Architect Vision
S — Situation When I reflect on my trajectory — from building a Kafka microservice at Ericsson to an IoT data pipeline at Noon to a stroke diagnosis system used in a real hospital — I see a pattern: I gravitate toward backend systems that operate at scale and have real-world consequences.
T — Task My five-year goal is to build on that foundation and grow into a senior engineer who designs distributed systems that other engineers build on top of.
A — Action In the near term I want to deepen my expertise in distributed systems by working on production systems at companies like this one, where scale is real and the engineering bar is high. I plan to continue contributing to open source — I already have merged PRs in facebook/react and kubernetes-client/java — to stay close to how the best engineers in the industry think. I also want to develop my ability to lead technically: mentoring junior engineers, driving architecture decisions, and translating complex tradeoffs into clear choices for non-technical stakeholders.
R — Result In five years I see myself as a senior engineer who ships systems that scale to millions of users and raises the technical bar of the teams I work on. This role is a direct step toward that — which is a big part of why I'm here.
Story 2: The Healthcare Tech Track

T — Task I want a role where the scope of impact is larger and the engineering challenges are harder.
A — Action I've been deliberate about what I build outside of work — open source contributions to facebook/react and apache/kafka, systems projects in Go and C, a healthcare AI tool piloted at a real hospital. All of that points toward wanting to work on systems that matter at scale.
R — Result I'm not leaving for a small reason. I'm moving toward a problem I'm genuinely excited about, at a company whose engineering culture matches how I want to work.
Story 3: Student Seeking First Full-Time Role
S — Situation I'm graduating from Grambling State in May 2028 with a 3.93 GPA, two production internships, open source contributions, and projects ranging from a container runtime to a healthcare AI system.
T — Task I'm looking for a full-time role that's genuinely challenging — not a role where I'll be comfortable within six months.
Behavioral — Motivation
p.428

S — Situation MedStroke AI, my stroke diagnosis system piloted at a Louisiana hospital, changed how I think about engineering. Watching a clinical tool I built reduce time-to-treatment by 35% made abstract impact concrete in a way that a dashboard metric never could.
T — Task My five-year vision is shaped by that experience: I want to be an engineer whose work has consequences in the physical world, not just on a screen.
A — Action I'm pursuing this by staying close to backend and systems engineering — the layer where scale, reliability, and correctness matter most. I want to work in companies building infrastructure that powers critical systems, whether that's healthcare tech, logistics, or fintech. I intend to grow into a technical lead role by being someone who not only ships good code but helps define what good looks like for a team.
R — Result In five years I expect to be owning end-to-end technical delivery on a complex system and helping a team grow around me. I want to be the engineer people go to when a problem is hard — not just when it's urgent.
Story 3: The Open Source to Industry Bridge
S — Situation My open source contributions — across facebook/react, apache/kafka, pgjdbc, and kubernetes-client/java — have taught me how elite engineering teams think about API design, backwards compatibility, and system evolution. It's the best engineering school I've found.

A — Action I've been intentional about the kind of engineer I'm building toward — systems-level depth, production-scale experience, and open source contribution. I've sought out hard problems rather than comfortable ones. This role is aligned with that trajectory.
R — Result I want to join a team where I'll be learning from the people around me for years, not months. Based on what I know about your team, that's what I'd get here.
Behavioral — Motivation
p.429

T — Task My five-year goal is to carry that standard into industry and eventually give back to open source at the level of being a maintainer, not just a contributor.
A — Action I see the next two years as my time to compound professionally — taking on increasingly complex systems problems in a production environment, shipping at scale, and learning from senior engineers who have done it before. By year three or four I want to be the person at a company who bridges the gap between what the team ships and how the broader open source ecosystem evolves. That might mean proposing internal tools as open source, leading adoption of emerging standards, or speaking at a conference.
R — Result In five years I want my name associated with a system or open source project that other engineers depend on. That's the kind of impact I'm working toward, and companies building at real scale are where that journey happens.

Behavioral — Motivation

Q53. What does your best day of work look like?

Story 1: The Kafka Breakthrough Morning

S — Situation

My best days have a consistent shape: a hard problem that has been resisting me, an uninterrupted block of time, and a real user on the other side who needs it solved.

T — Task

The clearest example was during the Ericsson internship when I was tracking a latency spike in the Kafka microservice that only appeared under high-throughput conditions.

A — Action

I blocked my entire morning — no Slack, no standups — and dug into consumer lag metrics, partition rebalancing logs, and JVM GC pauses. By noon I had isolated the issue: one of the six topic partitions was consistently running hot because of uneven key distribution. I rebalanced the partition key strategy, deployed to staging, and watched latency drop from 280ms to under 180ms. I wrote up the root cause clearly so any engineer on the 30-person team could understand it.

R — Result

The system hit sub-200ms consistently after that fix. The best part wasn't the fix itself — it was the deep focus that made it possible. My best day is one where I go deep on something real and come out the other side with clarity.

Story 2: The MedStroke Pilot Day

Behavioral — Motivation

p.433

S — Situation

The day the MedStroke AI system ran its first live session at the Louisiana hospital pilot was the best work day I can remember.

T — Task

I was responsible for making sure the FastAPI backend, the Random Forest model, and the React frontend all ran reliably under real clinical conditions with no support from me in the room.

A — Action

The morning of the pilot I ran a full system check, confirmed the model was serving predictions with correct confidence scores, and walked through the clinical interface one last time with the hospital contact. Then I stepped back and let the tool run. When I saw the first blood sample data come through the system and a prediction appear in under two seconds, I knew it was working. I spent the rest of the day monitoring logs, answering questions, and taking notes on everything the doctors said.

R — Result

Time-to-treatment dropped by 35% in that pilot. My best day is a day when something I built is used by a real person to make a real decision, and it works the way I designed it to. That's the feeling I'm always trying to recreate.

Story 3: The Open Source Merge Notification

S — Situation

The day a PR I'd spent two weeks refining was finally merged into kubernetes-client/java stands out as one of my best work days, even though I was sitting alone at my desk.

Behavioral — Motivation

p.434

T — Task

The contribution fixed a subtle concurrency bug in the informer framework that affected users running high-throughput watch loops. Getting it right required understanding a complex threading model and writing tests that reproduced a race condition reliably.

A — Action

My best day is the one where everything clicks: I read the codebase deeply enough to understand why the bug existed, wrote a fix that didn't introduce new complexity, and got the reproduction test to catch the race on every run. The maintainer's final review comment was, 'this is exactly the kind of careful fix this codebase needs.' That sentence made the entire two weeks worth it.

R — Result

The PR was merged and shipped in the next release. My best day looks like this: deep work, a problem that genuinely resisted me, and feedback that tells me I raised the bar. The merge notification at the end is just punctuation.

Behavioral — Motivation

p.435

Behavioral — Pressure & Resilience

Pressure & Resilience

Behavioral · 4 questions

Behavioral — Motivation

p.436

Behavioral — Pressure & Resilience

Q6. Tell me about a time you worked well under pressure.

Story 1: Sub-200ms Latency at Ericsson

S — Situation

At Ericsson, two weeks before the Kafka microservice was due to launch, load testing showed our p99 latency was sitting at 380ms — nearly double our 200ms SLA.

T — Task

I had to find and fix the bottleneck fast without destabilising the rest of the pipeline.

A — Action

I profiled the consumer group rebalancing behaviour and discovered that our partition assignment strategy was causing excessive rebalances under load. I switched to a sticky partition assignor, tuned the session timeout and heartbeat interval, and re-ran load tests in a staging environment. I also stayed focused and avoided the temptation to make multiple changes at once — one variable at a time.

R — Result

Latency dropped to consistently under 180ms. We shipped on time. Working under that pressure sharpened my instinct for systematic debugging rather than panic-driven guessing.

Story 2: IoT Pipeline Outage at Noon

Behavioral — Pressure & Resilience

p.437

S — Situation

At Noon, our IoT sensor pipeline went down in production during a peak monitoring window — sensors stopped reporting and the ops team had no visibility into device status.

T — Task

I was the engineer who had built the MQTT-to-PostgreSQL ingestion layer and was responsible for getting it back up quickly.

A — Action

I stayed calm, checked the MQTT broker logs first, found a broker certificate expiry that was silently dropping connections, renewed the cert, and validated the fix in staging before pushing to production. I documented the issue and added a certificate expiry monitoring alert to prevent recurrence.

R — Result

The pipeline was restored within 40 minutes. The incident led to a monitoring improvement that would catch cert expiries 30 days in advance going forward. My manager cited my composure under pressure in the post-mortem.

Story 3: React 19 Open Source Deadline

S — Situation

I had an open PR against facebook/react targeting a confirmed StrictMode regression in the React 19 reconciler. The React team announced a release candidate window, meaning PRs needed to be review-ready within days.

T — Task

I had to get my PR to a mergeable state — clean, well-tested, and explained — under real time pressure in a high-visibility codebase.

Behavioral — Pressure & Resilience

p.438

A — Action

I cut every non-essential change from the PR to keep it focused, wrote a clear reproduction case, added a regression test, and wrote a thorough PR description explaining the reconciler behaviour. I responded to reviewer comments within hours.

R — Result

The PR stayed active through the RC window. Contributing to production systems at Meta's scale under time pressure confirmed that I can perform when the stakes are real.

Behavioral — Pressure & Resilience

Q16. Describe a time when your workload was heavy and how you handled it.

Story 1: Ericsson + Coursework Simultaneously

S — Situation

During my Ericsson apprenticeship I was simultaneously enrolled in courses and managing three concurrent deliverables: the Kafka microservice, the event-driven pricing system, and the React dashboard.

T — Task

All three had overlapping timelines and I was the sole engineer on each.

A — Action

I time-blocked my week rigidly — deep engineering work in mornings, meetings and reviews in afternoons, coursework in evenings. I identified which of the three projects had the hardest external dependencies and sequenced around those. I also dropped non-essential activities for the duration.

R — Result

All three shipped on time. The pipeline hit sub-200ms latency, the pricing system went live, and the dashboard cut debugging time from 60 to 10 minutes. Heavy workloads are manageable when you sequence deliberately instead of reacting.

Story 2: NSBE Leadership + School + Applications

S — Situation

In the semester I became NSBE Assistant VP I was also carrying 18 credit hours and applying to summer internships.

T — Task

Each of those three things was a full commitment on its own. I had to handle all three without any of them slipping.

A — Action

I treated my week like a product with limited capacity. I set fixed slots for NSBE work — Sunday planning, Tuesday events — and protected study blocks as non-negotiable. I batched internship application writing into one Saturday morning per week. I said no to commitments that didn't map to any of those three priorities.

R — Result

We grew NSBE to 300+ members that semester, I maintained a 3.93 GPA, and I landed the Ericsson offer. The key was treating my time like a constrained resource, not a flexible one.

Story 3: MedStroke + Open Source + Academics

S — Situation

During a particularly intense stretch I was developing MedStroke AI toward a hospital pilot deadline, actively reviewing PRs on pgjdbc and kubernetes-client/java, and finishing a Data Structures & Algorithms course.

T — Task

Each had real external deadlines — the hospital, the open source release windows, and my professor.

Behavioral — Pressure & Resilience

p.441

A — Action

I used a simple rule: external deadlines trump internal ones. Hospital pilot prep came first. Open source reviews I batched into 30-minute windows between study sessions. I used the algorithms coursework to deepen my understanding of the B-tree work I was doing in NanoSQL — learning that served multiple goals simultaneously.

R — Result

The hospital pilot launched on schedule. Two PRs merged. I got an A in the course. Finding tasks that serve multiple goals is one of the best ways to handle a heavy workload.

Behavioral — Pressure & Resilience

p.442

Behavioral — Pressure & Resilience

Q21. Describe a time when you delivered a project under a tight deadline.

Story 1: Ericsson Kafka Microservice

S — Situation

At Ericsson, the Kafka microservice processing 400K+ daily events had a hard launch deadline tied to a business reporting cycle — it couldn't slip.

T — Task

I had to deliver a production-quality distributed system with sub-200ms latency SLA in a fixed window, while also learning the codebase and the Ericsson deployment environment.

A — Action

I front-loaded the riskiest work — the Kafka consumer group configuration and latency tuning — so I had maximum time to fix problems. I ran load tests early rather than late, found the rebalancing issue two weeks before launch, and fixed it with time to spare. I cut scope on nice-to-haves without being asked.

R — Result

Launched on time, hitting sub-200ms latency from day one, replacing manual SQL workflows across the team. Tight deadlines reward front-loading risk.

Story 2: MedStroke Hospital Pilot

S — Situation

The Louisiana hospital had a fixed date for the MedStroke AI pilot — a date set before the ML model was even close to ready.

Behavioral — Pressure & Resilience

p.443

T — Task

I had to deliver a working stroke diagnosis system with role-based access, NIHSS scoring, and automated tPA eligibility within a non-negotiable window.

A — Action

I scoped ruthlessly — everything that wasn't directly patient-facing was deferred. I set daily checkpoints rather than weekly, so slippage was caught in hours not days. I delivered a checkpoint model for integration while still improving it in parallel.

R — Result

The pilot launched on time and reduced time-to-treatment by 35%. Daily checkpoints and ruthless scoping are the two most effective tools for tight deadline delivery.

Story 3: Noon IoT Pipeline

S — Situation

At Noon, the IoT pipeline had to go live before a client demo — and the client had already announced the demo date before we started building.

T — Task

Live sensor data had to flow through MQTT into PostgreSQL and appear in a monitoring dashboard in time for the demo.

Behavioral — Pressure & Resilience

p.444

A — Action

I separated the integration concerns clearly — MQTT ingestion, transformation, and PostgreSQL storage were built as independent, testable units. This let me parallelize work with another engineer without coupling. I ran end-to-end smoke tests daily in the final two weeks rather than waiting for a full integration test at the end.

R — Result

The pipeline was live for the client demo. Daily smoke tests meant we caught integration issues 48 hours before they would have been discovered otherwise. Parallel work and early integration testing are the fastest path through a tight deadline.

A — Action

I identified which NSBE events had external dependencies that made them non-negotiable (the career fair already had companies registered) and which I could scale back or delegate. I delegated two tasks to other officers with clear ownership. I dropped the personal project timeline rather than drop my coursework or the external-facing NSBE commitments.

R — Result

The career fair ran successfully, my GPA held, and the personal project shipped a month later. Knowing when to say 'this timeline isn't real' and adjusting early is a skill — not a failure.

Behavioral — Pressure & Resilience

Q41. Tell me about a time when you had an excessive amount of work and you knew you could not meet the deadline. How did you manage?

Story 1: MedStroke Beta Testing Feedback Flood

S — Situation

One week before the hospital pilot, we received extensive feedback from the beta clinical testers — the UI needed significant changes and two backend flows had edge cases that failed in real clinical scenarios.

T — Task

There was no way to address all of it in one week at the quality bar I'd set. I had to decide what to do.

A — Action

I triaged the feedback into three buckets: blocks the pilot (must fix), degrades the pilot (should fix), and nice to have (defer). I communicated the triage to the team immediately, set realistic expectations with the hospital contact, and focused every hour on the blockers. I was honest that two 'should fix' items would be in the first post-pilot update, not the launch.

R — Result

The pilot launched with all blockers resolved. The hospital contact appreciated the transparency — they preferred an honest conversation to a surprise. The two deferred items shipped in the following week. Managing the impossible deadline means triaging transparently, not trying to do everything badly.

Story 2: Ericsson Sprint Overcommitment

Behavioral — Prioritisation

Prioritisation

Behavioral · 3 questions

Story 1: Production Bug vs Feature Work at Ericsson

S — Situation

Midway through building the Kafka microservice at Ericsson, a production issue was flagged in an earlier component I'd worked on — pricing data was arriving out of order in downstream consumers.

T — Task

I had active sprint commitments on the new microservice but a live production issue needed immediate attention.

A — Action

I immediately communicated to my manager that I was switching context to the production issue. I triaged it first to confirm severity — it was affecting live reporting — then paused all non-critical feature work. I fixed the consumer offset management issue, tested in staging, deployed, and only returned to the microservice once the fix was confirmed stable.

R — Result

The production issue was resolved within the day. I then reprioritised the remaining microservice tasks to recover the schedule. My manager appreciated the transparency and decisiveness — switching context without communicating would have been worse.

Story 2: NSBE + Internship + School Simultaneously

Behavioral — Pressure & Resilience · Prioritisation · Mastery · DSA · System Design · Behavioral

p.449

S — Situation

At Ericsson, I accepted more scope in a sprint than I could realistically deliver — I underestimated the F# learning curve and the Kafka tuning work simultaneously.

T — Task

I realised three days in that I was going to miss commitments. The question was how to handle it.

A — Action

I told my manager immediately — not at the end of the sprint. I explained what I'd underestimated and offered two options: drop the F# work to this sprint and carry the Kafka tuning, or reverse. My manager chose — the decision was theirs to make with full information. I didn't pretend I could somehow do both.

R — Result

We re-scoped cleanly. Both pieces shipped in the right sprint. Telling your manager early about a slip costs you some discomfort. Telling them late costs the whole team time. Early transparency is always the right call.

Story 3: NSBE Semester Overcommitment

S — Situation

In one semester I'd committed to organising three large NSBE events, carrying 18 credit hours, and completing a personal project with a self-imposed deadline. Two weeks in I knew it was too much.

T — Task

Something had to give — I needed to decide what and communicate it.

Behavioral — Pressure & Resilience

p.447

Behavioral — Prioritisation

Q12. Tell me about a time when you had to prioritize your tasks quickly.

Story 1: Production Bug vs Feature Work at Ericsson

S — Situation

Midway through building the Kafka microservice at Ericsson, a production issue was flagged in an earlier component I'd worked on — pricing data was arriving out of order in downstream consumers.

T — Task

I had active sprint commitments on the new microservice but a live production issue needed immediate attention.

A — Action

I immediately communicated to my manager that I was switching context to the production issue. I triaged it first to confirm severity — it was affecting live reporting — then paused all non-critical feature work. I fixed the consumer offset management issue, tested in staging, deployed, and only returned to the microservice once the fix was confirmed stable.

R — Result

The production issue was resolved within the day. I then reprioritised the remaining microservice tasks to recover the schedule. My manager appreciated the transparency and decisiveness — switching context without communicating would have been worse.

Story 2: NSBE + Internship + School Simultaneously

Behavioral — Prioritisation

p.450

S — Situation
During my first semester as NSBE Assistant VP, I was simultaneously handling internship applications, coursework for four CS classes, and our biggest NSBE event of the semester.
T — Task
Everything felt urgent and I had to decide what actually was.
A — Action
I applied the urgency/importance matrix. NSBE event logistics had a hard date — that was time-boxed and delegated where possible. Coursework deadlines were fixed — I blocked study time daily and didn't let it slip. Internship applications I batched into weekend sessions. I said no to two extra NSBE commitments that week that weren't load-bearing.
R — Result
The NSBE event ran successfully, I finished the semester with my 3.93 GPA intact, and I landed my Ericsson offer. Prioritising well doesn't mean doing everything — it means knowing what not to do.

Story 3: Simultaneous Noon Deadlines

S — Situation
At Noon, during the final two weeks of the IoT pipeline project, a new bug was discovered in the PostgreSQL schema that affected data integrity, on the same day my test suite was due for review.
T — Task
Both tasks were high priority. The bug was a data integrity issue — silent failures — and the test suite review had a fixed team meeting it was anchored to.

A — Action
I assessed which had irreversible consequences: the data integrity bug, if left unfixed, would corrupt records silently. I triaged the schema bug first, patched and validated it in a few hours, then delivered the test suite to review with a brief explanation of why it was coming in late. I communicated the tradeoff before the review rather than after.
R — Result
Both were delivered the same day. My manager said the triage judgment was exactly right — the schema bug was the more dangerous risk.

Behavioral — Prioritisation
Q39. How do you prioritize your workload? What do you do when your work feels like it's just too much to get done?
Story 1: Three Concurrent Ericsson Deliverables
S — Situation
At Ericsson I had three concurrent deliverables: the Kafka microservice, the event-driven pricing system in F#, and the React dashboard — all with overlapping timelines.
T — Task
I had to sequence them without any of the three slipping.
A — Action
I identified the external dependencies for each: the Kafka microservice had an upstream team dependency that made it the critical path. I prioritised it first. The F# system had a fixed integration test date. I time-boxed its daily work to protect the Kafka development. The dashboard had no external dependencies — I worked on it in smaller blocks. I communicated my prioritisation to my manager explicitly so there were no surprises.
R — Result
All three shipped. Sub-200ms latency on the microservice, real-time pricing data access on the F# system, and 60→10 minute debugging improvement on the dashboard. Prioritisation is a communication act as much as a planning act — your manager needs to know what you're deprioritising and why.

Story 2: When Work Feels Overwhelming

S — Situation
During the MedStroke hospital pilot sprint, I hit a point where the ML model, the API, the frontend, and the clinical documentation all felt urgent simultaneously and I couldn't see a path through.
T — Task
Overwhelm is a signal that I've lost clarity on what's actually critical — not that there's too much work.
A — Action
I stopped and listed every task in one place. Then I applied one question: 'What breaks the hospital pilot if it's not done?' That list was four items. Everything else got explicitly deferred for the sprint. I told the team what I was deferring and why, so no one was surprised.
R — Result
The pilot launched on time. The deferred items were delivered in the following sprint with no negative consequence. Overwhelm comes from trying to hold everything in your head simultaneously — the fix is externalising the list and then being ruthless about what's truly on the critical path.

Story 3: School + NSBE + Applications

S — Situation
Balancing 18 credit hours, NSBE leadership responsibilities, and a competitive internship application cycle simultaneously felt genuinely unsustainable at one point.

T — Task
I needed a system that would let me make progress on all three without burning out or dropping any of them.
A — Action
I gave each priority a fixed weekly time budget rather than letting them compete in real time. NSBE got Sundays and Tuesday evenings — capped. Coursework got morning blocks — non-negotiable. Applications were batched to Saturday mornings. When something tried to expand outside its budget, I pushed back rather than absorbing it. I also identified which NSBE tasks could be delegated to other officers.
R — Result
3.93 GPA. NSBE grew to 300+ members. Ericsson offer. Fixed budgets and delegation are the answer to unsustainable workloads — not working more hours.

Behavioral — Prioritisation
Q44. Describe a time when you had to work simultaneously on both high-priority urgent projects as well as long-term projects. How did you handle both?
Story 1: Production Bug vs Kafka Feature at Ericsson
S — Situation
At Ericsson, midway through building the Kafka microservice (long-term), a production pricing data issue was reported in an earlier component — event ordering was broken and affecting live reporting.
T — Task
The production issue was urgent and high-severity. The Kafka microservice had a hard launch deadline. Both needed my attention.
A — Action
I triaged the production issue first to confirm severity — it was affecting live business reporting, which made it the immediate priority. I communicated the context switch to my manager explicitly. I fixed the ordering issue, validated in staging, deployed, then wrote up a postmortem before returning to the microservice. I adjusted my microservice timeline by one day and communicated that proactively.
R — Result
Production restored within the day. Microservice launched on schedule with the one-day adjustment. The key was explicit communication at every switch — no one was surprised by where my time was going.

Story 2: Hospital Pilot Prep + New Feature Build

S — Situation

During MedStroke development, I was simultaneously preparing the system for the hospital pilot (urgent — fixed date) and building new NIHSS scoring features for the full product roadmap.

T — Task

I couldn't let pilot prep slip, but stopping all feature work for two weeks would set the roadmap back significantly.

A — Action

I allocated mornings to the pilot-critical work — testing, clinical validation, documentation — and afternoons to roadmap features. I set a rule: any bug discovered in pilot prep immediately preempted afternoon feature work. I was transparent with the team about this prioritisation so everyone understood why some features were moving slowly.

R — Result

The pilot launched fully prepared. Two roadmap features shipped during that window at a slower pace. The time-box approach — mornings for urgent, afternoons for long-term — kept both moving without either consuming the other.

Story 3: NSBE Career Fair + Semester Research Project

S — Situation

As NSBE Assistant VP, I was running our biggest career fair (urgent, external companies registered, fixed date) while simultaneously completing a semester-long systems research project with a final submission date.

T — Task

Both had hard deadlines within the same two-week window.

A — Action

I mapped the dependencies. Career fair logistics were front-loaded — confirmations, room setup, communication — and mostly done by day 5. Research project writing was back-loaded. I scheduled the career fair work for the first week, the research project for the second, and gave myself one overlap day as buffer. I delegated two career fair tasks to other officers.

R — Result

Career fair ran successfully with 300+ member attendance. Research project submitted on time. Sequential rather than parallel execution is often faster when both tasks require deep focus — the context switching cost is real.

Behavioral — Problem Solving

Problem Solving

Behavioral · 3 questions

Behavioral — Problem Solving

Q13. Describe a time when you anticipated potential problems and developed preventive measures.

Story 1: Load Testing Before Ericsson Launch

S — Situation

Before launching the Kafka microservice at Ericsson to process 400K+ daily events, I anticipated that our consumer throughput had never been tested at production-equivalent volumes.

T — Task

Rather than find out at launch, I wanted to proactively stress-test the system and identify bottlenecks in a controlled environment.

A — Action

I set up a load testing environment that replayed a synthetic event stream at 2x our expected daily volume over a compressed time window. I monitored consumer lag, partition rebalancing frequency, and end-to-end latency. I found a rebalancing issue that appeared only under sustained load — invisible in normal testing.

R — Result

I fixed the partition assignor configuration before production launch. The system hit sub-200ms latency from day one with no consumer lag incidents. Anticipating that load scenario saved a potential production crisis.

Story 2: MQTT Reconnection Edge Cases at Noon

S — Situation

When building the IoT pipeline at Noon, I knew from research that MQTT brokers have known edge cases around reconnection under poor network conditions — something common in IoT environments.

T — Task

The standard implementation wouldn't handle this — I needed to build resilience in before we went live with real sensors.

A — Action

I implemented exponential backoff for reconnection attempts, added circuit breaker logic so the pipeline wouldn't hammer a recovering broker, and wrote integration tests that simulated network partitions by forcibly disconnecting the test broker mid-stream.

R — Result

During UAT, we deliberately killed the broker connection and the pipeline recovered gracefully with zero data loss. The preventive design directly contributed to the 40% reduction in downtime detection — the system didn't create new downtime while recovering from events.

Story 3: MedStroke False Positive Risk

S — Situation

Before piloting MedStroke AI at the hospital, I anticipated that a model producing false negatives (missing real stroke risk) would be far more dangerous than false positives in a clinical context.

T — Task

I needed to configure the model threshold to reflect clinical priorities — not just maximise overall accuracy.

A — Action

I tuned the Random Forest classification threshold to bias toward sensitivity over specificity, added a clear confidence score output so doctors could weight the tool appropriately, and flagged high-uncertainty predictions explicitly in the UI.

R — Result

The hospital doctors reported that the confidence display and threshold tuning made the tool feel clinically trustworthy. The pilot reduced time-to-treatment by 35% and the medical staff adopted it rather than ignoring it — which is the real metric for a clinical tool.

Q36. Tell me about a time when you were faced with a problem that had a number of possible solutions. What was the problem and how did you determine the course of action?

Story 1: Ericsson Latency Problem — Multiple Possible Fixes

S — Situation

When Ericsson load tests showed 380ms latency against our 200ms SLA, there were at least four plausible causes: network overhead, producer batch size, consumer processing time, and consumer group rebalancing.

T — Task

I needed to identify the right fix without trying all four simultaneously — which would make it impossible to know what worked.

A — Action

I isolated one variable at a time. I measured network latency first — negligible. Producer batch size — already optimal. Consumer processing time — within bounds. Consumer group rebalancing frequency — abnormally high under load. I'd found the root cause methodically. I fixed the partition assignor configuration and session timeout.

R — Result

Latency dropped to under 180ms. The systematic isolation approach found the fix in one day. When there are multiple possible solutions, the right process is elimination — not trying everything and seeing what sticks.

Story 2: MedStroke ML Model Selection

S — Situation

For MedStroke AI's stroke risk prediction, I evaluated five possible approaches: logistic regression, Random Forest, gradient boosting, SVM, and a small neural network.

T — Task

With our dataset size, timeline, and clinical interpretability requirement, I needed a principled way to choose.

A — Action

I scored each option against three criteria: interpretability for doctors, accuracy given our dataset size, and implementation time within our timeline. I ran baseline experiments on each. Gradient boosting edged Random Forest on accuracy but was harder to explain. Logistic regression was too simple. Random Forest hit the right tradeoff across all three criteria.

R — Result

Random Forest shipped. The decision framework — explicit criteria weighted by use case constraints — gave me a defensible choice rather than a preference. I now use this approach for any decision with multiple viable options.

Story 3: IoT Pipeline Architecture at Noon

S — Situation

When designing the Noon IoT pipeline, I had three viable options: a batch ETL approach, a REST polling approach, or MQTT with Kafka Connect.

T — Task

Each had real tradeoffs and the right choice depended on latency requirements and device count — both of which were partially unclear at the time.

A — Action

I wrote a one-page decision doc mapping each option against the requirements I was confident in: device count (large), network reliability (variable), PostgreSQL as the sink (fixed). MQTT with Kafka Connect handled variable network conditions best and scaled to device count without polling overhead. I presented the doc and got quick sign-off.

R — Result

The MQTT/Kafka Connect pipeline shipped and reduced downtime detection by 40%. Writing the decision doc forced me to make the requirements explicit — which clarified the decision faster than any technical debate would have.

Q49. Tell me a time when you linked two or more problems together and identified an underlying issue.

Story 1: Ericsson Latency + Throughput Drop — Same Root Cause

S — Situation

At Ericsson, during load testing, we saw two distinct problems: consumer latency spiking to 380ms (above our 200ms SLA) and throughput dropping by 40% under sustained load. The team initially treated them as separate issues.

T — Task

I suspected they might be related — both appeared simultaneously under the same load conditions.

A — Action

I looked at the timeline of both events in the broker logs and found they were perfectly correlated — every latency spike coincided with a throughput drop. That correlation pointed to a single event causing both. I traced it to consumer group rebalancing: when the consumer group rebalanced, all consumers paused processing momentarily, causing both throughput to drop and latency to spike simultaneously.

R — Result

Fixing the rebalancing frequency — one configuration change — resolved both problems. Latency dropped to under 180ms and throughput stabilised. Treating two symptoms as one underlying problem saved us from solving two problems separately and potentially missing the real cause entirely.

Story 2: Noon Message Loss + PostgreSQL Slowness — Same Root Cause

S — Situation

At Noon, we had two reported issues: intermittent message loss in the IoT pipeline (1-2% of sensor readings missing) and occasional PostgreSQL query slowness under peak load. The team was investigating them separately.

T — Task

I noticed both issues appeared during the same time windows — peak device reporting periods — and suspected a common cause.

A — Action

I added message-level tracing across the pipeline and cross-referenced with the PostgreSQL slow query log. I found that message loss always preceded or coincided with PostgreSQL connection pool exhaustion — when the pool was full, new writes were being dropped silently rather than queued. The 'slowness' was actually timeout behaviour from the same pool exhaustion.

R — Result

Increasing the connection pool size and adding a write queue eliminated both symptoms. Solving the connection pool issue fixed both reported problems simultaneously. Correlation in timing is the first signal that two problems share a root cause.

Story 3: React StrictMode Regression — Two Reported Bugs, One Cause

S — Situation

While working on my React open source contribution, I noticed two separate GitHub issues: one reporting effects firing three times in StrictMode, and another reporting unexpected component remounting in concurrent mode.

T — Task

Both issues had separate reporters and were being tracked independently, but I thought they might share a cause.

A — Action

I read both issue threads carefully and found that both reporters had React 19 with StrictMode enabled. I built a reproduction case that triggered both behaviours simultaneously in the same component tree. Then I git-bisected both to the same commit — a change in how the reconciler scheduled work during StrictMode's double-invoke phase.

R — Result

I linked both issues in GitHub with a comment explaining the shared root cause and a single reproduction case. The maintainers confirmed it was the same regression. My PR targets both fixes together. Finding the common root cause meant one fix instead of two, and a cleaner fix at that.

Stakeholder Management

Behavioral · 2 questions

Q14. Describe a situation where you had to deal with a difficult customer or stakeholder.

Story 1: Hospital Doctors Skeptical of MedStroke AI

S — Situation

During the MedStroke AI hospital pilot in Louisiana, the neurologists were initially skeptical of the AI tool — they felt it was oversimplifying a nuanced clinical judgment and worried it would undermine their authority.

T — Task

I needed to address their concerns honestly without overpromising what the model could do.

A — Action

I met with them in person, listened to every concern without being defensive, and adjusted how I described the tool — not as a diagnostic replacement but as a pattern-recognition aid they could choose to consult or ignore. I also added a confidence score display and made the model's reasoning more visible so they could interrogate it.

R — Result

The doctors' engagement shifted from resistance to active curiosity. The pilot reduced time-to-treatment by 35% because doctors adopted the tool on their own terms. The lesson: technical capability alone doesn't earn stakeholder trust — transparency and listening do.

Story 2: Corporate Sponsorship for NSBE

S — Situation

As NSBE Assistant VP, I approached several tech companies for sponsorships. One company's HR contact was initially dismissive — they'd had poor engagement with student orgs in the past and didn't see the ROI.

T — Task

I needed to change their perception and make the case for partnership without being pushy.

A — Action

I came back with a concrete proposal: specific events where their engineers could interact with our 300+ members, data on our members' GPA distribution and job placement goals, and a cost-per-engagement number that compared favourably to career fair costs. I made it easy for them to say yes.

R — Result

They signed on and we secured \$3K in partnerships total across companies. The contact later told me the proposal stood out because it was specific and ROI-driven rather than generic. That taught me that difficult stakeholders usually have a legitimate reason for their resistance — your job is to understand it.

Story 3: Shifting Requirements at Noon

S — Situation

At Noon, partway through the IoT pipeline build, the client's requirements shifted — they now wanted historical data replay in addition to real-time ingestion, which hadn't been scoped.

T — Task

I needed to manage expectations with both the client and my manager without either overpromising or damaging the relationship.

A — Action

I documented exactly what the new feature would require in terms of schema changes and additional development time, presented two options — full scope with a revised timeline or phased delivery with real-time first — and recommended the phased approach clearly. I let the client choose with full information.

R — Result

They chose phased delivery. Real-time went live on schedule with 40% downtime reduction. The client appreciated being given real options rather than a surprise delay. Managing difficult stakeholders is about giving them honest choices, not just good news.

Q60. Tell me about a time you needed information from someone who wasn't responsive.

Story 1: Unblocking the Ericsson Pipeline Design

S — Situation

At Ericsson, the Kafka microservice I was building required specific event schema documentation from a team that owned an upstream pricing service. The schema would determine how I structured my consumer and transformation logic.

T — Task

The upstream team lead hadn't responded to two Slack messages over a week. I had a hard deadline and couldn't proceed without the schema.

A — Action

Instead of sending a third message, I changed my approach. I looked at the upstream service's existing API documentation and source code to infer the schema myself. I wrote out my best interpretation of it, flagged the three fields I was uncertain about, and sent a concise message to the team lead with a specific ask: 'I've inferred the schema below — can you confirm these three fields are correct?' A yes-or-no confirmation is much easier to give than an open-ended request. I also CC'd my manager so the lead knew there was visibility. I got a response within two hours.

R — Result

The schema was confirmed with one small correction. The pipeline design proceeded without further delay. The lesson: frame your ask as a confirmation, not a request for information from scratch. Most unresponsiveness is friction, not refusal — reduce the friction.

Story 2: The Noon Device Spec Blocker

S — Situation

At Noon, I was building the MQTT ingestion layer for the IoT pipeline and needed detailed documentation on the message format and reconnection behaviour of a specific device type. The hardware vendor was the only source of truth and had been slow to respond for two weeks.

T — Task

I couldn't build reliable reconnection handling without knowing the device's exact timeout and retry behaviour, and the vendor contact wasn't responding to the account team's emails.

A — Action

I took a three-pronged approach. First, I set up a test environment with the actual device and ran my own characterisation tests — deliberately triggering disconnects and observing the reconnection pattern in the packet logs. Second, I searched the vendor's public GitHub repositories and found a sample client implementation that revealed their timeout assumptions. Third, I wrote up what I had learned and sent it to the vendor contact with a specific technical question — not 'can you send me the docs' but 'I observed X timeout behaviour in my tests; is this the specified behaviour or a quirk of this firmware version?' The vendor responded within a day.

R — Result

I had the answer I needed and also had first-hand empirical data to validate it. Waiting for unresponsive stakeholders is often optional — you can frequently make progress by gathering your own evidence and then asking a specific question rather than an open one.

Story 3: Open Source Maintainer Stall

S — Situation
During my pgjdbc open source contribution, a PR I had submitted sat unreviewed for three weeks. The relevant maintainer had responded once to say 'I'll look at this soon' and then went quiet.
T — Task
I needed the feedback to know whether my approach was on the right track, and I had already done substantial follow-up work that depended on the direction.
A — Action
I didn't send repeated pings. Instead, I added a concise PR comment that summarised the status: 'Since my last update I've addressed the edge case in the error handler and added two new tests covering the concurrent path. Happy to adjust the approach if anything here seems off — just wanted to flag the update so it's easier to continue where we left off.' I also posted a brief note in the project's contributor Discord asking if there was a better reviewer to tag for JDBC connection pool changes. A second maintainer picked it up the next day.
R — Result
The PR was reviewed and merged within a week of that update. The key was making it as easy as possible to re-engage: a summary of what had changed, a clear statement of what I needed, and a low-friction alternative path to move forward. Pinging is less effective than making the next step obvious.

S — Situation
MedStroke AI was a full-stack stroke diagnosis system with real-time patient monitoring, role-based access control, NIHSS scoring, automated tPA eligibility, and a Random Forest ML model — built for a hospital pilot.
T — Task
I was responsible for the ML pipeline, the FastAPI backend, and the overall system architecture.
A — Action
I designed the ML pipeline around a Random Forest model trained on blood sample data, implemented the NIHSS scoring logic against clinical guidelines, and built role-based access so neurologists and nurses had appropriate data visibility. I ran the hospital pilot iteratively — shipping, getting feedback, and shipping again within the pilot window.
R — Result
The system reduced time-to-treatment by 35% at the Louisiana hospital pilot. Building a healthcare AI system taught me that technical correctness and clinical trustworthiness are two different bars — you need to clear both.

Story 3: Go Container — Container Runtime from Scratch

S — Situation
I built Go Container, a container runtime in Go, to deeply understand what Docker and similar tools actually do at the Linux kernel level.
T — Task
This meant implementing process isolation from first principles — no containerisation libraries, just the Linux kernel primitives.

Technical

Behavioral · 4 questions

A — Action
I used Linux namespaces to give each container its own process space, filesystem, and hostname. I used cgroups to cap process counts and automatically clean up after container exit. I set up chroot and mounted pseudo-filesystems to create complete filesystem separation from the host. Each layer was tested independently before integration.
R — Result
A functional container runtime that can isolate processes, enforce resource limits, and maintain filesystem separation from the host. Building it from scratch gave me a depth of understanding of Linux internals and containerisation that reading Docker documentation never would have.

Behavioral — Technical
Q27. Tell me about a complex technical project you've worked on.
Story 1: Ericsson Kafka Microservice at Scale
S — Situation
At Ericsson I built a Kafka-streaming microservice deployed on Kubernetes that processed 400K+ daily events across 6 topics, replacing a system of manual SQL workflows.
T — Task
The system needed sub-200ms end-to-end latency, fault tolerance across broker failures, and clean integration with upstream REST APIs and downstream reporting consumers.
A — Action
I designed the consumer group topology to balance partition load, used sticky partition assignment to reduce rebalancing under load, implemented idempotent producers to handle retries without duplicate events, and wrote a load test suite that simulated production volume before launch. I also built a React dashboard to make the pipeline observable.
R — Result
The system processed 400K+ daily events at sub-200ms latency, eliminated manual SQL workflows, and the dashboard cut incident resolution time from 60 to 10 minutes. It's the most production-impactful system I've built.

Story 2: MedStroke AI — Full-Stack Clinical Tool

Behavioral — Technical
Q28. How do you stay up-to-date with the latest technological advancements?
Story 1: Open Source as Active Learning
S — Situation
Reading about new technologies is passive. Contributing to production open source systems is active — you have to understand the code deeply enough to change it correctly.
T — Task
I use open source contributions as my primary method of staying current with how production systems evolve.
A — Action
I actively contribute to facebook/react, apache/kafka, pgjdbc, and kubernetes-client/java. When React 19 shipped reconciler changes, I had to read the source to understand them — not just a blog post. When I merged a PR into pgjdbc, I had to understand PostgreSQL's connection protocol at a byte level.
R — Result
I have PRs merged into systems used by developers at JPMorgan, LinkedIn, and Meta. I learn technologies at the level of production maintainers, not tutorial consumers. That gap matters in interviews and on the job.

Story 2: Building from Scratch to Learn Internals

S — Situation
When I wanted to understand databases and containers deeply, I didn't read documentation — I built NanoSQL and Go Container from scratch.
T — Task
You can't stay current by just reading about technology. You have to build with it.
A — Action
For NanoSQL I implemented B-tree indexing, persistent storage, and custom memory management in C — because understanding SQLite's internals makes me a better database user. For Go Container I implemented Linux namespaces and cgroups directly — because understanding what Docker does at the kernel level makes me a better infrastructure engineer.
R — Result
I now use PostgreSQL, Docker, and Kafka with a depth of understanding that comes from having built equivalent systems. That foundation makes it much faster to learn the next tool in any of those domains.

Story 3: Following Real Codebase Evolution

S — Situation
Most 'stay current' advice involves reading newsletters and watching conference talks. My approach is to follow the actual codebases of major projects through their commit history and RFCs.
T — Task
I want to understand why technology evolves, not just what changed.

A — Action
I followed the React 19 RFC process from proposal to implementation, read the Kafka KIP process for consumer group changes, and subscribe to PostgreSQL mailing list summaries. When I see a change, I read the PR that implemented it — not just the release notes.
R — Result
This gives me about 6–12 months of lead time on where the ecosystem is moving, because RFCs and open issues tell you what's coming before it ships. In my React 19 open source work, I found the StrictMode regression because I was following the reconciler changes closely enough to notice the behaviour had changed.

Behavioral — Technical
Q29. Give an example of a time you had to debug a challenging technical issue.
Story 1: Kafka Consumer Latency at Ericsson
S — Situation
Two weeks before launch at Ericsson, load testing showed p99 latency at 380ms against our 200ms SLA — and the cause wasn't obvious.
T — Task
I needed to find and fix the bottleneck without making random changes that might introduce new issues.
A — Action
I methodically isolated variables. First I ruled out network — inter-broker latency was fine. Then I checked producer throughput — normal. Then I looked at consumer group rebalancing frequency in the broker logs and found it was happening every 15–30 seconds under load. I traced it to our session timeout being too short for the processing time of high-volume batches, triggering false-positive timeouts and rebalances. I switched to sticky partition assignment and tuned the heartbeat interval.
R — Result
Latency dropped to under 180ms. Systematic isolation — ruling out one layer at a time — found the root cause in a day. The lesson: in distributed systems debugging, eliminate entire layers before going deep on any one layer.
Story 2: React 19 StrictMode Regression

S — Situation
While working on my open source React contribution, I noticed that certain component effects were firing three times in StrictMode in React 19, where React 18 fired them twice — a behaviour change not documented in the migration guide.
T — Task
I needed to determine if this was intentional, a regression, or expected new behaviour — with enough evidence to open a credible issue.
A — Action
I git-bisected the React 19 commits to find the exact commit that changed the behaviour. I wrote a minimal reproduction case — 20 lines of code that demonstrated the issue without any other dependencies. I then traced the relevant reconciler code path in the source to understand the mechanism. I opened a detailed issue with the bisection commit, the reproduction case, and the code path.
R — Result
Maintainers confirmed it was an unintentional regression. My PR targeting the fix is active. Rigorous debugging — bisection, minimal reproduction, source tracing — is what separates a credible open source issue from noise.

Story 3: MQTT Intermittent Message Loss at Noon

S — Situation
At Noon, after deploying the IoT pipeline, we started seeing intermittent message loss — maybe 1–2% of sensor readings were missing in PostgreSQL, but we couldn't reproduce it reliably.

T — Task
Intermittent bugs are the hardest kind — the failure doesn't happen when you're watching.
A — Action
I added structured logging at every pipeline stage — MQTT ingestion, transformation, and PostgreSQL insert — with message IDs I could trace end-to-end. I ran the pipeline under sustained load in staging for 24 hours and collected the logs. I found that message loss always correlated with PostgreSQL connection pool exhaustion under burst load — the connection pool was too small for our peak volume.
R — Result
I increased the connection pool size and added a queue buffer in front of the PostgreSQL writer. Message loss dropped to zero in subsequent tests. The lesson: intermittent bugs require observation infrastructure first — you can't debug what you can't see.

Behavioral — Technical
Q33. What is the biggest technical challenge you have worked on?
Story 1: Kafka at 400K Events/Day with Sub-200ms SLA
S — Situation
At Ericsson, the hardest technical challenge was building a Kafka microservice that processed 400K+ daily events across 6 topics with a hard sub-200ms latency requirement — in a production Kubernetes environment I was new to.
T — Task
The challenge wasn't just throughput — it was throughput plus latency plus fault tolerance plus observability, all at once.
A — Action
I front-loaded the riskiest technical work: consumer group configuration and latency profiling. I used sticky partition assignment to prevent unnecessary rebalancing, tuned heartbeat intervals and session timeouts under simulated load, and built load tests that ran at 2x expected volume. I found and fixed the rebalancing bug two weeks before launch.
R — Result
Sub-200ms latency from day one. The challenge taught me that distributed systems performance requires understanding the failure modes first — then designing around them — not optimising after the fact.
Story 2: React 19 Reconciler — Open Source Contribution

S — Situation

Finding and contributing a fix for a React 19 StrictMode regression in the reconciler is the most technically challenging open source work I've done.

T — Task

The React reconciler is among the most complex JavaScript ever written — concurrent rendering, work-in-progress fibers, priority scheduling — and I had to understand it well enough to propose a correct fix.

A — Action

I git-bisected 200+ commits to find the regression, read the fiber reconciler source to understand the StrictMode double-invoke mechanism, wrote a minimal 20-line reproduction case, and proposed a fix with a regression test. I documented every step in the PR description so maintainers could validate my reasoning.

R — Result

Maintainers confirmed the regression and the PR is active. This was the biggest technical challenge not because of the code volume but because of the precision required — a wrong fix in the React reconciler affects every React application in production.

Story 3: Building a Container Runtime from Linux Primitives

S — Situation

Go Container was my most challenging personal project — implementing container isolation from first principles using Linux kernel primitives with no containerisation libraries.

T — Task

I needed to implement the actual isolation mechanisms: process namespace separation, filesystem isolation, and resource limits — without Docker or any abstraction layer.

A — Action

I used Linux clone() with CLONE_NEWPID, CLONE_NEWNS, CLONE_NEWUTS flags for namespace isolation, cgroups v2 for resource controls, and chroot with pseudo-filesystem mounting for filesystem isolation. I read kernel documentation — not Go documentation — to understand the underlying mechanisms. I tested each isolation layer independently.

R — Result

A working container runtime that can isolate processes and enforce resource limits. The challenge was that kernel documentation assumes knowledge of C systems programming, not Go. Building at that level of abstraction means reading primary sources and being comfortable with things that are not in tutorials.